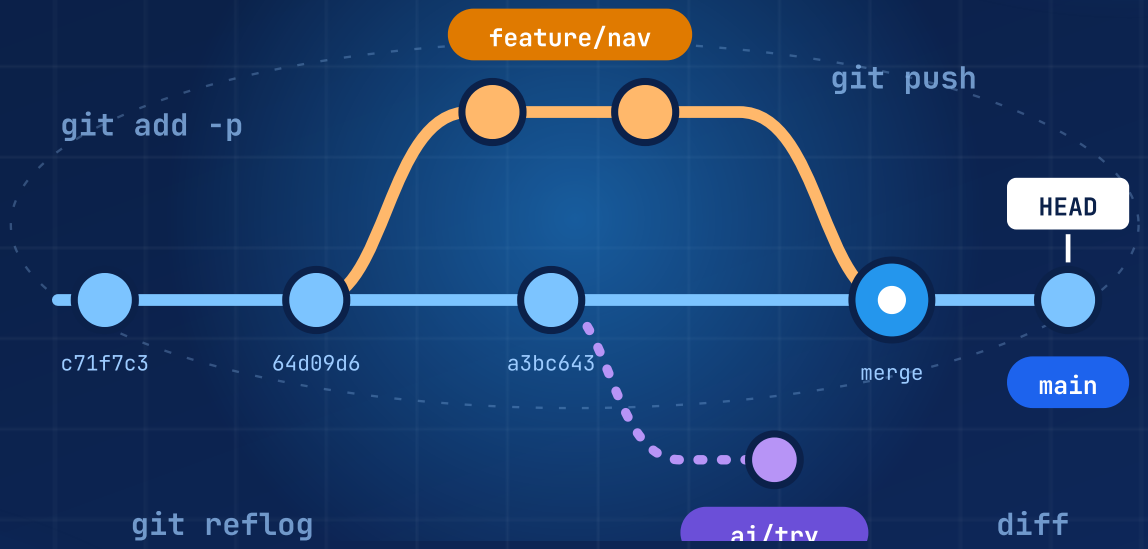
كتاب مذاكرة عملي · بالعامية المصري

Git و GitHub

من أول Commit لحد ما تشتغل مع فريق ومع الـ AI بأمان

ليه Version Control أصلاً، الأماكن التلاتة، الـ Commits والتاريخ، التراجع عن أي غلطة، الـ Branches والـ Merge والـ Conflicts، GitHub والـ Pull Requests، الـ Workflow الصح، والإنقاذ لما الدنيا تبوظ. كل أمر متشرّح حتة حتة.



`git commit -m "feat: add navbar"` `git add -p` `git status`

`git restore --staged .` `git push -u origin main` `git switch -c feature/nav`

`git reflog`

كتابك للمذاكرة

من الصفر خطوة بخطوة

19 فصل + تحديات في كل فصل

+ Cheat Sheet وجدول Flags وجدول رموز

المحتويات

الكتاب ده ماشي معاك بالتدريج على 7 مراحل: من "يعني إيه Version Control أصلاً" لحد ما تشتغل على GitHub مع ناس تانية ومع أدوات الـ AI من غير ما تخاف تبوّظ حاجة. كل مرحلة بتبني على اللي قبلها، وكل أمر متشرّح حتة حتة، والحاجات اللي بتتكرر عليها علامة **حاجة مشتركة** وإحالة لمكانها.

إزاي تستخدم الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **حاجة مشتركة**: Flag أو فكرة بتتكرر في أوامر كتير
- **إحالة**: رقم الجزء والصفحة اللي فيها الشرح الأصلي
- **مشكلة شائعة**: العَرَض والسبب والحل
- **ملخص + صح ولا غلط + أسئلة**: آخر كل فصل
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك
- **لو حابب تتعمق**: تفاصيل زيادة تقدر تعديها أول مرة
- **تشريح الأمر**: كل حتة في الأمر عليها رقم وشرح
- **تحديات الفصل**: سهل / متوسط / صعب، والحلول في الملحق

01 ليه Git أصلاً؟ 7
1.1 المشكلة: الساعة 2 بالليل والكود باظ 7
1.2 يعني إيه Version Control؟ 8
1.3 Git بيفكر إزاي: Snapshots مش فروقات 9
1.4 Git مش GitHub 10
1.5 Git هيفرق معاك إزاي إنت بالذات؟ 11
1.6 ليه من الـ Terminal مش من الأزرار بس؟ 11

02 تسطيب Git وتظبيطه صح 13
2.1 التسطيب على الـ Mac: هل Git عندك أصلاً؟ 13
2.2 git config: 3 مستويات للإعدادات 15
2.3 إنت مين؟ user.name و user.email 15
2.4 اسم الـ Branch الأولاني: main 17
2.5 الـ Editor: افتح VS Code مش Vim 17
2.6 إعدادات تانية مفيدة 18
2.7 إزاي تجيب المساعدة 20

03 أول Repo: الأماكن التلاتة والـ Commit 22
3.1 git init: حوّل فولدر لـ Repo 22
3.2 فولدر .git: المخزن 23
3.3 الأماكن التلاتة 24
3.4 حالات الملف 25
3.5 git status: أهم أمر في Git 25
3.6 git add: حط في الـ Staging 27
3.7 git commit: سجّل اللحظة 28
3.8 مثال كامل: أول مشروع JS 29

04 قراية التاريخ: log و diff و show 32
4.1 git log: شريط التاريخ 32
4.2 إزاي تسمّي أي Commit 34
4.3 git diff: إيه اللي اتغير بالظبط؟ 35
4.4 git show: تفاصيل Commit واحد 38

05 الـ .gitignore: حاجات Git ميشوفهاش 40
5.1 ليه نتجاهل ملفات؟ 40
5.2 قواعد الـ Patterns 41
5.3 قالب جاهز لمشاريع JavaScript 42

5.4 .gitignore عام لكل مشاريعك 42
5.5 git check-ignore: ليه الملف ده متجاهَل؟ 43
5.6 اتضاف بالغلط قبل الـ .gitignore 44
5.7 git rm و git mv 44

06 التراجع عن أي غلطة 46
6.1 قبل أي تراجع: سؤالين 46
6.2 git restore: ارجع ملف في الورشة أو الـ Staging 47
6.3 git commit --amend: صلّح آخر Commit 48
6.4 git reset: رجّع الـ Branch لورا 49
6.5 git revert: الإلغاء الآمن 50
6.6 git clean: الملفات الجديدة Untracked 51
6.7 خريطة قرار التراجع 52

07 الـ HEAD و Branches 54
7.1 المشكلة: عاوز أجرّب من غير ما أبوّظ 54
7.2 الـ Branch = لافتة على Commit 54
7.3 HEAD بيتحرك إزاي 55
7.4 git branch: اعرض واعمل 56
7.5 git switch: اتنقّل بين الـ Branches 56
7.6 الـ Detached HEAD: متتخضش 57
7.7 مسح وتغيير اسم الـ Branches 58

08 الـ Merge وحل الـ Conflicts 61
8.1 git merge: رجّع الشغل للخط الأساسي 61
8.2 النوع 1: Fast-forward 62
8.3 النوع 2: الـ 3-way Merge و Merge Commit 62
8.4 الـ Conflict: ليه بيحصل؟ 63
8.5 حل الـ Conflict خطوة بخطوة 64
8.6 خط الرجوع: git merge --abort 66
8.7 إزاي تقلل الـ Conflicts 66

09 GitHub والـ Remotes والدخول بأمان 68
9.1 الحساب على GitHub 68
9.2 يعني إيه Remote؟ 68
9.3 HTTPS ولا SSH؟ 70
9.4 الطريق 1: مفتاح SSH 70
9.5 الطريق 2: HTTPS على الـ Mac بالـ Keychain 73

10 pull و fetch و push و clone 76
10.1 git clone: هات Repo لأول مرة 76
10.2 git push: ابعت شغلك 77
10.3 origin/main: صورة آخر مرة شفت فيها GitHub 78
10.4 git fetch: هات المعلومات بس 78
10.5 merge + fetch = git pull 79
10.6 ahead و behind و diverged 80
10.7 لما الـ push يترفض 80

11 التعاون: Issues و Forks و Pull Requests 83
11.1 الـ Pull Request: "ممكن تدمجوا شغلي؟" 83
11.2 صفحة الـ PR 84
11.3 الـ Code Review 85
11.4 طرق الدمج على GitHub 86
11.5 الـ Fork: تساهم في مشروع مش بتاعك 87
11.6 الـ Issues: قايمة المهام والمشاكل 89

12 Workflow للمبتدئين: GitHub Flow 91
12.1 ليه Workflow؟ 91
12.2 قواعد GitHub Flow 91
12.3 تسمية الـ Branches 92
12.4 الروتين اليومي 93
12.5 وانت شغال لوحدك؟ 93
12.6 حماية main على GitHub 94

12.7 Workflows تانية هتسمع عنها 94

13 رسايل Commit محترمة و Commits مركّزة 97
13.1 المشكلة: تاريخ مالوش معنى 97
13.2 قواعد الرسالة الكويسة 98
13.3 Conventional Commits 99
13.4 Atomic commits: Commit واحد = تغيير واحد 100
13.5 git add -p: اختار حتت من الملف 100
13.6 صلّحت الرسالة؟ إمتى ينفع 102

14 الـ Stash والـ Tags والـ Releases 104
14.1 المشكلة: لازم أسيب اللي في إيدي دلوقتي 104
14.2 أوامر الـ Stash 105
14.3 الـ Tags: علامة ثابتة على Commit 106
14.4 Semantic Versioning: معنى الأرقام 107
14.5 الـ Releases على GitHub 108

15 الـ Rebase والـ Cherry-pick بهدوء 110
15.1 الحكاية: تاريخ مليان عُقد 110
15.2 الـ Rebase بالرسم 110
15.3 القاعدة الذهبية 112
15.4 Conflict أثناء الـ Rebase 112
15.5 الـ Interactive rebase: نضّف تاريخك 113
15.6 git pull --rebase 114
15.7 git cherry-pick: Commit واحد بعينه 114

16 الـ Reflog شبكة الأمان، و Bisect و Blame 116
16.1 الحكاية: "مسحت شغل يومين" 116
16.2 git reflog: دفتر تحركات HEAD 116
16.3 git bisect: لاقي الـ Commit اللي دخّل الـ Bug 118
16.4 git blame: مين كتب السطر ده؟ 119

17 GitHub كمنصة: README و Pages و Actions و gh 122
17.1 الـ README: واجهة مشروعك 122
17.2 GitHub Pages: موقعك على النت ببلاش 123
17.3 GitHub Actions: روبوت بيشتغل مع كل push 125
17.4 gh: GitHub من الـ Terminal 126

18 Git والـ AI: اشتغل بأمان 129
18.1 الـ AI عدّل 14 ملف 129
18.2 القاعدة الأولى: Commit قبل الـ AI 129
18.3 Branch لكل تجربة 130
18.4 اقرا قبل ما تقبل 131
18.5 الأسرار: القاعدة اللي مفيهاش هزار 132
18.6 المفتاح اتسرّب. اعمل إيه؟ 133

19 الإنقاذ: الغلطات الشائعة وحلها 135
19.1 أول 60 ثانية: متعملش حاجة 135
19.2 غلطات في الـ Commits 135
19.3 "شغلي ضاع!" 137
19.4 محبوس في نص عملية 138
19.5 رسائل الـ push والـ pull 139
19.6 مشاكل الملفات 140

أ Cheat Sheet وجدول الـ Flags وجدول الرموز 143
أ.1 جدول الـ Flags المشتركة 145
أ.2 جدول الرموز 147

ب قاموس المصطلحات 149

ج الإجابات والحلول 152

خريطة الكتاب

بص على الطريق كله مرة واحدة قبل ما تبدأ. متستعجلش: كل مرحلة مبنية على اللي قبلها، والمراحل الأخيرة (Rebase و Reflog وغيرهم) هتبقى سهلة جداً لو الأساس في المرحلة 2 و 3 واضح. الأرقام اللي جوه الدوايز أرقام المراحل، وفوقها أرقام الفصول.



الطريق من أول ما تسمع كلمة Git لحد ما تبقى مرتاح معاه في أي مشروع.

1 · البداية

ليه Version Control موجود أصلاً، والفرق بين Git و GitHub، وتسطيب Git وتظبيطه صح من أول يوم.

2 · الأساس على جهازك

الأماكن التلاتة والـ Commits، قراية التاريخ والفروقات، الـ gitignore.، والتراجع عن أي غلطة.

3 · الـ Branches

يعني إيه Branch و HEAD، والـ Merge بأنواعه، وحل الـ Conflicts خطوة بخطوة.

4 · GitHub والتعاون

الـ Remotes و SSH، و clone و push و pull و fetch، والـ Pull Requests والـ Forks والـ Issues.

5 · الشغل المنظّم

Workflow بسيط للمبتدئين (GitHub Flow)، رسايل Commit محترمة، والـ Stash والـ Tags والـ Releases.

6 · أدوات متقدمة بهدوء

Rebase مقابل Merge و Cherry-pick، والـ Reflog كشبكة أمان، و Bisect و Blame.

7 · GitHub والـ AI والإنقاذ

README و GitHub Pages و Actions و gh، الشغل مع أدوات الـ AI بأمان، وإنقاذ نفسك من أشهر الأخطاء.

Best Practice

متقراش الكتاب وبس: افتح Terminal جنبك واعمل Repo تجارب اسمه `git-playground`، وجرّب كل أمر بإيدك. Git بيتفهم بالصوابع مش بالعين. وخلي الـ **Cheat Sheet** وجدول الـ **Flags** وجدول **الرموز** في الملاحق جنبك وانت بتشتغل.

01

المرحلة 1 · البداية

الفصل الأول

ليه Git أصلاً؟

قبل أي أمر، لازم تحس بالمشكلة اللي Git بيحلها. لو عمرك سميت ملف final_v2_FINAL.js، أو الـ AI غيّر في مشروعك حاجات ومبقتش عارف ترجع، يبقى الفصل ده بتاعك. هنفهم يعني إيه Version Control، Git بيفكر إزاي، والفرق بين Git و GitHub.

بعد الفصل ده هتكون عارف

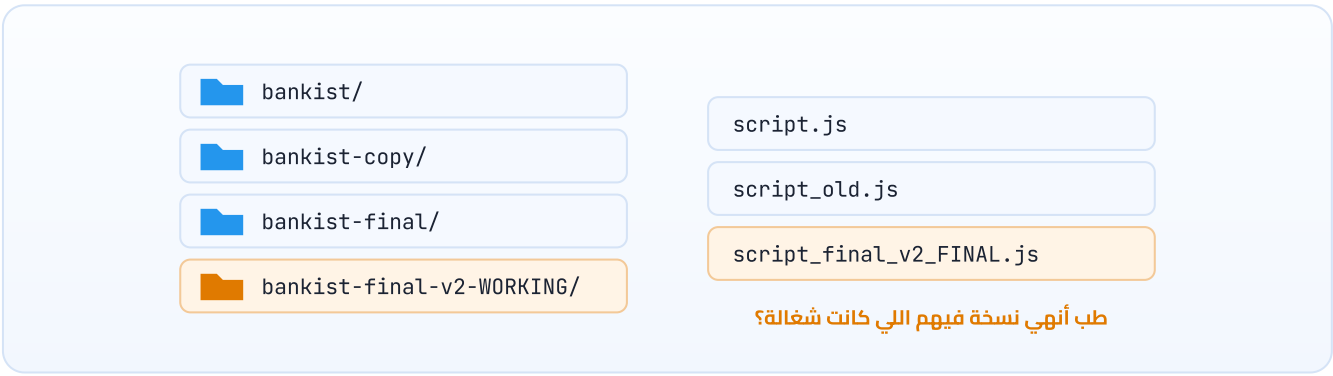
- المشكلة اللي Version Control بيحلها
- الفرق بين الأنظمة المحلية والمركزية والموزّعة
- إن Git بيحفظ Snapshots كاملة مش فروقات
- يعني إيه Commit و Hash بشكل مبدئي
- الفرق بين Git و GitHub
- ليه هتتعلمه من الـ Terminal مش من الأزرار بس

1.1 المشكلة: الساعة 2 بالليل والكود باظ

تخيل إنك ماشي في كورس الـ JavaScript، وبقالك أسبوع بتبني تطبيق البنك الصغير بتاع الكورس. أخيراً الـ Login شغال، والحركات بتظهر، والرصيد بيتحسب صح. إحساس جميل. قلت لنفسك: "طب أخلّي الـ AI يضيف خاصية تحويل الفلوس وأنام". كتبتله الطلب، وفي ثانيتين غيّر في 4 ملفات. شغّلت الصفحة... شاشة بيضا. ولا Login ولا حركات ولا أي حاجة.

طبيعي أول حاجة تعملها `Ctrl+Z`. بس الـ AI كان كاتب في ملفات انت مش فاتحها أصلاً، والـ Undo مش فاكر حاجة. وبتحاول تفتكر الكود كان شكله إيه الصبح... ومش فاكر. والأسوأ: مش عارف هو غيّر إيه بالظبط عشان ترجعه.

لو الموقف ده حصلك (أو هيحصلك، صدقني)، فانت مش لوحدك. وأغلبنا عمل الحل البدائي اللي جه في دماغه بعدها: "خلاص، من النهارده قبل أي تعديل كبير هنسخ الفولدر".



شكل 1.1 "نظام النسخ اليدوي": بعد أسبوعين هيبقى عندك ده، ومحدش عارف إيه الفرق بين نسخة والتانية.

وبعد أسبوعين، يبقى عندك فولدر اسمه `bankist-final`، وجنبه `bankist-final-v2-WORKING`، وجوه واحد فيهم ملف اسمه `script_final_v2_FINAL.js`. وكلمة FINAL دي، زي ما كلنا عارفين، عمرها ما بتبقى الأخيرة.

المشكلة إن الطريقة دي بتحل نص المشكلة بس. عندك نسخة قديمة، تمام، بس **مش عارف إيه اللي اتغير** بينها وبين الحالية غير لما تفتح الملفين جنب بعض وتقارن بعينك سطر سطر. و**مش عارف ليه** اتغير: مفيش ولا ملاحظة بتقول "النسخة دي فيها إصلاح زرار التحويل". وكل نسخة فيها المشروع كله، ومع `node_modules` ممكن الفولدر الواحد يبقى مئات الميجات. ولو حد تاني عاوز يشتغل معاك على نفس المشروع؟ هتبعتوا Zip لبعض على الواتساب وكل واحد يدمج بإيده؟

الناس واجهت المشكلة دي من بدري جداً، قبلنا بكتير، وطلعوا بحل اسمه **Version Control**. خلينا نشوف الفكرة.

1.2 يعني إيه Version Control؟

Version Control System (أو VCS) = برنامج بيسجّل تاريخ مشروعك. كل ما توصل لنقطة كويسة، تقوله "احفظ اللحظة دي"، ومعاها رسالة بتقول عملت إيه. وبعدين تقدر:

- ترجع لأي لحظة قديمة في ثانية.
- تشوف إيه اللي اتغير بالظبط بين أي لحظتين، سطر بسطر.
- تعرف مين غيّر السطر ده، وإمتى، وليه.
- تجرّب فكرة مجنونة في "خط موازي" من غير ما تلمس النسخة الشغالة.

> **التشبيه اللي هنفضل نستخدمه**
>
> فكّر فيه زي الـ **Save Points في لعبة فيديو**. قبل الوحش الكبير بتعمل Save. لو خسرت، بترجع للـ Save مش لأول اللعبة. في Git، الـ Save Point ده اسمه **Commit**.

الحكاية من الأول: 3 أجيال

أول حل ظهر كان بسيط: برنامج على جهازك بيحفظ نسخ قديمة من كل ملف في مكان مخفي. ده أحسن من النسخ اليدوي، بس لسه على جهاز واحد. ولما الفرق كبرت، ظهرت فكرة السيرفر المركزي (زي SVN): التاريخ كله على سيرفر واحد، وكل مبرمج بيسحب منه ويبعتله. حلو، لحد ما السيرفر يقع أو النت يقطع، وساعتها الفريق كله يقعد يشرب شاي لأن محدش يقدر يعمل Commit ولا يشوف التاريخ.

وبعدين جه الجيل التالت، اللي Git منه: **الموزّع**. الفكرة إن كل واحد عنده نسخة **كاملة** من المشروع بالتاريخ كله على جهازه. تشتغل في القطر من غير نت عادي، وكل حاجة سريعة لأنها محلية، ولو السيرفر اتحرق فأي نسخة عند أي حد ترجّعه.



شكل 1.2 من نسخ على جهازك بس، لسيرفر واحد الكل معتمد عليه، لـ Git: كل نسخة فيها التاريخ كله.

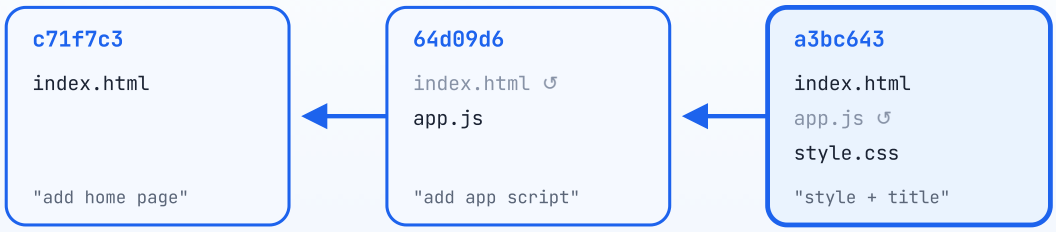
النوع	أمثلة	الميزة	العيب
محلي	RCS، والنسخ اليدوي	بسيط	على جهاز واحد، ومفيش شغل جماعي
مركزي	CVS ،SVN	الفريق كله على سيرفر واحد	من غير إنترنت أو لو السيرفر وقع، مفيش Commits ولا تاريخ
موزّع	**Git**، Mercurial	كل نسخة كاملة، وبتشتغل Offline وسريع جداً	محتاج تفهم فكرة الأماكن التلاتة (الفصل 3)

جدول 1.1 الأجيال التلاتة في جدول (ملخص الحكاية)

> **لو حابب تتعمق: Git اتولد إزاي**
>
> Git اتعمل سنة **2005** على إيد Linus Torvalds (صاحب Linux) عشان يدير كود الـ Linux Kernel، بعد ما الأداة اللي كانوا بيستخدموها اتسحبت منهم. كان عاوزه سريع، وموزّع، ويستحمل آلاف المطورين. ودلوقتي هو الأداة الأساسية في العالم كله تقريباً.

1.3 Git بيفكر إزاي: Snapshots مش فروقات

أهم فكرة في الكتاب كله: كل **Commit** هو **صورة كاملة (Snapshot)** لمشروعك في اللحظة دي، مش مجرد "الفرق عن اللي قبله". والملفات اللي متغيرتش، Git مش بيخزّنها تاني، بيعمل إشارة للنسخة القديمة، فالموضوع خفيف جداً.



كل Commit بيشاور على اللي قبله (الأب)، و ↺ = ملف متغيرش فـ Git بيعيد استخدامه

شكل 1.3 سلسلة Commits: كل واحد صورة كاملة للمشروع + رسالة + بيشاور على أبوه.

كل Commit جواه:

الحتة	يعني إيه	مثال
Snapshot	شكل كل ملفات المشروع في اللحظة دي	index.html و app.js
Message	رسالة إنت بتكتبها: عملت إيه	`feat: add home page`
Author و Date	مين عمله وإمتى	`Sara <sara@example.com>`
Parent	الـ Commit اللي قبله (أبوه)	`c71f7c3`
Hash	بصمة فريدة محسوبة من كل اللي فوق	`64d09d60f21e…`

جدول 1.2 محتويات أي Commit

الـ Hash: بصمة الـ Commit

كل Commit ليه اسم عبارة عن 40 حرف ورقم (Hexadecimal) زي `64d09d60f21e2d9e543ea9afe345c927b6db3c58`. Git بيحسبه من محتوى الـ Commit نفسه، فلو أي حرف اتغير، الـ Hash كله يتغير. وعشان كده التاريخ في Git صعب يتلعب فيه من غير ما يبان. وفي الحياة اليومية بنستخدم أول 7 حروف بس: `64d09d6`.

لو حابب تتعمق: SHA-1 و SHA-256

الـ Hash ده معمول بخوارزمية اسمها SHA-1. Git بيجهّز للتحويل لـ SHA-256 (64 حرف) كافتراضي في **Git 3.0**، اللي لسه منزلش ومالوش ميعاد رسمي لحد أكتوبر 2026 (آخر نسخة مستقرة 2.56). بالنسبالك مفيش أي فرق في الاستخدام.

1.4 Git مش GitHub

دي أكتر لخبطة عند المبتدئين، وطبيعي جداً، لأن الاسمين شبه بعض والناس بتستخدمهم مكان بعض. فكّر فيها كده: **Git** هو الكاميرا اللي في إيدك. بتصوّر بيها لحظات مشروعك (الـ Commits) وبتحتفظ بالصور على جهازك، ومش محتاج نت عشان تصوّر. أما **GitHub** فهو زي ألبوم صور على النت: بترفع عليه الصور عشان تبقى في أمان، وعشان تشاركها مع ناس، وهما يعلّقوا عليها ويضيفوا صورهم.

يعني Git برنامج مجاني ومفتوح المصدر على جهازك، و GitHub موقع (بقى تبع Microsoft من 2018) بيستضيف نسخ من مشاريع Git، وبيضيف حاجات Git نفسه معندوش: Pull Requests للمراجعة، و Issues للمهام، و Pages لنشر المواقع، و Actions للأتمتة. وفيه مواقع تانية بتعمل نفس دور GitHub زي GitLab و Bitbucket، بس Git نفسه واحد في كل الحالات.

	Git	GitHub
هو إيه	أداة (برنامج)	خدمة (موقع)
فين	على جهازك	على النت
محتاج إنترنت؟	لأ	أيوه
بديله	مفيش بديل منتشر بنفس القوة	GitLab و Bitbucket و Codeberg
التشبيه	الكاميرا اللي بتصور	ألبوم الصور على النت اللي بتشاركه

جدول 1.3 Git مقابل GitHub (ملخص)

خلي بالك

ممكن تستخدم Git سنين من غير GitHub خالص. وممكن تستخدم GitHub من غير ما تفهم Git، بس هتتلخبط أول ما حاجة تبوظ. عشان كده الكتاب بيبدأ بـ Git على جهازك (المراحل 2 و 3) وبعدين GitHub (المرحلة 4).

1.5 Git هيفرق معاك إزاي إنت بالذات؟

وانت بتتعلم JavaScript

كل محاضرة أو خطوة في مشروع الكورس = Commit. لو بوّظت حاجة، ترجع. ولو عاوز تقارن الكود بتاعك بكود المحاضرة، `git diff` هيوريك الفرق.

كـ Content Engineer

Git مش للكود بس: أي ملفات نصية (Markdown، HTML، JSON، توثيق) ليها تاريخ ومراجعة. وفرق كتير بتدير المحتوى والتوثيق بـ Git و Pull Requests.

مع الـ AI

الـ AI ممكن يغيّر 10 ملفات في ثانية. لو عامل Commit قبلها، الرجوع أمر واحد. ولو مش عامل، ربنا يسترها. ده موضوع الفصل 18 كله.

للشغل والـ Portfolio

GitHub بقى "السي في" بتاع المبرمج. مشاريعك عليه، ومواقعك تتنشر ببلاش بـ GitHub Pages، وأي شركة هتفترض إنك عارف Git.

1.6 ليه من الـ Terminal مش من الأزرار بس؟

VS Code فيه تاب **Source Control**، وفيه برامج زي GitHub Desktop. وده كويس جداً وهنستخدمهم. بس كل زرار فيهم في الآخر بيشغّل أمر Git. لو فاهم الأوامر:

- هتفهم الزرار بيعمل إيه قبل ما تدوس عليه.
- لما حاجة تبوظ، رسايل الخطأ هتبقى مفهومة، وهي غالباً بتقولك الحل.
- هتعرف تتعامل مع أي سيرفر أو أداة مفيهاش واجهة.
- والـ AI نفسه لما يقترح عليك أمر Git، هتعرف هل هو آمن ولا هيمسح شغلك.

حاجة مشتركة مع كتاب الـ Terminal

كل اللي اتعلمته في كتاب الـ Terminal شغال هنا: `git` برنامج عادي في الـ PATH، والكلمة اللي بعده (`status` و `add` و `commit`) اسمها **Subcommand**، وبعدها Options و Arguments بنفس القواعد بالظبط (فصل "تشريح أي أمر"). `-m` قصيرة، `--message` طويلة، و `--` معناها "خلصت الـ Options". هنفكّر بالقواعد دي في كل فصل.

تشريح الأمر

```
1   2      3           4
git commit --message "feat: add home page"
```

1. `git` البرنامج نفسه (الأمر الرئيسي).
2. `commit` الـ Subcommand: "اعمل Save Point". Git فيه عشرات الـ Subcommands، وكل واحد ليه Options خاصة بيه.
3. `--message` Option طويلة معناها "دي رسالة الـ Commit"، واختصارها `-m`.
4. `"feat: add home page"` قيمة الـ Option. الـ Quotes عشان الـ Shell يعتبر الجملة كلها كلمة واحدة رغم المسافات (زي ما شرحنا الـ Quoting في كتاب الـ Terminal).

ملخص الفصل

- Version Control = تاريخ لمشروعك: ترجع لأي لحظة، وتشوف إيه اتغير ومين وليه.
- Git نظام موزّع: كل نسخة فيها التاريخ كامل، وبيشتغل من غير إنترنت.
- الـ Commit = Snapshot كاملة + رسالة + كاتب + تاريخ + أب، وليه Hash فريد.
- Git برنامج على جهازك، و GitHub موقع بيستضيف الـ Repos ويضيف أدوات تعاون.
- هنتعلم من الـ Terminal عشان نفهم الأزرار، مش عشان نستغنى عنها.

صح ولا غلط؟

1. Git محتاج إنترنت عشان تعمل Commit. صح / غلط
2. كل Commit بيخزن الفرق عن اللي قبله بس. صح / غلط
3. GitHub هو نفس Git بس على النت. صح / غلط

أسئلة مراجعة

1. اذكر 3 مشاكل في طريقة "نسخ الفولدر قبل أي تعديل".
2. إيه ميزة النظام الموزّع على المركزي؟
3. إيه 5 حاجات موجودة جوه كل Commit؟
4. ليه لو حرف واحد اتغير في Commit قديم، الـ Hash بتاعه يتغير؟ وده بيفيد في إيه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح الـ Terminal واكتب `git --version`. لو طلعلك رقم، Git متسطب. لو طلعلك `command not found`، الفصل الجاي هيحلها.
2. متوسط دوّر على جهازك على أي ملفات أو فولدرات بأسماء زي `copy` أو `final` أو `old` (افتكر `find . -iname "*final*"` من كتاب الـ Terminal). دي المشاريع اللي محتاجة Git.
3. صعب افتح أي Repo مشهور على GitHub (زي مشروع مفتوح المصدر بتستخدمه)، ودوّر على عدد الـ Commits والـ Contributors. اكتب جملة: ليه مشروع زي ده مستحيل يتعمل بالنسخ اليدوي؟

02

المرحلة 1 · البداية

الفصل التاني

تسطيب Git وتظبيطه صح

قبل أول Commit، Git محتاج يعرف إنت مين، والـ Branch الأولاني اسمه إيه، ويفتحلك أنهي Editor لما يحتاج منك تكتب رسالة. الإعدادات دي بتتعمل مرة واحدة في العمر، ولو اتعملت غلط هتفضل تزعجك. خلينا نسطّب ونظبط صح من أول يوم.

بعد الفصل ده هتكون عارف

- تسطيب Git على الـ Mac بـ Homebrew والفرق بينه وبين نسخة Apple
- مستويات الإعداد التلاتة: system و global و local
- ضبط user.name و user.email والإيميل الخاص بتاع GitHub
- ضبط اسم الـ Branch الافتراضي main
- ضبط الـ Editor (VS Code) والخروج من Vim لو اتحبست فيه
- إعدادات مفيدة تانية، وإزاي تجيب المساعدة لأي أمر

2.1 التسطيب على الـ Mac: هل Git عندك أصلاً؟

فيه احتمال كبير إن Git موجود على الـ Mac بتاعك من غير ما تعرف. Apple بتحطه ضمن حاجة اسمها **Command Line Tools** (أدوات المطورين). فقبل ما نسطّب أي حاجة، خلينا نسأل الـ Terminal (الـ zsh اللي اتعودت عليه من كتاب الـ Terminal):

تشريح الأمر

```
git --version
```

1. `git` البرنامج.
2. `--version` Option طويلة (زي ما اتعلمنا، الـ `--` قبل كلمة كاملة). بتطبع رقم النسخة وتخرج.

هيحصل واحد من 3 حاجات. لو طلعلك رقم زي `git version 2.50.1 (Apple Git-155)`، يبقى Git موجود، والكلمة `Apple Git` معناها إنه نسخة Apple اللي جاية مع Command Line Tools، وغالباً أقدم شوية من آخر نسخة. ولو طلعلك شباك بيقولك إن الأمر محتاج Command Line Developer Tools ويعرض تسطّبها، دوس **Install** واستنى كام دقيقة، وده نفس اللي بيعمله الأمر `xcode-select --install`. ولو طلع `command not found` (نادر على الـ Mac)، يبقى نسطّبه بنفسنا.

الطريقة اللي أنصحك بيها: تسطّب Git من **Homebrew**، مدير الحزم اللي شفناه في كتاب الـ Terminal. ميزته إنك بتاخد آخر نسخة، وبتحدّثها بأمر واحد مع باقي أدواتك:

تشريح الأمر

```
brew install git
 1    2     3
```

1. `brew` — Homebrew: مدير الحزم على macOS.
2. `install` — سطّب.
3. `git` — اسم الحزمة (اسمها في Homebrew اسمه `Formula`).

```
Terminal
you@laptop:~/portfolio$ brew install git
you@laptop:~/portfolio$ which -a git
/opt/homebrew/bin/git
/usr/bin/git
you@laptop:~/portfolio$ git --version
git version 2.56.0
```

لاحظ إن `which -a` لقى نسختين: نسخة Homebrew في `/opt/homebrew/bin` (على أجهزة Apple Silicon، وعلى Intel بتبقى في `/usr/local/bin`)، ونسخة Apple في `/usr/bin`. والـ Shell بيشغّل أول واحدة يلاقيها في الـ `PATH`، وده نفس الكلام اللي شرحناه في أول فصل في كتاب الـ Terminal. لو `git --version` لسه بيقول `Apple Git` بعد التسطيب، يبقى `/opt/homebrew/bin` مش قبل `/usr/bin` في الـ PATH: اتأكد إن السطر `eval "$(/opt/homebrew/bin/brew shellenv)"` موجود في `~/.zprofile`، وافتح Terminal جديد.

حدّث Git كل فترة

`brew upgrade git` بيجيب آخر نسخة. Git بيطلع نسخة جديدة كل 3 شهور تقريباً، وفيها تحسينات وإصلاحات أمنية.

النظام	الأمر	ملاحظة
macOS	`brew install git`	المفضّل، أو `xcode-select --install` لنسخة Apple
Windows	`winget install --id Git.Git -e --source winget`	بييجي معاه Git Bash و Credential Manager
Debian / Ubuntu	`sudo apt install git`	بعد `sudo apt update`
Fedora	`sudo dnf install git`	

جدول 2.1 لو هتسطّب على جهاز تاني (للمرجع)

لو حابب تتعمق: آخر نسخة وأرقامها

آخر نسخة مستقرة من Git وقت كتابة الكتاب (أكتوبر 2026) هي **2.56.0**، ونزلت 28 سبتمبر 2026. Git ماشي على أرقام `2.x` من سنة 2014، وكل نسخة Minor جديدة كل حوالي 12 أسبوع. أي نسخة 2.40 أو أحدث فيها كل اللي في الكتاب، والأوامر `git switch` و `git restore` محتاجة 2.23 على الأقل (2019). ونسخة Apple بتتأخر عن الأصلية كام شهر، وعشان كده بنفضّل Homebrew.

2.2 git config: 3 مستويات للإعدادات

قبل ما نكتب أول إعداد، محتاجين نفهم حاجة صغيرة: الإعدادات دي بتتحفظ فين؟ تخيل إنك بتشتغل على مشاريعك الشخصية بإيميلك العادي، وفي يوم بدأت تشتغل على مشروع للشغل، وعاوز الـ Commits فيه تبقى بإيميل الشركة. لو الإعداد مكان واحد، هتفضل تغيّره رايح جاي. عشان كده Git بيقرا إعداداته من 3 أماكن متداخلة، زي الدوايز جوه بعض، والأقرب للمشروع هو اللي بيكسب:



شكل 2.1 مستويات الإعداد: المشروع جوه المستخدم جوه الجهاز.

المستوى	الـ Flag	الملف	بيأثر على
System	`--system`	`/etc/gitconfig`	كل المستخدمين على الجهاز (نادراً ما بنلمسه)
Global	`--global`	`~/.gitconfig`	إنت، في كل الـ Repos. **ده اللي هنستخدمه دلوقتي**
Local	`--local` (الافتراضي)	`.git/config`	Repo واحد بس

جدول 2.2 مستويات git config

2.3 إنت مين؟ user.name و user.email

فاكر لما قلنا إن كل Commit جواه مين عمله؟ الاسم والإيميل دول Git بياخدهم من الإعدادات. ولو معندوش، هيرفض يعمل Commit خالص، أو أسوأ: هيخمّن حاجة زي `sara@Saras-MacBook-Air.local`، وتلاقي تاريخ مشروعك على GitHub باسم غريب ومن غير صورتك. فأول إعدادين في حياتك:

تشريح الأمر

```
git config --global user.name "Sara Ahmed"
```

1. `git config` الـ Subcommand اللي بيقرا ويكتب الإعدادات.

2 `--global` اكتب الإعداد في `~/.gitconfig`، فيتطبق على كل مشاريعك.

3 `user.name` اسم الإعداد: `section.key`. هنا الـ Section اسمه `user` والمفتاح `name`.

4 `"Sara Ahmed"` القيمة. الـ Quotes لازمة عشان فيه مسافة.

تشريح الأمر

```
git config --global user.email "sara@example.com"
```

1 `git config --global` نفس الكلام: إعداد عام.

2 `user.email` الإيميل اللي هيتكتب في كل Commit. GitHub بيستخدمه يربط الـ Commit بحسابك ويحط صورتك جنبه.

3 `"sara@example.com"` القيمة.

خصوصية: إيميل noreply بتاع GitHub

الإيميل ده ببقى **علني** في تاريخ أي Repo public. لو مش عاوز إيميلك الحقيقي يظهر، GitHub بيديك إيميل خاص من **Settings ← Emails** شكله `12345678+sara@users.noreply.github.com`، استخدمه في `user.email`، وفعّل "Keep my email address private".

عشان تتأكد:

```
Terminal
you@laptop:~/portfolio$ git config user.name
Sara Ahmed
you@laptop:~/portfolio$ git config --list --show-origin
file:/home/sara/.gitconfig    user.name=Sara Ahmed
file:/home/sara/.gitconfig    user.email=sara@example.com
```

تشريح الأمر

```
git config --list --show-origin
```

1 `git config` الإعدادات.

2 `--list` اعرض كل الإعدادات اللي Git شايفها دلوقتي (من التلات مستويات). اختصارها `-l`.

3 `--show-origin` جنب كل إعداد، اكتب جاي من أنهي ملف. مفيد جداً لما إعداد "مش راضي يتغير" وتكتشف إنه متعرّف في مكان تاني.

حاجة مشتركة: أمر واحد يقرا ويكتب

`git config key` من غير قيمة = **اقرا**. و `git config key value` = **اكتب**. نفس الفكرة هتشوفها في `git branch` (من غير اسم يعرض، وباسم يعمل) و `git remote` و `git tag` و `git stash` : الأمر من غير Arguments بيعرض، ومعاه بيعمل.

2.4 اسم الـ Branch الأولاني: main

لسه مشرحناش الـ Branches (الفصل 7)، بس كفاية دلوقتي تعرف إن كل Repo بيبدأ بـ Branch واحد. زمان اسمه الافتراضي كان `master`، ودلوقتي GitHub ومعظم الأدوات بتستخدم `main` (GitHub غيّره من أكتوبر 2020). خلي Git على جهازك يمشي على نفس الاسم:

تشريح الأمر

```
git config --global init.defaultBranch main
```

1. `git config --global` إعداد عام.
2. `init.defaultBranch` الإعداد اللي `git init` بيقراه عشان يعرف يسمي أول Branch إيه. Section `init` ومفتاح `defaultBranch` (الإعدادات مش حساسة لحالة الحروف، بس بنكتبها كده عشان تتقري).
3. `main` الاسم.

دقة: main و Git 3.0

من غير الإعداد ده، Git 2.x لسه بيسمي أول Branch `master` وبيطبعلك تنبيه. Git أعلن إن **Git 3.0** هيخلي `main` هو الافتراضي، بس Git 3.0 لسه منزلش. فالإعداد ده لسه مهم.

2.5 الـ Editor: افتح VS Code مش Vim

فيه نكتة مشهورة بين المبرمجين: "إزاي تولّد كلام عشوائي؟ حط مبتدئ قدام Vim واطلب منه يخرج". الموضوع ده هيحصلك أول مرة تكتب `git commit` من غير `-m`: Git بيحتاج منك تكتب رسالة، فبيفتح الـ Editor الافتراضي، وعلى الـ Mac ده غالباً **Vim**. وتلاقي شاشة سودا غريبة، والكتابة مش بتظهر، و `Ctrl+C` مش بيعمل حاجة. خلينا نتجنب الموقف ده من الأول، ونخلي Git يفتح VS Code:

تشريح الأمر

```
git config --global core.editor "code --wait"
```

1. `git config --global` إعداد عام.
2. `core.editor` البرنامج اللي Git يفتحه لما يحتاج منك نص.
3. `"code --wait"` الأمر بتاع VS Code. على الـ Mac لازم تحطه في الـ PATH مرة واحدة: افتح VS Code، و `Cmd+Shift+P`، واكتب `Shell Command: Install 'code' command in PATH`. و `--wait` أهم حتة: بتخلي الـ Terminal **يستنى** لحد ما تقفل التاب. من غيرها Git هيفتكر إنك خلصت وانت لسه بتكتب.

الـ Editor	القيمة
VS Code	`"code --wait"`
Cursor	`"cursor --wait"`
nano (بسيط جوه الـ Terminal)	`nano`
Vim	`vim`

جدول 2.3 قيم core.editor الشائعة

مشكلة 1 اتحبست في Vim ومش عارف أخرج!

عملت `git commit` من غير `-m` ولقيت شاشة غريبة ومفيش حاجة بتتكتب.

الحل: دوس `Esc` (عشان تخرج لوضع الأوامر)، وبعدين: `i` عشان تبدأ تكتب الرسالة، وبعد ما تخلص `Esc` ثم `:wq` و Enter = Write و Quit (احفظ واخرج). ولو عاوز تلغي الـ Commit: `Esc` ثم `:q!` = اخرج من غير حفظ. وبعدها اضبط `core.editor` عشان متتحبسش تاني.

2.6 إعدادات تانية مفيدة

نهايات السطور (Windows مقابل Mac/Linux)

Windows بينهي السطر بحرفين (`CRLF`)، و macOS و Linux بحرف واحد (`LF`). لو فريق فيه الاتنين، كل ملف هيبان "متغير بالكامل" من غير سبب.

انت على Mac، فالافتراضي عندك LF وكله تمام. بس لو زميل على Windows ابعتلك ملف بـ CRLF، الإعداد ده بيخلي Git يحوّله لـ LF وهو داخل الـ Repo:

تشريح الأمر

```
git config --global core.autocrlf input
```

1 `core.autocrlf` تحويل نهايات السطور تلقائياً.

2 `input` على **macOS و Linux**: حوّل CRLF لـ LF وانت بتحفظ في Git بس، ومتلمسش الملفات وهي طالعة. (وعلى Windows القيمة المناسبة `true` : حوّل لـ LF وانت داخل، ورجّعها CRLF وانت طالع).

الأحسن: .gitattributes في المشروع

الحل اللي بيمشي على الفريق كله مهما كانت إعدادات كل واحد: ملف اسمه `.gitattributes` في المشروع فيه سطر `* text=auto` . Git بيتعامل مع كل الملفات النصية صح تلقائياً.

سلوك git pull

هنفهم `pull` في الفصل 10، بس Git الحديث بيرفض يعمل `pull` لو الفروع اتفرعت ومقلتلوش يعمل إيه. اختار الأبسط للمبتدئين:

تشريح الأمر

```
git config --global pull.rebase false
```

1 `pull.rebase` لما `git pull` يلاقي شغل جديد عندك وعلى السيرفر، يدمجهم إزاي؟

2 `false` بالـ Merge (الطريقة الأوضح). هنشوف `true` (Rebase) في الفصل 15.

اختصارات (Aliases)

تشريح الأمر

```
git config --global alias.lg "log --oneline --graph --all"
```

1 `alias.lg` اعمل Subcommand جديد اسمه `lg` . زي الـ `alias` في الـ Shell اللي اتعلمناه في كتاب الـ Terminal، بس جوه Git.

2 `"log --oneline --graph --all"` اللي هيتنفذ لما تكتب `git lg` . (هنشرح كل Flag هنا في الفصل 4).

ملف ~/.gitconfig بعد كل ده

كل الأوامر اللي فوق كتبت في ملف نصي عادي. تقدر تفتحه بـ `git config --global --edit` :

~/.gitconfig

1 اسمك وإيميلك (هنا إيميل GitHub الخاص). الـ `[user]` هو الـ Section، و `name` و `email` المفاتيح.	`[user]`
	`  name = Sara Ahmed`
	`  email = 12345678+sara@users.noreply.github.com`

2 أي `git init` يبدأ بـ Branch اسمه main.	`[init]` `  defaultBranch = main`

3 افتح VS Code واستنى، وظبّط نهايات السطور (input على Mac/Linux).	`[core]`
	`  editor = code --wait`
	`  autocrlf = input`

4 pull يدمج بالـ Merge.	`[pull]` `  rebase = false`

5 اختصاراتك: `git lg` و `git st`.	`[alias]`
	`  lg = log --oneline --graph --all`
	`  st = status -sb`

لو حابب تتعمق: صيغة git config الجديدة

من Git 2.46 (يوليو 2024)، `git config` بقى ليه Subcommands واضحة: `git config set --global user.name "Sara"` و `git config get user.name` و `git config list`. الصيغة القديمة اللي في الكتاب لسه شغالة عادي، وهي اللي هتلاقيها في أغلب الشروحات.

2.7 إزاي تجيب المساعدة

`git help commit`	الشرح الكامل لـ `commit` (نفس `man git-commit`، و `q` للخروج زي `less`).
`git commit --help`	نفس اللي فوق.
`git commit -h`	ملخص سريع للـ Options في الـ Terminal نفسه (أسرع حاجة لما تنسى Flag).
`git help -g`	قايمة بـ Guides مفيدة، زي `git help everyday`.

حاجة مشتركة: -h و --help

زي ما اتعلمنا في كتاب الـ Terminal: `-h` = ملخص قصير، و `--help` = الدليل الكامل. نفس القاعدة في كل Subcommands بتاعة Git.

رسايل Git بتقولك الحل

أهم عادة هتتعلمها: **اقرا رسايل Git**. معظمها فيها سطر بيبدأ بـ `(use "git ..." to ...)` أو `hint:` بيقولك الأمر اللي محتاجه بالظبط. هنشوف ده في كل فصل.

ملخص الفصل

- `git --version` للتأكد، و `brew install git` على الـ Mac، و `which -a git` يوريك أنهي نسخة بتشتغل.
- الإعدادات على 3 مستويات: system و global و local، والأقرب بيكسب.
- أول 3 إعدادات: `user.name` و `user.email` و `init.defaultBranch main`.
- `core.editor "code --wait"` عشان متتحبسش في Vim (ولو اتحبست: `Esc` ثم `:wq`).
- `git config --list --show-origin` يوريك كل إعداد جاي منين، و `-h` / `--help` للمساعدة.

صح ولا غلط؟

1. الإعداد اللي بـ `--local` بيغلب اللي بـ `--global`. صح / غلط
2. من غير `--wait`، الـ `code` هيشتغل كـ Editor لـ Git عادي. صح / غلط
3. الإيميل في الـ Commits يبقى ظاهر لأي حد في الـ Repos الـ public. صح / غلط

أسئلة مراجعة

1. عاوز إيميل الشغل يظهر في Repo الشغل بس، وإيميلك الشخصي في الباقي. تعمل إيه؟
2. ليه بنضبط `init.defaultBranch` لـ main؟
3. إيه الفرق بين `git commit -h` و `git commit --help`؟
4. إعداد مش راضي يتغير رغم إنك غيرته بـ `--global`. إيه الأمر اللي هيكشف السبب؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اضبط الإعدادات الأساسية التلاتة وبعدين اعرضهم بـ `git config --global --list`.
2. متوسط اضبط `core.editor` للـ Editor بتاعك، وجرّب `git config --global --edit`. لازم الملف يتفتح في الـ Editor والـ Terminal يستنى.
3. صعب اعمل 2 Aliases: `git st` لـ `status -sb`، و `git last` يعرض آخر Commit (`log -1 HEAD --stat`). جرّبهم بعد ما تعمل أول Repo في الفصل الجاي.

03

المرحلة 2 · الأساس على جهازك

الفصل التالت

أول Repo: الأماكن التلاتة والـ Commit

ده أهم فصل في الكتاب. لو فهمت الأماكن التلاتة (Working Directory و Staging Area و Repository)، كل أوامر Git بعد كده هتبقى منطقية. هنعمل أول Repo، ونمشي ملف من أول ما يتعمل لحد ما يدخل التاريخ، ونقرا رسايل git status كلمة كلمة.

بعد الفصل ده هتكون عارف

- ☑ git init وإيه اللي جوه فولدر .git.
- ☑ الأماكن التلاتة وليه Git محتاج Staging Area
- ☑ حالات الملف: untracked و modified و staged و committed
- ☑ قراية git status بالتفصيل والنسخة المختصرة -s
- ☑ git add بكل أشكاله و git commit بكل أشكاله
- ☑ إمتى تعمل Commit وقد إيه يبقى حجمه

3.1 git init: حوّل فولدر لـ Repo

خلينا نبدأ من الصفر خالص، بمشروع Portfolio صغير هتعمله وانت بتتعلم: صفحة `index.html` وملف `script.js`. لحد دلوقتي ده فولدر عادي جداً، زيه زي فولدر الصور. Git مش شايفه ومش مهتم بيه. عشان Git يبدأ يسجّل تاريخ الفولدر ده، لازم تقوله صراحة: "الفولدر ده بقى مشروع، تابعه". والفولدر بعد كده اسمه **Repository** (أو Repo اختصاراً): فولدر مشروعك + التاريخ بتاعه.

Terminal

```
you@laptop:~/portfolio$ mkdir portfolio && cd portfolio
you@laptop:~/portfolio$ git init
Initialized empty Git repository in /home/sara/portfolio/.git/
```

تشريح الأمر

```
mkdir portfolio && cd portfolio && git init
  1        2        3              4
```

1. `mkdir portfolio` — اعمل فولدر جديد (من كتاب الـ Terminal).
2. `&&` — كمّل بالأمر اللي بعده **لو** اللي قبله نجح.
3. `cd portfolio` — ادخل الفولدر.
4. `git init` — INITialize: اعمل Repo فاضي هنا. عملياً بيعمل فولدر مخفي اسمه `.git` وبس. ملفاتك مبتتلمسش.

git init باسم فولدر

`git init my-app` بيعمل الفولدر `my-app` ويحوّله Repo في خطوة واحدة. ولو عندك مشروع موجود فعلاً، ادخله واعمل `git init` عادي: الملفات هتفضل زي ما هي، بس Git لسه مش بيتابعها (هتظهر Untracked).

متعملش git init جوه Repo تاني

أشهر غلطة: تعمل `git init` في الـ Home كله (`~`) أو في فولدر فوق مشاريعك، فيبقى عندك Repo جوه Repo. قبل `git init` اتأكد بـ `pwd` إنت فين، وبـ `git status` إن الفولدر مش Repo أصلاً (المفروض يقولك `not a git repository`).

3.2 فولدر .git: المخزن

الـ `.git` مخفي لأن اسمه بيبدأ بنقطة (افتكر `ls -a`). ده المكان اللي فيه **كل** التاريخ:

```
Terminal
you@laptop:~/portfolio$ ls -a
.  ..  .git
you@laptop:~/portfolio$ ls .git
HEAD  config  description  hooks  info  objects  refs
```

الحاجة	فيها إيه
`objects/`	كل الـ Snapshots والملفات والـ Commits، متخزنة بالـ Hash بتاعها
`refs/`	أسماء الـ Branches والـ Tags (كل واحد ملف فيه Hash)
`HEAD`	إنت واقف فين دلوقتي (الفصل 7)
`config`	إعدادات الـ Repo ده بس (الـ `--local`)
`index`	الـ Staging Area (بيظهر بعد أول `git add`)

جدول 3.1 أهم حاجات جوه .git.

لو حابب تتعمق: جوه objects فيه إيه؟

Git في الحقيقة "قاعدة بيانات" بسيطة جداً: كل حاجة متخزنة كـ **Object** اسمه هو الـ Hash بتاع محتواه. وفيه 4 أنواع: **blob** = محتوى ملف واحد (من غير اسمه)، و **tree** = فولدر: قايمة أسماء ملفات وكل اسم بيشاور على blob أو tree، و **commit** = بيشاور على tree الجذر + الأب + الكاتب + الرسالة، و **tag** = للـ Annotated tags (الفصل 14). تقدر تتفرج بنفسك:

```
Terminal
you@laptop:~/portfolio$ git cat-file -t HEAD
commit
you@laptop:~/portfolio$ git cat-file -p HEAD
tree 7d2f9a3c1b...
parent c71f7c3e71...
author Sara <sara@example.com> 1792483200 +0300
committer Sara <sara@example.com> 1792483200 +0300

feat: add app script
you@laptop:~/portfolio$ git cat-file -p 7d2f9a3
100644 blob d914c60...  app.js
100644 blob 986a4a1...  index.html
```

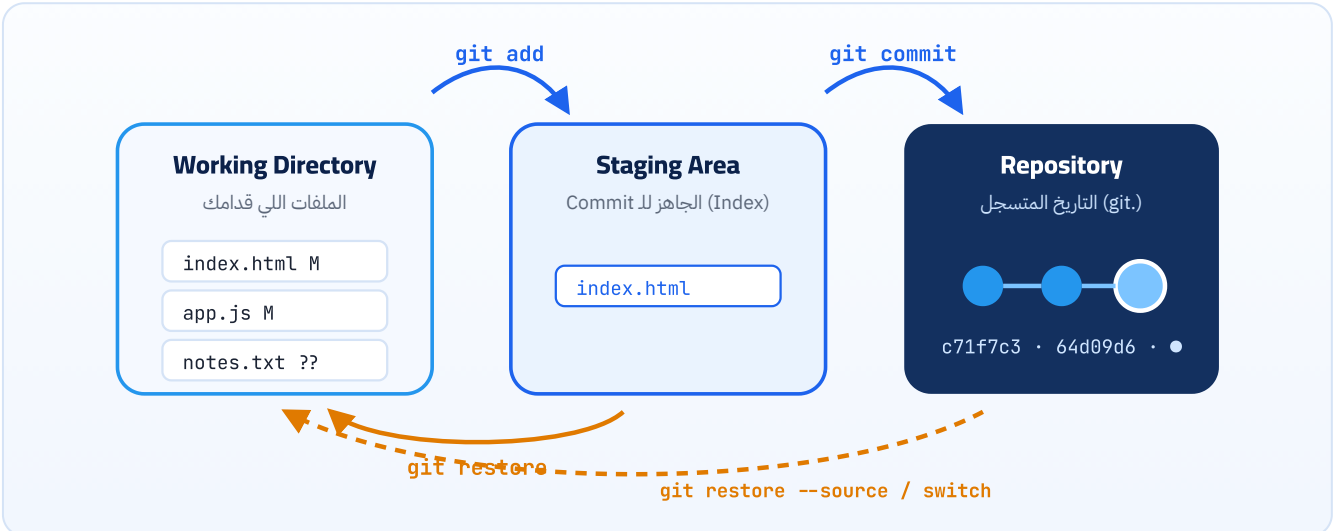
الـ `-t` = Type (نوع الـ Object)، و `-p` = Pretty-print (اعرض المحتوى بشكل مقروء). وعشان كده الـ Branch مجرد ملف فيه Hash commit، والـ commit بيشاور على tree، والـ tree على blobs. ولو ملفين محتواهم واحد، هيبقوا blob واحد. والأرقام `1792483200` دي الوقت بالثواني من 1970 (Unix time).

خلي بالك

متعدلش حاجة جوه `.git` بإيدك. ولو مسحت فولدر `.git`، الملفات بتاعتك هتفضل، بس **التاريخ كله هيضيع** والفولدر يرجع فولدر عادي. وده برضه الطريقة لو عاوز "تلغي" Repo عملته بالغلط: `rm -rf .git` (بحذر، زي ما اتعلمنا في فصل الأوامر الخطيرة).

3.3 الأماكن التلاتة

هنا بقى الفكرة اللي بتفرق Git عن "Save" العادي، واللي لو فهمتها كويس نص الكتاب هيبقى سهل. لما بتعمل Save في VS Code، الملف بيتحفظ على الديسك وخلاص. لكن Git مش بيسجّل كل Save. Git بيستناك انت تقوله "اللحظة دي تستاهل تتسجّل"، وبيسيبك كمان تختار **أنهي تعديلات** بالظبط تدخل اللحظة دي. وعشان كده ملفاتك بتعدي على 3 أماكن:



شكل 3.1 الأماكن التلاتة: بتعدّل في الـ Working Directory، تختار بـ git add، وتسجّل بـ git commit.

1 · Working Directory

الفولدر اللي إنت شايفه وبتعدّل فيه في VS Code. ده "ورشة الشغل".

2 · Staging Area

منطقة تحضير: بتحط فيها التغييرات اللي **عاوزها** في الـ Commit الجاي بس. اسمها التقني **Index**.

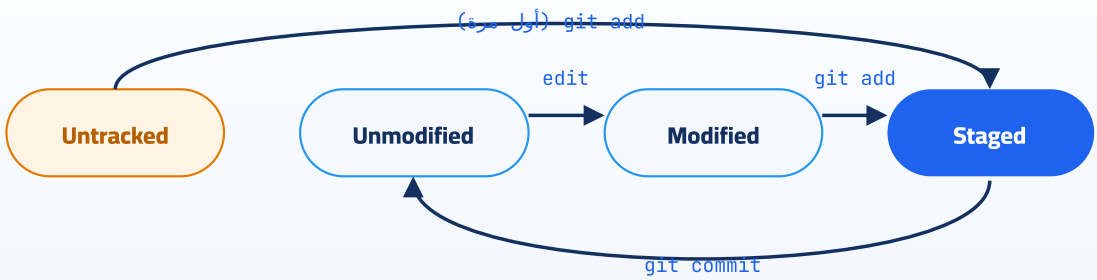
3 · Repository

التاريخ المتسجل جوه `.git`: كل Commit عملته. ده "الأرشيف".

ليه Staging Area؟ مش كفاية Save؟

تخيل إنك صلّحت Bug في `app.js`، وفي نفس الوقت بدأت تجرّب لون جديد في `style.css` ولسه مش متأكد منه. الـ Staging بيخليك تعمل Commit لإصلاح الـ Bug **لوحده**، والتجربة تفضل في الورشة. يعني كل Commit يحكي حكاية واحدة. فكّر فيها زي التصوير: الـ Staging = الناس اللي واقفين قدام الكاميرا، والـ Commit = لحظة الضغط على الزرار.

3.4 حالات الملف



شكل 3.2 دورة حياة الملف: يتعمل (Untracked)، تضيفه، تعدّله، تجهّزه، تسجّله، ويرجع Unmodified.

الحالة	يعني إيه	بيظهر في status تحت
Untracked	ملف جديد، Git عمره ما سجّله	`Untracked files`
Unmodified	زي آخر Commit بالظبط	مبيظهرش خالص
Modified	اتغير عن آخر Commit ولسه مش في الـ Staging	`Changes not staged for commit`
Staged	متحضّر للـ Commit الجاي	`Changes to be committed`

جدول 3.2 حالات الملف وإزاي تعرفها

3.5 git status: أهم أمر في Git

لو هتحفظ أمر واحد بس من الكتاب كله، احفظ ده. `git status` زي ما تسأل Git "إحنا فين دلوقتي؟"، وهو بيرد بكل أدب: إنت على أنهي Branch، وكل ملف في أنهي مكان من التلاتة، وكمان بيقترح عليك الأمر الجاي. المبرمجين المتمرسين بيكتبوه كل دقيقتين من غير ما ياخدوا بالهم، زي ما بتبص في المراية قبل ما تخرج. `git status` بيقولك إنت فين، وكل ملف في أنهي حالة، والأمر الجاي المفروض يبقى إيه. اعمله قبل وبعد أي حاجة.

Terminal

```
you@laptop:~/portfolio$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   index.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt
```

قراية git status

`On branch main`	إنت واقف على Branch اسمه `main`.
`Changes to be committed:`	**الـ Staging Area**: دول اللي هيدخلوا الـ Commit الجاي. (`new file` = جديد، `modified` = متعدل، `deleted` = اتمسح).
`Changes not staged for commit:`	**الورشة**: ملفات Git عارفها واتعدلت، بس لسه مش في الـ Staging.
`Untracked files:`	ملفات جديدة Git مش بيتابعها خالص.
`(use "git ..." to ...)`	نصايح Git: الأمر اللي ينقلك للخطوة الجاية أو يرجعك. الـ `<file>` معناها "اكتب اسم الملف هنا" (مش بتكتب الأقواس).

النسخة المختصرة: -s و -b

Terminal

```
you@laptop:~/portfolio$ git status -sb
## main
A  index.html
 M app.js
?? notes.txt
```

تشريح الأمر

```
git status -sb
```

1 `git status` اعرض الحالة.

2 `-s` Short: سطر واحد لكل ملف بكود من حرفين (الجدول تحت).

3 `b` Branch: ضيف سطر `##` فيه اسم الـ Branch. ولأن الـ Flags القصيرة بتتجمع (`-s -b` = `-sb`) زي `ls -la` بالظبط.

الكود	العمود الشمال (Staging)	العمود اليمين (الورشة)	المعنى
`??`	؟	؟	Untracked
`A`	A	فاضي	ملف جديد متجهّز (Added)
`M`	فاضي	M	متعدل ولسه مش متجهّز
`M`	M	فاضي	متعدل ومتجهّز
`MM`	M	M	اتجهّز، وبعدين اتعدل تاني (جزء متجهّز وجزء لأ)
`D` / `D`	نفس الفكرة		اتمسح (Deleted)
`R`	R		اتغير اسمه (Renamed)

جدول 3.3 أكواد git status -s

3.6 git add: حط في الـ Staging

تشريح الأمر

```
git add index.html app.js
```

1 `git add` انقل **الشكل الحالي** للملفات دي للـ Staging Area.

2 `index.html app.js` Arguments: ملف أو أكتر، أو فولدرات، أو Patterns. اسمها في Git **Pathspec**.

الأمر	بيضيف إيه
`git add file.js`	ملف واحد
`git add src/`	فولدر بكل اللي جواه
`git add .`	كل التغييرات في الفولدر الحالي واللي جواه (جديد ومتعدل وممسوح)
`git add -A`	All: كل التغييرات في الـ Repo كله، حتى لو إنت في فولدر فرعي

الأمر	بيضيف إيه
`git add -u`	Update: المتعدل والممسوح بس، من غير الملفات الجديدة
`git add '*.js'`	كل ملفات js (بالـ Quotes عشان Git هو اللي يفك النجمة في كل الفولدرات)
`git add -p`	Patch: يسألك على كل حتة اتغيرت، تختار تضيفها ولا لأ (الفصل 13)

جدول 3.4 أشكال git add

حاجة مشتركة: . و * زي الـ Terminal

الـ `.` هنا هي "الفولدر الحالي" من كتاب الـ Terminal. والـ `*` من غير Quotes الـ Shell هو اللي بيفكها (Globbing) قبل ما Git يشوفها، فـ `git add *.js` هتجيب ملفات الفولدر الحالي بس. ومع Quotes (`'*.js'`) الـ Shell بيسيبها زي ما هي و Git يفكها في كل الفولدرات.

git add . مش دايماً صاحبك

`git add .` مريح، بس بيضيف **أي حاجة**: ملف `.env` فيه باسوردات، أو `node_modules` كله. عشان كده لازم `.gitignore` (الفصل 5)، ولازم تبص على `git status` قبل الـ Commit دايماً.

3.7 git commit: سجّل اللحظة

تشريح الأمر

```
git commit -m "feat: add home page"
```

1. `git commit` خد اللي في الـ Staging Area، واعمل منه Snapshot جديدة في التاريخ. اللي برا الـ Staging مش هيدخل.
2. `-m` Message: الرسالة جاية بعدي على طول. من غيرها Git هيفتح الـ Editor.
3. `"feat: add home page"` الرسالة. (الـ `feat:` دي صيغة اسمها Conventional Commits، الفصل 13).

Terminal

```
you@laptop:~/portfolio$ git commit -m "feat: add home page"
[main (root-commit) c71f7c3] feat: add home page
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
```

قراية ناتج git commit

```
[main (root-commit) c71f7c3]
```

الـ Commit اتعمل على `main`، و`root-commit` = أول Commit في التاريخ (مالوش أب)، و`c71f7c3` أول 7 حروف من الـ Hash.

إحصائية: ملف واحد اتغير، وسطر واحد اتضاف.	`1 file changed, 1 insertion(+)`
ملف جديد، والـ `100644` = ملف عادي صلاحياته `644` (افتكر الصلاحيات من كتاب الـ Terminal). لو Script قابل للتشغيل هتشوف `100755`.	`create mode 100644 index.html`

أشكال تانية للـ Commit

`git commit`	يفتح الـ Editor: أول سطر = العنوان، وسطر فاضي، وبعده شرح أطول. الأحسن للـ Commits المهمة.
`git commit -m "title" -m "body"`	كل `-m` بتعمل فقرة: الأولى عنوان والتانية تفاصيل.
`git commit -am "fix: typo"`	`-a` = All tracked: اعمل add لكل الملفات **المتابَعة** المتعدلة واعمل Commit مرة واحدة. **مش بتضيف الملفات الجديدة** (Untracked).
`git commit -v`	Verbose: يحطلك الـ diff جوه الـ Editor وانت بتكتب الرسالة، عشان تفتكر غيرت إيه.

مشكلة 1 **Author identity unknown**

أول Commit طلع: `Please tell me who you are` ومعاه `fatal: unable to auto-detect email address`.

الحل: مظبطتش اسمك وإيميلك. ارجع لـ ↩ 2.3 ص 15 واعمل `git config --global user.name` و `user.email`، وبعدين أعد الـ Commit.

مشكلة 2 **nothing added to commit**

عملت `git commit -m "..."` وطلع `nothing added to commit but untracked files present`.

الحل: نسيت `git add`. الـ Commit بياخد من الـ Staging بس، والـ Staging فاضي. اعمل `git add` للملفات الأول.

3.8 مثال كامل: أول مشروع JS

جرّب بإيدك من الأول للآخر

هنمشي على مشروع صغير خطوة بخطوة، وبعد كل خطوة بنعمل `git status`.

1. **اعمل المشروع:** `git init portfolio` و `cd portfolio`.
2. **اكتب ملفين:** `index.html` و `script.js` فيه `console.log("hi")`. الـ status: الاتنين `??` Untracked.
3. **جهّز واحد بس:** `git add index.html`. الـ status: `A index.html` و `?? script.js`.
4. **سجّل:** `git commit -m "feat: add home page"`. الـ status: `script.js` لسه Untracked.

5 **الباقي:** `git add script.js` ثم `git commit -m "feat: add script"` . الـ status: `nothing to commit, working tree clean` .

6 **عدّل:** غيّر سطر في `script.js` . الـ status: `M script.js` . وكرر الدورة.

```
Terminal
you@laptop:~/portfolio$ git status
On branch main
nothing to commit, working tree clean
```

الجملة دي (`working tree clean`) معناها إن الأماكن التلاتة متطابقة: كل حاجة متسجلة. دي أأمن حالة تكون فيها قبل أي تجربة كبيرة أو قبل ما تخلي الـ AI يلمس الكود.

إمتى أعمل Commit؟

القاعدة: **كل ما توصل لحاجة شغالة وليها معنى.** خلصت جزء من محاضرة؟ Commit. صلّحت Bug؟ Commit. قبل ما تجرّب حاجة ممكن تبوّظ؟ Commit. الـ Commits الصغيرة الكتير أحسن بكتير من Commit واحد ضخم آخر اليوم اسمه "stuff". وكل Commit يحكي حكاية واحدة.

ملخص الفصل

- `git init` بيعمل فولدر `.git` فيه التاريخ كله. متلمسوش، ومسحه = مسح التاريخ.
- 3 أماكن: **Working Directory** (بتعدّل) ← `git add` ← **Staging** (بتختار) ← `git commit` ← **Repository** (التاريخ).
- حالات الملف: Untracked و Unmodified و Modified و Staged.
- `git status` قبل وبعد كل حاجة، و `-sb` للنسخة المختصرة.
- `git commit -m` بياخد من الـ Staging بس، و `-am` مبيضيفش الملفات الجديدة.

صح ولا غلط؟

1. `git commit -am` بيضيف الملفات الجديدة اللي Git مش بيتابعها. صح / غلط
2. لو عدّلت ملف بعد ما عملته `git add` ، التعديل الجديد هيدخل الـ Commit تلقائياً. صح / غلط
3. مسح فولدر `.git` بيمسح ملفات المشروع. صح / غلط

أسئلة مراجعة

1. اشرح الأماكن التلاتة بمثال من مشروعك.
2. ليه Git محتاج Staging Area؟ إيه المشكلة لو الـ Commit بياخد كل التغييرات على طول؟
3. إيه معنى `MM` في `git status -s` ؟ وإزاي توصلها؟
4. إيه الفرق بين `git add .` و `git add -A` و `git add -u` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Repo اسمه `git-playground` فيه 3 ملفات، واعمل لكل ملف Commit لوحده.
2. متوسط وصّل ملف لحالة `MM` : عدّله، `git add` ، عدّله تاني. اعمل `git commit -m` وبعدين `git status` : إيه اللي دخل وإيه اللي فضل؟
3. صعب بعد أول Commit، اعمل `ls .git/objects` وبعدين `git cat-file -p HEAD` . اوصف اللي شايفه: فين الـ tree، والـ author، والرسالة؟

04

المرحلة 2 · الأساس على جهازك

الفصل الرابع

قراية التاريخ: log و diff و show

عملنا Commits، طب نشوفها إزاي؟ ونعرف اتغير إيه بين النهارده وإمبارح؟ وإيه اللي الـ AI غيّره بالظبط قبل ما نوافق عليه؟ الفصل ده عن تلات أوامر هتعيش معاهم: git log للتاريخ، و git diff للفروقات، و git show لتفاصيل Commit واحد. وهنتعلم نسمّي أي Commit بـ HEAD~1 وأخواتها.

بعد الفصل ده هتكون عارف

- ☑ git log وأهم الـ Flags بتاعته (oneline و graph و all و stat و p)
- ☑ الفلترة بالكاتب والتاريخ والرسالة والملف
- ☑ تسمية الـ Commits: Hash و HEAD و HEAD~n و HEAD^
- ☑ git diff بين الأماكن التلاتة وبين أي Commits
- ☑ قراية ناتج الـ diff سطر سطر
- ☑ git show لـ Commit أو لملف في نسخة قديمة

4.1 git log: شريط التاريخ

فاكر تطبيق البنك اللي وقع في أول فصل؟ تخيل نفس الموقف، بس المرة دي المشروع عليه Git. الصبح التطبيق كان شغال، وبالليل مش شغال. أول سؤال هيجي في دماغك: "إيه اللي حصل من الصبح لحد دلوقتي؟". زمان كنت هتحاول تفتكر. دلوقتي هتسأل Git، وهو بيحتفظ بشريط كامل لكل لحظة سجلتها: مين، وإمتى، وكتب إيه في الرسالة. الشريط ده بنشوفه بـ `git log`:

Terminal

```
you@laptop:~/portfolio$ git log
commit 64d09d60f21e2d9e543ea9afe345c927b6db3c58 (HEAD -> main)
Author: Sara <sara@example.com>
Date:   Tue Oct 20 10:00:00 2026 +0300

    feat: add app script

commit c71f7c3e7148d9c2cc6969c611f0f386ec3b6491
Author: Sara <sara@example.com>
Date:   Tue Oct 20 09:41:12 2026 +0300

    feat: add home page
```

قراية git log

`commit 64d09d60f21e…`	الـ Hash الكامل (40 حرف). الأحدث فوق.
`(HEAD -> main)`	"إنت هنا": HEAD بيشاور على `main`، و main بيشاور على الـ Commit ده (الفصل 7).

`Author: / Date:`	مين وإمتى (من `user.name` و `user.email`). `+0300` = فرق التوقيت عن UTC.
`feat: add app script`	الرسالة، بمسافة في الأول.

حاجة مشتركة: الـ Pager

لو التاريخ طويل، Git بيعرضه في `less` زي ما اتعلمنا في كتاب الـ Terminal: `Space` صفحة، `كلمة/` للبحث، و `q` للخروج. ده نفس الكلام في `git diff` و `git show` و `git branch`. ولو عاوز الناتج يتطبع على طول من غير Pager: `git --no-pager log`.

أهم Flags لـ git log

الشكل الطويل ده مفيد، بس بعد 50 Commit هيبقى مرهق. في الحياة اليومية محدش بيستخدمه كده. الشكل اللي هتكتبه كل يوم هو ده، وخليه في دماغك كأنه "الخريطة":

تشريح الأمر

```
1       2          3         4      5
git log --oneline --graph --all -n 10
```

1. `git log` اعرض التاريخ.
2. `--oneline` كل Commit في سطر: Hash قصير + العنوان. أكتر Flag هتستخدمه.
3. `--graph` ارسم خطوط الـ Branches والـ Merges بحروف (`*` و `|` و `/`) على الشمال.
4. `--all` اعرض كل الـ Branches، مش بس اللي إنت عليه.
5. `-n 10` آخر 10 Commits بس. ويتكتب `-10` برضه.

Terminal

```
you@laptop:~/portfolio$ git log --oneline --graph --all
* 4ea67eb (feature/title) feat: change title
| *   48c8db7 (HEAD -> main) Merge branch 'x2'
| |\
| | * 0dd1b27 docs: add notes
| |/
| * 38c2aec fix: title text
|/
* 64d09d6 feat: add app script
* c71f7c3 feat: add home page
```

ودي بقى شجرة التاريخ: كل `*` = Commit، والخطوط بتوريك مين جه من مين. متقلقش لو الرسمة مش واضحة دلوقتي، هنفهمها خطوة خطوة في فصول الـ Branches.

وأحياناً انت مش عاوز التاريخ كله، عاوز تدوّر على حاجة معينة: "إيه اللي عملته الأسبوع ده؟"، "مين لمس ملف app.js؟"، "أنهي Commit ضاف كلمة transfer أول مرة؟". `git log` عنده Flag لكل سؤال من دول، ودي أهمهم في جدول تفتكره بيه:

الـ Flag	بيعمل إيه	مثال
`--stat`	تحت كل Commit: الملفات اللي اتغيرت وعدد السطور	`git log --stat -3`
`-p`	Patch: اعرض الـ diff الكامل لكل Commit	`git log -p -1`
`--author=`	Commits كاتب معين (جزء من الاسم يكفي)	`git log --author=Sara`
`--since=` / `--until=`	من / لحد تاريخ	`git log --since="2 weeks ago"`
`--grep=`	دوّر في رسايل الـ Commits	`git log --grep=fix -i`
`-S`	الـ Commits اللي ضافت أو شالت نص معين من الكود	`git log -S "transfer"`
`-- file`	الـ Commits اللي لمست الملف ده بس	`git log --oneline -- app.js`
`--follow`	تابع الملف حتى لو اسمه اتغير	`git log --follow -- src/app.js`

جدول 4.1 Flags مهمة لـ git log

تشريح الأمر

```
git log --oneline --since="2 weeks ago" --author=Sara -- app.js
```

1. `git log --oneline` تاريخ مختصر.
2. `--since="2 weeks ago"` من أسبوعين لحد دلوقتي. Git بيفهم إنجليزي بسيط (`yesterday`، `3 days ago`) أو تاريخ (`2026-10-01`). والـ `=` بتربط الـ Option بقيمتها.
3. `--author=Sara` اللي كاتبهم فيه "Sara".
4. `--` "خلصت الـ Options، اللي بعدي أسماء ملفات". نفس معنى `--` في كتاب الـ Terminal. مهمة لو اسم ملف شبه اسم Branch.
5. `app.js` الملف.

4.2 إزاي تسمّي أي Commit

أوامر كتير (diff و show و reset و revert) محتاجة تقولها "أنهي Commit". عندك كذا طريقة:



شكل 4.1 العد لورا من HEAD: الـ 1~ الأب، والـ 2~ الجد، وهكذا.

الطريقة	يعني إيه
`64d09d6`	الـ Hash القصير (أول 7 حروف، أو أكتر لو فيه تشابه)
`HEAD`	الـ Commit اللي إنت واقف عليه دلوقتي
`HEAD~1` أو `HEAD~`	الأب (اللي قبله بواحد)
`HEAD~3`	تلات خطوات لورا
`HEAD^`	الأب الأول (زي `~1`). و `HEAD^2` = الأب التاني في Merge Commit
`main~2` / `main`	آخر Commit في main / اتنين قبله
`v1.0`	Tag (الفصل 14)

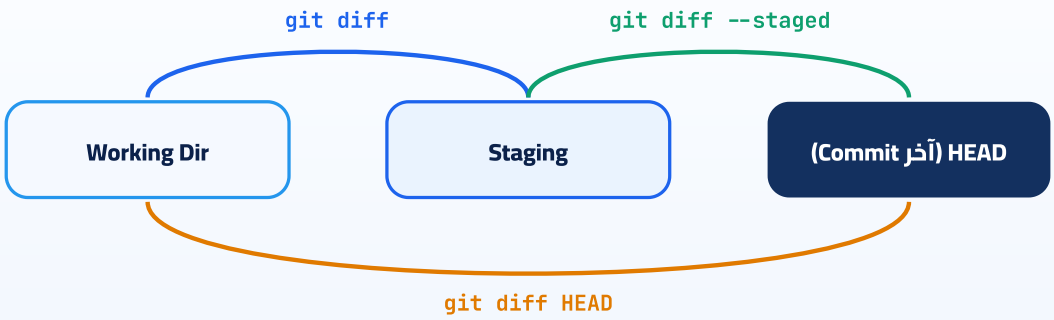
جدول 4.2 طرق تسمية الـ Commit

> ⚠ **خلي بالك من ^ في zsh على الـ Mac**
>
> في zsh (الافتراضي على الـ Mac من 2019) لو مفعّل `extendedglob` (زي كتير من إعدادات oh-my-zsh)، ممكن `HEAD^` تطلّع `no matches found` لأن الـ `^` ليها معنى Globbing هناك. الحل: استخدم `HEAD~1` (أسهل) أو حط Quotes: `"HEAD^"`.

4.3 git diff: إيه اللي اتغير بالظبط؟

الـ log بيقولك **إمتى** اتغيرت حاجة، بس مش بيقولك **إيه** اللي اتغير بالظبط. هنا ييجي أهم أمر هتستخدمه مع أدوات الـ AI تحديداً: `git diff`. لما الـ AI يقولك "عدّلت الملف عشان أصلّح المشكلة"، الـ diff هو اللي بيوريك الحقيقة سطر سطر: إيه اللي اتشال وإيه اللي اتضاف. ومحدش محترم بيعمل Commit لحاجة من غير ما يبص على الـ diff بتاعها.

بس فيه حتة صغيرة بتلخبط الناس: انت عندك 3 أماكن (الورشة والـ Staging والتاريخ)، فـ "الفرق" ممكن يكون بين أي اتنين فيهم. والمهم تعرف كل شكل من `git diff` بيقارن مين بمين:



شكل 4.2 تلات أشكال لـ git diff بين الأماكن التلاتة.

الأمر	بيقارن	بيجاوب على
`git diff`	الورشة مقابل الـ Staging	"إيه اللي عدلته ولسه معملتلوش add؟"
`git diff --staged`	الـ Staging مقابل آخر Commit	"إيه اللي هيدخل الـ Commit الجاي؟" (`--cached` نفس المعنى)
`git diff HEAD`	الورشة مقابل آخر Commit	"كل اللي اتغير من آخر Commit"
`git diff a1b2c3d HEAD`	Commit مقابل Commit	"إيه الفرق بين النسختين دول؟"
`git diff main feature`	Branch مقابل Branch	"الـ Branch ده فيه إيه زيادة؟"

جدول 4.3 أشكال git diff

> **أشهر لخبطة**
>
> عملت `git add` وبعدين `git diff` وملقيتش حاجة؟ طبيعي! `git diff` العادي بيقارن الورشة بالـ Staging، وانت لسه حاطط كل حاجة في الـ Staging فمفيش فرق. استخدم `git diff --staged`.

قراية ناتج الـ diff

Terminal

```
you@laptop:~/portfolio$ git diff
diff --git a/app.js b/app.js
index d914c60..1000cb0 100644
--- a/app.js
+++ b/app.js
@@ -1,3 +1,4 @@
 const balance = 100;
-console.log("hi");
+console.log("Welcome!");
+const rate = 0.05;
 export default balance;
```

تشريح الـ diff	
`diff --git a/app.js b/app.js`	بنقارن `a/` (النسخة القديمة) بـ `b/` (الجديدة) لنفس الملف.
`index d914c60..1000cb0 100644`	Hash نسخة الملف القديمة..الجديدة، وصلاحياته. مش هتحتاجه غالباً.
`--- a/app.js` `+++ b/app.js`	الـ `---` = القديم، والـ `+++` = الجديد. ولو ملف جديد هتلاقي `/dev/null` مكان القديم.
`@@ -1,3 +1,4 @@`	عنوان الحتة (Hunk): في القديم من سطر 1 لمدة 3 سطور، وفي الجديد من سطر 1 لمدة 4 سطور.
` const balance = 100;`	سطر بمسافة في الأول = متغيرش (Context عشان تفهم المكان).
`-console.log("hi");`	`-` = سطر اتشال (بيبان أحمر).
`+console.log("Welcome!");`	`+` = سطر اتضاف (بيبان أخضر). و"تعديل" سطر في Git = شيل القديم + ضيف الجديد.

Flags مفيدة لـ git diff

`git diff --stat`	ملخص: كل ملف وعدد السطور اللي اتضافت واتشالت (`+++--`). ممتاز لما الـ AI يغيّر حاجات كتير.
`git diff --name-only`	أسماء الملفات اللي اتغيرت بس.
`git diff --word-diff`	الفروق بالكلمة مش بالسطر (`[-قد يم-]{+جد يد+}`). ممتاز للنصوص و Markdown.
`git diff -- app.js`	ملف واحد بس (الـ `--` تاني).
`git diff HEAD~3 HEAD --stat`	إيه اللي اتغير في آخر 3 Commits.

شوف الـ diff في VS Code

تاب **Source Control** في VS Code بيعرض نفس الـ diff بس جنب بعض وملوّن. استخدم اللي يريحك، المهم **متعملش Commit لحاجة مقريتهاش**. ولو عاوز VS Code يبقى أداة الـ diff بتاعة Git: `git config --global diff.tool vscode` و `git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'`، وبعدين `git difftool`.

4.4 git show: تفاصيل Commit واحد

تشريح الأمر

```
1       2        3
git show 64d09d6 --stat
```

1. `git show` اعرض حاجة واحدة بالتفصيل. من غير Argument = `HEAD`.
2. `64d09d6` الـ Commit (أي طريقة تسمية من ↩ 4.2 ص 34).
3. `--stat` بدل الـ diff الكامل، ملخص الملفات. من غيرها هيعرض الـ diff كله.

تشريح الأمر

```
1       2  3  4
git show HEAD~2:src/app.js
```

1. `git show` اعرض.
2. `HEAD~2` من نسخة المشروع من تلات Commits.
3. `:` النقطتين هنا معناها "ملف جوه الـ Commit ده".
4. `src/app.js` مسار الملف من أول الـ Repo. الناتج: الملف زي ما كان ساعتها، من غير ما تغيّر حاجة عندك. وتقدر تحفظه: `git show HEAD~2:src/app.js > old-app.js`.

مثال من الحياة

الـ Bankist app كان شغال إمبارح والنهارده لأ؟ `git log --oneline --since=yesterday` يوريك الـ Commits من إمبارح، وبعدين `git diff HEAD~3 --stat` يوريك الملفات اللي اتلمست، و `git show` على كل واحد يوريك التغيير بالظبط. في الفصل 16 هنشوف `git bisect` اللي بيعمل التدوير ده أوتوماتيك.

ملخص الفصل

- `git log --oneline --graph --all` أكتر شكل مفيد، و `q` للخروج من الـ Pager.
- فلترة: `--author` و `--since` و `--grep` و `-S` و `-- file`.
- التسمية: Hash قصير، `HEAD`، `HEAD~n`، اسم Branch أو Tag.
- `git diff` = ورشةStaging/، و `--staged` = Staging/Commit، و `HEAD` = ورشةCommit/.
- `-` اتشال، `+` اتضاف، و `git show HEAD~2:file` يوريك ملف في نسخة قديمة.

صح ولا غلط؟

1. بعد `git add` لكل حاجة، `git diff` العادي هيبقى فاضي. صح / غلط
2. `HEAD~1` و `HEAD^` نفس الـ Commit في التاريخ العادي من غير Merge. صح / غلط
3. `git show HEAD~2:app.js` يرجّع الملف لنسخته القديمة في الورشة. صح / غلط

؟ أسئلة مراجعة

1. اكتب أمر يعرض آخر 5 Commits لمس فيهم أي حد ملف `index.html`، سطر لكل واحد.
2. عاوز تشوف اللي هيدخل الـ Commit الجاي قبل ما تعمله. أنهي أمر؟
3. اشرح `@@ -10,4 +10,6 @@`.
4. إيه فايدة `--` في `git log -- app.js`؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في الـ playground، اعمل 5 Commits صغيرة وبعدين جرّب `git log` بأشكاله: `--oneline`، `--stat`، `-p -1`.
2. متوسط عدّل ملف، اعمل `git add`، عدّله تاني. جرّب `git diff` و `git diff --staged` و `git diff HEAD` واشرح الفرق بين التلاتة من الناتج.
3. صعب استخدم `git log -S` عشان تلاقي الـ Commit اللي ضاف كلمة معينة في الكود لأول مرة، وبعدين `git show` عليه.

05

المرحلة 2 · الأساس على جهازك

الفصل الخامس

الـ gitignore.: حاجات Git ميشوفهاش

مش كل حاجة في فولدر المشروع مكانها Git. node_modules فيها آلاف الملفات اللي بتتعمل بـ npm install، و env. فيه مفاتيح وباسوردات، وملفات النظام زي DS_Store. ملهاش لازمة. الفصل ده عن ملف gitignore.: قواعده، وقالب جاهز لمشاريع JavaScript، وإزاي تشيل ملف من Git لو اتضاف بالغلط.

بعد الفصل ده هتكون عارف

- ليه بنتجاهل ملفات ومين أشهرهم في مشاريع JS
- قواعد الـ Patterns: * و ** و / و ! و #
- قالب gitignore. جاهز لمشاريع JavaScript
- الـ gitignore. العام لكل مشاريعك
- git check-ignore عشان تعرف ليه ملف متجاهَل
- git rm --cached لملف اتضاف قبل ما تتجاهله، و git rm و git mv

فاكر node_modules و npm install؟

الفصل ده بيتكلم كتير عن `node_modules` و `npm install` وفولدرات الـ Build (`dist` و `build`) وملف `.env`. لو الكلمات دي جديدة عليك، بص على صندوق "npm و node_modules و package.json في دقيقة" في كتاب الـ Terminal (فصل 11، Homebrew ومديري الحزم). وباختصار: `node_modules` = المكتبات اللي npm نزّلها، وبتتعمل تاني بـ `npm install`. والـ Build = نسخة نهائية متولّدة من الكود. و `.env` = ملف نصي فيه إعدادات وأسرار زي الـ API Key (مفتاح سري بيدّيهولك أي موقع علشان برنامجك يستخدم خدماته). الحاجات دي كلها يا بتتولّد يا سرية، فمكانها مش Git.

5.1 ليه نتجاهل ملفات؟

أول مرة هتعمل `npm install` لمكتبة صغيرة في مشروعك وتكتب `git status`، هتتخض: فولدر اسمه `node_modules` ظهر، وجواه حرفياً آلاف الملفات، لمكتبات انت عمرك ما سمعت عنها (لأن كل مكتبة جايبة معاها مكتبات). وفيه نكتة مشهورة إن `node_modules` أتقل حاجة في الكون، وهي مش بعيدة عن الحقيقة. وجنبه ملف صغير بريء اسمه `.env` انت حاطط فيه الـ API Key بتاع خدمة الطقس اللي بتستخدمها في المشروع.

لو عملت `git add .` وCommit و push، هيحصل حاجتين وحشين. الأولى إن الـ Repo هيبقى تقيل جداً، وكل `npm install` هيعمل تغييرات مالهاش معنى. وأخطر من كده: ملف `.env` فيه الـ API Key بتاعك، ولو اتعمله Push على GitHub public، فيه Bots بتمسح GitHub بتدوّر على المفاتيح دي.

حاجات بتتولّد

`node_modules/` (بترجع بـ `npm install`)، و `dist/` و `build/` (ناتج الـ Build).

أسرار

`.env` و `.env.local` وأي مفاتيح أو شهادات (`*.pem`).

ملفات النظام والـ Editor

`.DS_Store` (macOS)، `Thumbs.db` (Windows)، و `.idea/`.

Logs ومؤقت

`*.log` و `npm-debug.log*` و `.cache/` و `coverage/`.

القاعدة الذهبية

الـ Repo فيه **اللي إنت كتبته** (الكود، والإعدادات من غير أسرار) + **اللي يلزم عشان أي حد يبني المشروع** (`package.json` و `package-lock.json`). أي حاجة بتتولّد من حاجة تانية أو سرية = `.gitignore`.

5.2 قواعد الـ Patterns

`.gitignore` ملف نصي عادي في أول المشروع، كل سطر فيه Pattern. لو ملف طابق Pattern وكان Untracked، Git هيعمل نفسه مش شايفه.

الـ Pattern	بيطابق إيه
`.env`	أي ملف أو فولدر اسمه `.env` في أي مكان في المشروع
`*.log`	`*` = أي حروف: كل الملفات اللي بتنتهي بـ `.log`
`node_modules/`	الـ `/` في الآخر = فولدرات بس بالاسم ده (في أي مكان)
`/dist`	الـ `/` في الأول = في أول المشروع بس، مش `src/dist`
`docs/*.pdf`	ملفات pdf جوه `docs` مباشرة (مش في فولدرات جواه)
`**/temp`	`**` = أي عمق من الفولدرات
`logs/**/*.log`	أي log في أي مكان جوه `logs`
`img?.png`	`?` = حرف واحد بالظبط
`!keep.log`	`!` = **استثناء**: متتجاهلش ده رغم إن اللي فوق تجاهله
`# comment`	تعليق

جدول 5.1 قواعد الكتابة في gitignore.

حاجة مشتركة: نفس رموز الـ Globbing

`*` و `?` و `[abc]` هنا نفس معانيها في الـ Globbing اللي اتعلمناه في كتاب الـ Terminal. الفرق إن Git هو اللي بيفكها مش الـ Shell، وإن `**` شغالة على طول (في bash محتاجة `globstar`). ونفس الـ Patterns دي بتستخدمها في `git add '*.js'`.

> **فخ الاستثناء جوه فولدر متجاهَل**
>
> لو كتبت `logs/` وبعدين `!logs/keep.log` ، الاستثناء **مش هيشتغل**: Git مش بيدخل جوه فولدر متجاهَل أصلاً. الحل: تجاهل المحتوى مش الفولدر: `logs/*` وبعدين `!logs/keep.log` .

5.3 قالب جاهز لمشاريع JavaScript

.gitignore

1 المكتبات اللي `npm install` بيجيبها. بترجع من `package.json` و `package-lock.json` (ودول **لازم** يتعملهم Commit).	`# dependencies` `node_modules/`

2 ناتج الـ Build (Vite أو Parcel)، بيتولّد من الكود.	`# build output`
	`dist/`
	`build/`
	`.parcel-cache/`

3 كل ملفات `.env` ، ما عدا `.env.example` : نسخة من غير قيم حقيقية عشان اللي بعدك يعرف المتغيرات المطلوبة.	`# secrets`
	`.env`
	`.env.*`
	`!.env.example`

4 Logs بتتعمل وانت بتشغّل.	`# logs`
	`*.log`
	`npm-debug.log*`

5 ملفات النظام، وإعدادات VS Code الشخصية، مع استثناء ملف الإضافات المقترحة للفريق.	`# OS / editor`
	`.DS_Store`
	`Thumbs.db`
	`.vscode/*`
	`!.vscode/extensions.json`

> **متكتبهوش من الصفر**
>
> GitHub عنده قوالب جاهزة لكل لغة: لما تعمل Repo جديد على الموقع اختار **Add .gitignore** ← **Node**. ولو بتستخدم Vite (`npm create vite@latest`) فهو بيعملك `.gitignore` جاهز. واعمل الملف ده **قبل أول** `git add` .

5.4 .gitignore عام لكل مشاريعك

ملفات زي `.DS_Store` دي بتاعة جهازك إنت، مش بتاعة المشروع. حطها مرة واحدة في ملف عام:

تشريح الأمر

```
git config --global core.excludesFile ~/.gitignore_global
```

1 `git config --global` إعداد عام (↩ 2.2 ص 15).

2 `core.excludesFile` "فيه ملف تجاهل إضافي، طبّقه على كل الـ Repos".

3 `~/.gitignore_global` مسار الملف في الـ Home (الـ `~` من كتاب الـ Terminal). اعمله واكتب فيه `.DS_Store` و `Thumbs.db`.

المكان	بيتشارك مع الفريق؟	استخدمه لـ
`.gitignore` في المشروع	أيوه (بيتعمله Commit)	حاجات المشروع: node_modules و dist و .env
`~/.gitignore_global`	لأ	حاجات جهازك: .DS_Store و Editor
`.git/info/exclude`	لأ	حاجة خاصة بيك في Repo واحد

جدول 5.2 أماكن قواعد التجاهل

5.5 git check-ignore: ليه الملف ده متجاهَل؟

تشريح الأمر

```
git check-ignore -v .env keep.log
```

1 `git check-ignore` قولّي الملفات دي متجاهَلة ولا لأ.

2 `-v` Verbose: قولّي كمان **أنهي ملف وأنهي سطر** هو السبب.

3 `.env keep.log` الملفات اللي بنسأل عليها.

Terminal

```
you@laptop:~/portfolio$ git check-ignore -v .env keep.log
.gitignore:2:.env       .env
.gitignore:4:!keep.log  keep.log
```

كل سطر: `Pattern الملف:رقم السطر:الـ` ثم اسم الملف. السطر التاني بيبدأ بـ `!` يعني آخر قاعدة طابقت كانت **استثناء**، فـ keep.log مش متجاهَل. ولو عاوز تشوف كل المتجاهَل: `git status --ignored` (المتجاهَل بيظهر بـ `!!` في `-s`).

5.6 اتضاف بالغلط قبل الـ .gitignore

مشكلة 1 **ضفت .env لـ .gitignore وبرضه بيظهر متعدل!**

الـ `.gitignore` بيأثر على الملفات **Untracked** بس. لو الملف اتعمله Commit قبل كده، Git بيفضل يتابعه عادي.

الحل: شيله من متابعة Git من غير ما تمسحه من جهازك:

```
Terminal
git rm --cached .env
git commit -m "chore: stop tracking .env"
```

ولو فولدر: `git rm -r --cached node_modules` .

تشريح الأمر

```
git rm -r --cached node_modules
1   2  3        4
```

1. `git rm` امسح من Git.
2. `-r` Recursive: الفولدر وكل اللي جواه (نفس `-r` بتاعة `rm` و `cp` في كتاب الـ Terminal).
3. `--cached` أهم حتة: امسحه من الـ **Index** (الـ Staging) بس، وسيبه على الديسك. من غيرها الملفات هتتمسح فعلاً من جهازك.
4. `node_modules` المسار.

⚠ لو السر اتعمله Push خلاص

`git rm --cached` بيشيل الملف من الـ Commits **الجاية** بس. لو `.env` اتعمله Push قبل كده، المفتاح لسه موجود في التاريخ وأي حد يقدر يشوفه. الحل الحقيقي: **غيّر المفتاح فوراً** (Revoke و Rotate) من الموقع اللي عمله. التفاصيل في الفصل 18.

5.7 git rm و git mv

الأمر	بيعمل إيه
`git rm file.js`	يمسح الملف من الديسك ويجهّز المسح في الـ Staging (خطوتين في واحدة: `rm` + `git add`)
`git rm --cached file.js`	يبطّل يتابعه بس، والملف يفضل
`git mv old.js new.js`	يغيّر الاسم ويجهّزه (= `mv` + `git add` للاتنين)
`git mv app.js src/`	ينقل لفولدر

جدول 5.3 المسح وتغيير الأسماء

لو مسحت أو غيرت الاسم من VS Code

عادي جداً. Git هيشوف ملف اتمسح وملف جديد اتعمل، ولما تعمل `git add -A` هيفهم إنه Rename لوحده (`R` في status) لو المحتوى قريب. `git mv` مجرد اختصار.

ملخص الفصل

- بنتجاهل: اللي بيتولّد (node_modules و dist)، والأسرار (env.)، وملفات النظام.
- `*` أي حروف، `**` أي عمق، `/` في الآخر فولدر، في الأول من الجذر، `!` استثناء.
- `.gitignore` للمشروع، و `core.excludesFile` لجهازك.
- `git check-ignore -v` يقولك أنهي قاعدة هي السبب.
- الـ gitignore. مبيأثرش على ملف متابَع: `git rm --cached` . والسر اللي اتعمله Push لازم يتغير.

صح ولا غلط؟

1. إضافة ملف لـ `.gitignore` بتشيله من Git لو كان متعمله Commit. صح / غلط
2. `package-lock.json` المفروض يتحط في `.gitignore` . صح / غلط
3. `git rm --cached file` بيمسح الملف من جهازك. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `dist/` و `/dist` و `dist` في gitignore.؟
2. ليه بنعمل Commit لـ `.env.example` ومش `.env` ؟
3. كتبت `logs/` و `!logs/keep.log` والاستثناء مشتغلش. ليه؟ وإزاي تصلحه؟
4. إيه الخطوات لو اكتشفت إن `node_modules` متعمله Commit؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في مشروع npm (`npm init -y` و `npm i lodash`)، اعمل `.gitignore` بالقالب، وتأكد بـ `git status` إن `node_modules` اختفى.
2. متوسط اعمل `.env` و `.env.example` ، واتأكد بـ `git check-ignore -v` إن الأول متجاهَل والتاني لأ.
3. صعب اعمل Commit لملف `debug.log` عمداً، وبعدين ضيف `*.log` للـ gitignore.، وخلّي Git يبطّل يتابعه من غير ما تمسحه. اتأكد بـ `git ls-files` .

06

المرحلة 2 · الأساس على جهازك

الفصل السادس

التراجع عن أي غلطة

ده الفصل اللي هيخليك متخافش من Git. عدّلت ملف وعاوز ترجعه؟ ضفت حاجة للـ Staging بالغلط؟ نسيت ملف في آخر Commit؟ عاوز تلغي آخر 3 Commits؟ عاوز تلغي Commit اتعمله Push؟ كل حالة ليها أمر، ولكل أمر درجة خطورة. وفي الآخر خريطة قرار تختار منها في ثانية.

بعد الفصل ده هتكون عارف

- أول سؤال قبل أي تراجع: الغلطة في أنهي مكان؟ واتعملها Push ولا لأ؟
- git restore للورشة و git restore --staged للـ Staging
- git commit --amend لتصليح آخر Commit
- git reset بأنواعه التلاتة soft و mixed و hard بالرسم
- git revert لإلغاء Commit بأمان حتى بعد الـ Push
- git clean للملفات الجديدة، وخريطة قرار التراجع

6.1 قبل أي تراجع: سؤالين

خليني أحكيلك 4 مواقف هيحصلولك كلهم في أول شهر. الأول: الـ AI عدّل `script.js` والتعديل وحش، وانت عاوز ترجع الملف زي ما كان. التاني: عملت `git add .` ولقيت إن ملف `notes.txt` الشخصي دخل في الـ Staging. التالت: عملت Commit وبعدها على طول افتكرت إنك نسيت تضيف ملف. والرابع، وده الأصعب: عملت Commit فيه Bug، وعملته Push، وزميلك عمل Pull وبدأ يشتغل عليه.

الأربع مواقف اسمهم "عاوز أتراجع"، بس كل واحد فيهم حله مختلف تماماً. وده السبب إن أوامر التراجع في Git كتير وبتلخبط: مش لأن Git معقد، لأن "التراجع" نفسه ليه معاني كتير. فالسر إنك قبل أي أمر تسأل نفسك سؤالين:

1 · الغلطة في أنهي مكان؟

في الورشة (ملف عدلته)؟ في الـ Staging (ضفت حاجة غلط)؟ ولا في التاريخ (Commit اتعمل)؟ افتكر الأماكن التلاتة (↩ 3.3 ص 24).

2 · اتعملها Push؟

لو الـ Commit لسه على جهازك بس، تقدر تعدّل التاريخ براحتك. لو اتعمله Push وحد تاني ممكن يكون خده، **متعدلش التاريخ**: استخدم `revert`.

الخبر الحلو

أي حاجة اتعملها **Commit** مرة، تقريباً مستحيل تضيع، حتى بعد `reset --hard`. الـ `reflog` (الفصل 16) يرجّعها. اللي ممكن يضيع بجد هو التعديلات اللي **عمرها ما اتعملها Commit ولا add**. وعشان كده: Commit كتير.

6.2 git restore: ارجع ملف في الورشة أو الـ Staging

ارمي تعديلاتك على ملف

تشريح الأمر

```
git restore app.js
```

1. `git restore` رجّع ملفات لنسخة متسجلة.
2. `app.js` الملف. من غير Options، بيرجّع نسخة الورشة لنسخة الـ **Staging** (واللي هي نفس آخر Commit لو معملتش add). يعني تعديلاتك اللي مش متعملها add هتتمسح.

> ⚠ **ده مفيهوش رجوع**
>
> `git restore file` و `git restore .` بيمسحوا تعديلات **عمرها ما اتسجلت**، فمفيش reflog ولا أي حاجة ترجّعها (يمكن Undo الـ Editor أو Local History في VS Code لو محفوظ). اتأكد بـ `git diff` الأول إنك عاوز ترميها.

شيل حاجة من الـ Staging (من غير ما تضيع)

تشريح الأمر

```
git restore --staged app.js
```

1. `git restore` رجّع.
2. `--staged` المرة دي رجّع الـ **Staging** (مش الورشة) لنسخة آخر Commit. يعني "ألغي الـ git add". تعديلاتك في الملف نفسه **مش هتتلمس**، آمن 100%.
3. `app.js` الملف، أو `.` للكل.

جيب ملف من نسخة قديمة

تشريح الأمر

```
git restore --source=HEAD~2 -- src/app.js
```

1. `git restore` رجّع.
2. `--source=HEAD~2` من الـ Commit ده بدل الـ Staging (افتكر ↩ 4.2 ص 34).
3. `--` خلصت الـ Options.
4. `src/app.js` الملف هيرجع زي ما كان من تلات Commits، في الورشة. وبعدها تعمل `git add` و Commit عادي لو عاجبك.

عاوز	الجديد (2.23+)	القديم اللي هتلاقيه في شروحات
ارمي تعديلات الورشة	`git restore f`	`git checkout -- f`
شيل من الـ Staging	`git restore --staged f`	`git reset HEAD f`
الاتنين مرة واحدة	`git restore --staged --worktree f`	`git checkout HEAD -- f`

جدول 6.1 git restore وأمثاله القديمة

حاجة مشتركة: --

الـ `--` تاني! في `git restore -- f` و `git log -- f` و `git diff -- f` و `git checkout -- f`، معناها في كل مرة: "اللي بعدي ملفات". وده بيفرق جداً في `git checkout` لو عندك Branch وملف بنفس الاسم.

6.3 git commit --amend: صلّح آخر Commit

عملت Commit ولقيت غلطة في الرسالة، أو نسيت تضيف ملف:

تشريح الأمر

```
git add forgotten.js && git commit --amend --no-edit
```

1. `git add forgotten.js` — حط الملف المنسي في الـ Staging.
2. `&&` — لو نجح، كمّل.
3. `git commit --amend` — بدل ما تعمل Commit جديد، **استبدل** آخر Commit بواحد جديد فيه محتواه + اللي في الـ Staging دلوقتي.
4. `--no-edit` — خلي الرسالة زي ما هي ومتفتحش الـ Editor.

`git commit --amend -m "fix: correct message"`	غيّر رسالة آخر Commit بس (لو الـ Staging فاضي).
`git commit --amend --reset-author --no-edit`	لو الـ Commit اتعمل باسم أو إيميل غلط وبعدين ظبطتهم.

amend = Commit جديد

الـ amend بيعمل Commit **جديد بـ Hash جديد** ويرمي القديم. لو القديم كان اتعمله Push، التاريخ عندك هيختلف عن GitHub وهتحتاج Force push (الفصل 15). القاعدة: **amend للي لسه على جهازك بس.**

6.4 git reset: رجّع الـ Branch لورا

ده الأمر اللي سمعته أكيد واتخوّفت منه، وليك حق جزئياً. خلينا نفهمه بهدوء. افتكر إن الـ Branch مجرد لافتة على Commit (هنشرحها بالتفصيل في الفصل 7، بس الفكرة دلوقتي كفاية). `git reset` بيشيل اللافتة ويحطها على Commit أقدم، كأن الـ Commits اللي بعده متعملتش. والسؤال: التغييرات اللي كانت فيهم تروح فين؟ ده اللي الـ Flag بيحدده:



شكل 6.1 الأنواع التلاتة لـ git reset: كل نوع بيرجّع أماكن أكتر من اللي قبله.

تشريح الأمر

```
git reset --soft HEAD~1
```

1. `git reset` حرّك الـ Branch الحالي (و HEAD معاه).
2. `--soft` حرّك التاريخ بس. التغييرات اللي كانت في الـ Commit الملغي **تفضل في الـ Staging**، جاهزة لـ Commit تاني.
3. `HEAD~1` لحد فين: الأب. يعني "ألغي آخر Commit". و `HEAD~3` = آخر 3.

النوع	التغييرات تروح فين	استخدامه	الخطورة
`--soft`	الـ Staging	"عاوز أعيد الـ Commit ده بشكل تاني" أو أجمع آخر 3 Commits في واحد	آمن
`--mixed`	الورشة (مش متجهزة)	"عاوز ألغي الـ Commit وأختار تاني أضيف إيه"	آمن
`--hard`	بترمي	"عاوز أرجع للنسخة القديمة بالظبط وأرمي كل اللي بعدها"	**خطر**

جدول 6.2 أنواع git reset وإمتى تستخدم كل واحد

مثال: جمّع آخر 3 Commits في واحد

عملت 3 Commits صغيرين: `wip` و `wip2` و `fix typo`. عاوزهم Commit واحد محترم:

```
Terminal
git reset --soft HEAD~3
git commit -m "feat: add transfer form"
```

الـ `--soft` رجّع التاريخ 3 خطوات وساب كل التغييرات متجهزة، فـ Commit واحد جمعهم.

قبل reset --hard

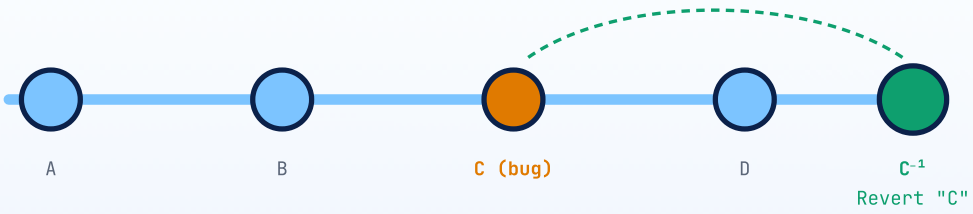
اعمل `git status`. لو فيه تعديلات مش متعملها Commit، `--hard` هيمسحها **من غير رجعة**. لو مش متأكد: `git stash` الأول (الفصل 14)، أو اعمل Branch احتياطي `git branch backup`. والـ Commits نفسها بترجع بـ reflog، وكمان Git بيحفظ مكانك القديم في `ORIG_HEAD`: `git reset --hard ORIG_HEAD` بيلغي آخر reset.

git reset على ملف

`git reset file.js` (من غير soft/mixed/hard ومع اسم ملف) = شيل الملف من الـ Staging. ده نفس `git restore --staged file.js`. ولو شفته في رسايل Git القديمة متتخضش.

6.5 git revert: الإلغاء الآمن

`reset` بيمسح Commits من التاريخ، وده مشكلة لو اتعملهم Push. `revert` بيعمل العكس: بيضيف **Commit جديد** بيعكس التغيير. التاريخ بيفضل زي ما هو، وبيتضاف عليه.



شكل 6.2 الـ revert بيضيف Commit جديد بيعكس C، و D فضل زي ما هو.

تشريح الأمر

```
git revert 38c2aec --no-edit
```

1. `git revert` اعمل Commit جديد بيعكس التغييرات اللي في الـ Commit ده (اللي اتضاف يتشال، واللي اتشال يرجع).
2. `38c2aec` الـ Commit اللي عاوز تلغيه، مش لازم يكون الأخير.
3. `--no-edit` استخدم الرسالة الجاهزة (`Revert "fix: title text"`) من غير ما تفتح الـ Editor.

القاعدة

Commit **على جهازك بس**؟ `reset` أو `amend` براحتك. Commit **اتعمله Push**؟ `revert` . ولو الـ revert عمل Conflict، هتحله بنفس طريقة الفصل 8، وبعدين `git revert --continue` .

revert لـ Merge Commit

الـ Merge Commit ليه أبين، فـ Git محتاج تقوله ترجع لأنهي واحد: `git revert -m 1 <merge-hash>` . الـ `-m 1` = Mainline: الأب الأول، يعني الـ Branch اللي كنت عليه وقت الـ Merge (غالباً main). هنفهم الـ Merge في الفصل 8.

6.6 git clean: الملفات الجديدة Untracked

`restore` و `reset` مبيلمسوش الملفات **Untracked**. لو عاوز تمسحها (مثلاً ملفات عملها الـ AI ومش عاوزها):

تشريح الأمر

```
git clean -n -d
```

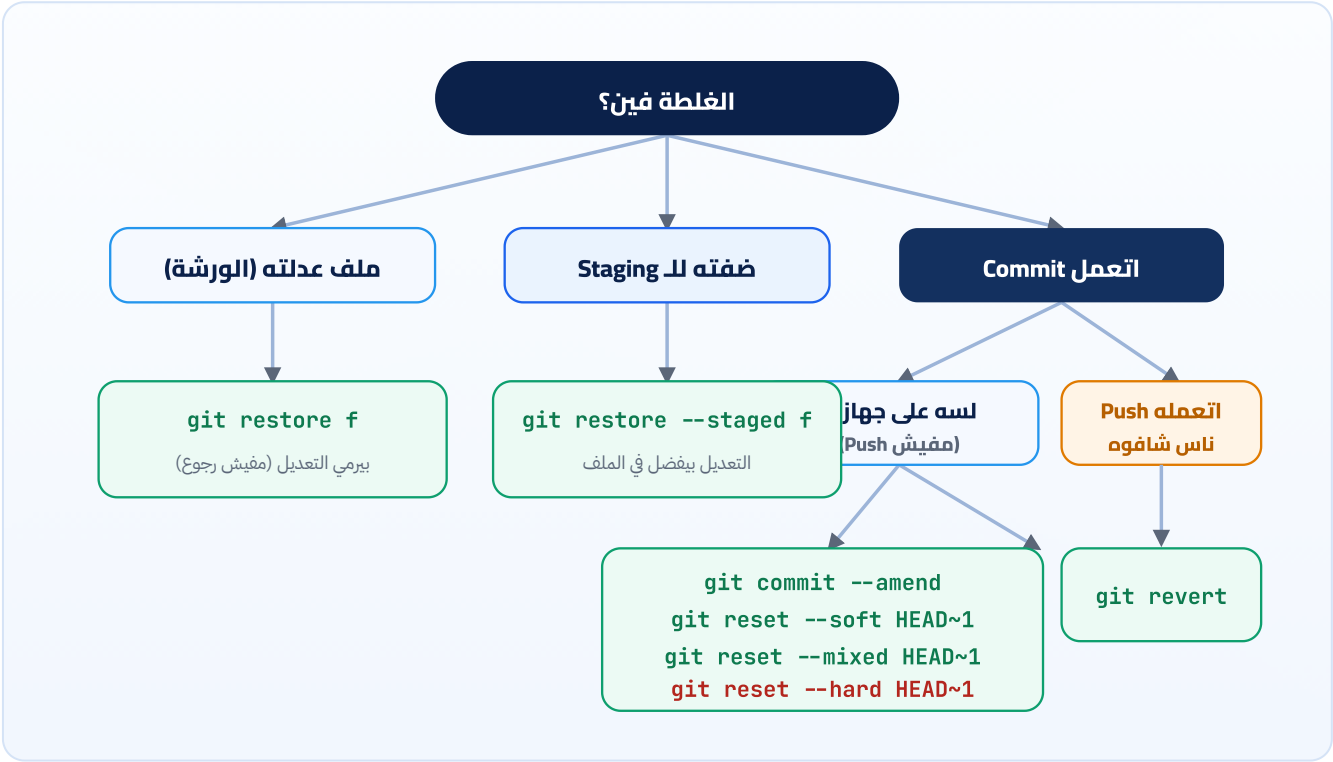
1. `git clean` امسح الملفات اللي Git مش بيتابعها.
2. `-n` Dry run: **اعرض** اللي هيتمسح من غير ما تمسح. ابدأ بيها دايماً (زي `--dry-run` في rsync).
3. `-d` Directories: الفولدرات الجديدة كمان، مش الملفات بس.

`git clean -n -d`	شوف الأول.
`git clean -f -d`	`-f` = Force: امسح بجد (Git بيرفض من غيرها كأمان).
`git clean -i`	Interactive: يسألك ملف ملف.
`git clean -fdx`	`-x` = حتى المتجاهَل في .gitignore: هيمسح `node_modules` و **.env**! استخدمها لو فاهم.

حاجة مشتركة: -n و -f

الـ `-n` = جرّب من غير تنفيذ (Dry run) في `git clean` و `git add -n` و `git rm -n` و `git push -n`، زي `rsync -n` في كتاب الـ Terminal. والـ `-f` = Force في `git clean` و `git push -f` و `git branch -f`، زي `rm -f`. أي `-f` في Git = "فكّر مرتين".

6.7 خريطة قرار التراجع



شكل 6.3 خريطة القرار: ابدأ من فوق، وجاوب على السؤالين.

الموقف	الأمر	الخطورة
ارمي تعديلاتي على ملف	`git restore f`	بيمسح التعديل
ارمي كل التعديلات في المشروع	`git restore .`	بيمسح التعديلات
ألغي git add	`git restore --staged f`	آمن
غيّر رسالة آخر Commit / ضيف ملف نسيته	`git commit --amend`	آمن لو مش Pushed
ألغي آخر Commit وخلي التغييرات متجهزة	`git reset --soft HEAD~1`	آمن
ألغي آخر Commit وخلي التغييرات في الملفات	`git reset HEAD~1`	آمن
ارجع لـ Commit قديم وارمي كل حاجة بعده	`git reset --hard <hash>`	**خطر** على اللي مش متعمله Commit
ألغي Commit اتعمله Push	`git revert <hash>`	آمن

الموقف	الأمر	الخطورة
جيب ملف واحد من نسخة قديمة	`git restore --source=<hash> f`	بيكتب فوق الملف
امسح الملفات الجديدة اللي مش متابَعة	`git clean -n` ثم `-fd`	بيمسح نهائي
عملت reset غلط	`git reset --hard ORIG_HEAD` أو `git reflog`	بينقذك (الفصل 16)

جدول 6.3 عاوز إيه؟ استخدم إيه؟

ملخص الفصل

- سؤالين: الغلطة فين (ورشةStaging/Commit/)؟ واتعملها Push؟
- `git restore f` يرمي تعديل الورشة (مفيش رجوع)، و `--staged` يلغي الـ add بأمان.
- `--amend` يصلّح آخر Commit، بس بيعمل Hash جديد.
- `reset --soft` للـ Staging، و `--mixed` (الافتراضي) للورشة، و `--hard` يرمي.
- `revert` للي اتعمله Push، و `git clean -n` قبل `-f` دايماً.

صح ولا غلط؟

1. `git restore --staged f` بيمسح التعديلات اللي في الملف. صح / غلط
2. `git revert` بيمسح الـ Commit من التاريخ. صح / غلط
3. `git reset` من غير Flag بيشتغل كـ `--mixed`. صح / غلط

أسئلة مراجعة

1. عملت Commit ونسيت ملف، ولسه معملتش Push. إيه أسهل حل؟
2. اشرح الفرق بين `reset --soft` و `--mixed` و `--hard` من حيث الأماكن التلاتة.
3. زميلك عمل Pull لـ Commit فيه Bug انت عملته. تلغيه إزاي؟ وليه مش reset؟
4. إيه الفرق بين `git restore f` و `git clean -f`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل عدّل ملف وارمي التعديل بـ `git restore`. وبعدين عدّله، اعمل add، وشيله من الـ Staging بـ `--staged`، واتأكد إن التعديل لسه في الملف.
2. متوسط اعمل 3 Commits، وبعدين جمّعهم في Commit واحد بـ `reset --soft`. قارن `git log --oneline` قبل وبعد.
3. صعب اعمل Commit، وبعدين `git reset --hard HEAD~1`، وحاول ترجّعه بـ `git reset --hard ORIG_HEAD`. (ولو فشلت، استنى الفصل 16 وارجعلها بـ reflog).

07

المرحلة 3 · الـ Branches

الفصل السابع

الـ Branches و HEAD

عاوز تجرّب Navbar جديد أو تخلّي الـ AI يعيد كتابة ملف كامل، من غير ما تلمس النسخة الشغالة؟ ده بالظبط اللي الـ Branch معمول عشانه. هنفهم إن الـ Branch مجرد لافتة صغيرة على Commit، وإن HEAD هو سهم إنت واقف فين، وبعدين نعمل ونتنقل ونمسح Branches بأمان.

بعد الفصل ده هتكون عارف

- المشكلة اللي الـ Branches بتحلها
- إن الـ Branch مجرد مؤشر (لافتة) على Commit، مش نسخة من المشروع
- يعني إيه HEAD وإزاي بيتحرك
- git branch و git switch بكل الـ Flags المهمة
- إيه اللي بيحصل لتعديلاتك لما تتنقل بين Branches
- الـ Detached HEAD: يعني إيه ومتخافش منه
- مسح وتغيير أسماء الـ Branches بأمان

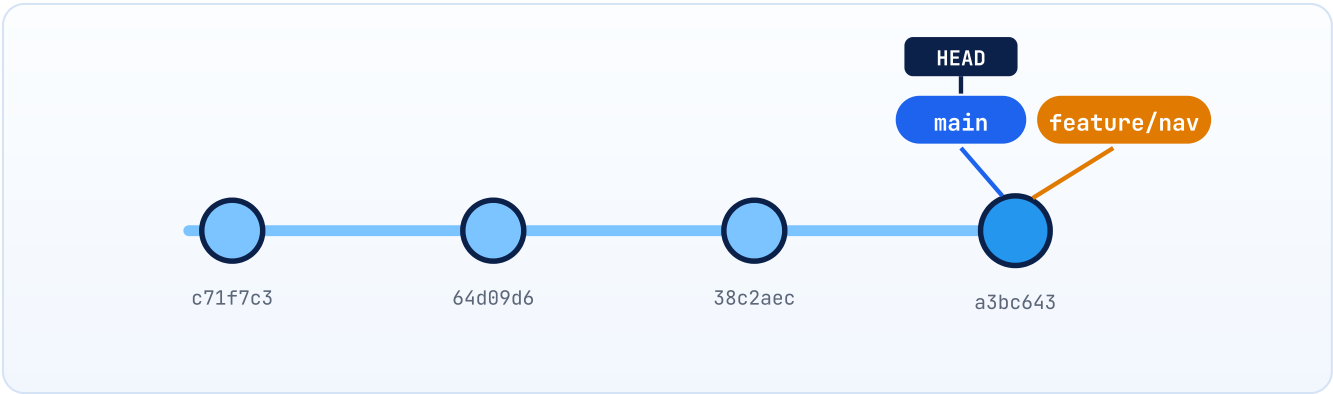
7.1 المشكلة: عاوز أجرّب من غير ما أبوّظ

موقع الـ Portfolio بتاعك شغال ومنشور. عاوز تعمل تصميم جديد للـ Navbar، وده هياخد كام يوم. في النص، اكتشفت Bug في صفحة Contact لازم يتصلح النهارده. لو كل شغلك على خط واحد، الإصلاح هيتخلط بتصميم الـ Navbar اللي لسه مش جاهز.

الحل: **خطوط شغل متوازية**. خط للـ Navbar، وخط للإصلاح، والخط الأساسي `main` فاضل نضيف وشغال. كل خط اسمه **Branch**.

7.2 الـ Branch = لافتة على Commit

ده أهم حاجة تفهمها: الـ Branch في Git **مش نسخة** من المشروع. هو مجرد **اسم بيشاور على Commit**، زي لافتة أو Sticky note. عشان كده عمل Branch جديد بياخد جزء من الثانية ومبياخدش مساحة. (فعلياً، كل Branch ملف صغير جوه `.git/refs/heads/` فيه 40 حرف: الـ Hash).



شكل 7.1 بعد git branch feature/nav: لافتتين على نفس الـ Commit، و HEAD على main.

الـ Branch

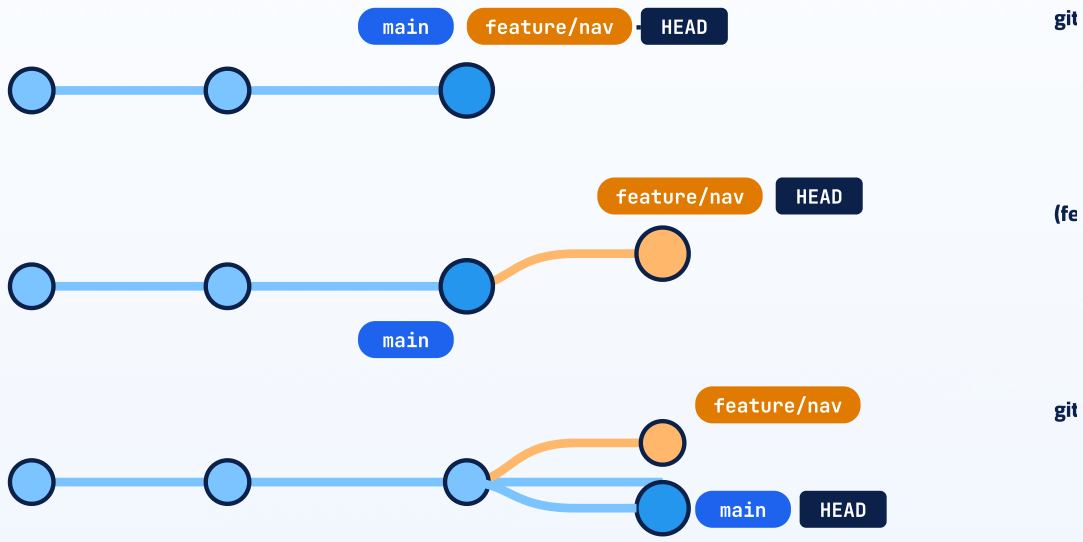
اسم بيشاور على آخر Commit في خط شغل. ولما تعمل Commit وانت عليه، **اللافتة بتتحرك لقدام لوحدها** للـ Commit الجديد.

HEAD

سهم "إنت هنا". عادةً بيشاور على Branch (مش على Commit مباشرة). الـ Branch اللي الـ HEAD عليه هو اللي هيتحرك مع الـ Commit الجاي.

7.3 HEAD بيتحرك إزاي

خلينا نتابع 3 لحظات:



شكل 7.2 الـ Branch اللي عليه HEAD هو بس اللي بيتحرك مع كل Commit، وكده الخطين اتفرعوا.

1. **تنقّلت لـ feature/nav:** HEAD بقى بيشاور عليه. الاتنين على نفس الـ Commit، فالملفات زي ما هي.
2. **عملت Commit:** اللافتة `feature/nav` اتحركت للـ Commit الجديد، و `main` فضلت مكانها.
3. **رجعت لـ main وعملت Commit:** main اتحرك في اتجاه تاني. دلوقتي الخطين **اتفرعوا** (Diverged)، وده بالظبط سبب كلمة Branch (فرع شجرة).

إزاي تعرف HEAD فين

`git status` أول سطر: `On branch main` . أو `git branch` (اللي جنبه `*`). أو `git log --oneline -1` هتلاقي `(HEAD -> main)` . وكمان في الـ Prompt لو بتستخدم oh-my-zsh أو Starship (من كتاب الـ Terminal)، اسم الـ Branch بيظهر على طول.

7.4 git branch: اعرض واعمل

```
Terminal
you@laptop:~/portfolio$ git branch
  feature/title
* main
  notes
you@laptop:~/portfolio$ git branch -vv
  feature/title 4ea67eb feat: change title
* main          e9db4ff [origin/main] docs: update readme
  notes         0dd1b27 docs: add notes
```

`git branch`	اعرض الـ Branches المحلية، و `*` = اللي إنت عليه.
`git branch -v`	Verbose: ومع كل واحد آخر Commit فيه. و `-vv` كمان بيوريك مربوط بأنهي Branch على GitHub (الفصل 10).
`git branch -a`	All: المحلية + اللي على الـ Remote.
`git branch feature/nav`	اعمل Branch جديد **من غير ما تتنقل له**.
`git branch --merged`	الـ Branches اللي اتدمجت خلاص في اللي إنت عليه (آمن تمسحها).

تسمية الـ Branches

استخدم أسماء بتوصف الشغل، بحروف صغيرة وشرطة: `feature/dark-mode` و `fix/contact-form` و `docs/readme` و `ai/refactor-api`. الـ `/` مجرد جزء من الاسم بيخلي الأدوات تجمّعهم، مش فولدر حقيقي. وابعد عن المسافات والحروف العربي في أسماء الـ Branches.

7.5 git switch: اتنقّل بين الـ Branches

تشريح الأمر

```
git switch -c feature/nav
```

1. `git switch` — غيّر الـ Branch اللي إنت عليه (حرّك HEAD)، وحدّث ملفات الورشة عشان تبقى زي آخر Commit فيه.
2. `-c` — Create: اعمل Branch جديد بالاسم ده الأول، وبعدين اتنقل له. من غيرها لازم يكون موجود.
3. `feature/nav` — الاسم. هيبدأ من الـ Commit اللي إنت عليه دلوقتي.

`git switch main`	اتنقل لـ Branch موجود.

`git switch -`	ارجع للـ Branch اللي كنت عليه قبل كده (زي `cd -` في كتاب الـ Terminal).
`git switch -c fix/form main`	اعمل Branch جديد يبدأ من `main` مهما كنت واقف فين.
`git switch -C feature/nav`	كبيرة = اعمله، ولو موجود **اكتب فوقه** (خطر شوية).

عاوز	الجديد	القديم
اتنقل لـ Branch	`git switch main`	`git checkout main`
اعمل واتنقل	`git switch -c new`	`git checkout -b new`
ارجع لملف قديم	`git restore f`	`git checkout -- f`

جدول 7.1 git switch مقابل git checkout

> **ليه أمرين جداد؟**
>
> `git checkout` كان بيعمل حاجات كتير مختلفة (يتنقل بين Branches ويرجّع ملفات)، وده كان بيلخبط الناس ويعمل كوارث. فـ Git 2.23 (2019) قسّمه لـ `switch` (للـ Branches) و `restore` (للملفات). استخدم الجداد، بس اعرف القديم لأنه في كل حتة على النت وفي اقتراحات الـ AI.

إيه اللي بيحصل لتعديلاتي لما أتنقل؟

Git بيغيّر الملفات في الورشة عشان تطابق الـ Branch الجديد. وتعديلاتك اللي مش متعملها Commit:

- **لو مفيش تعارض:** بتتنقل معاك للـ Branch الجديد كما هي (ودي بتلخبط ناس: التعديل مش "تبع" Branch لحد ما تعمله Commit).
- **لو الملف نفسه مختلف في الـ Branch التاني:** Git بيرفض عشان ميضيعش شغلك:

```
Terminal
you@laptop:~/portfolio$ git switch main
error: Your local changes to the following files would be overwritten by checkout:
        index.html
Please commit your changes or stash them before you switch branches.
Aborting
```

الحل: يا تعمل Commit للشغل ده على الـ Branch بتاعه، يا تعمل `git stash` (تشيله على جنب مؤقتاً، الفصل 14). وأحسن عادة: **اتنقل وانت** `working tree clean` .

7.6 الـ Detached HEAD: متتخضش

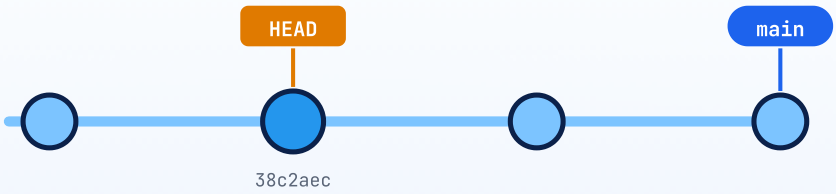
لو اتنقلت لـ **Commit** مش لـ Branch (مثلاً عشان تتفرج على نسخة قديمة)، HEAD بيشاور على الـ Commit مباشرة، من غير لافتة:

```
Terminal
you@laptop:~/portfolio$ git switch --detach 38c2aec
HEAD is now at 38c2aec fix: title text
you@laptop:~/portfolio$ git status
HEAD detached at 38c2aec
```

تشريح الأمر

```
git switch --detach 38c2aec
```

1. `git switch` حرّك HEAD.
2. `--detach` "عارف إن ده مش Branch، وده قصدي". `git switch` بيرفض يروح لـ Commit من غيرها كأمان، أما `git checkout 38c2aec` بيروح على طول ويطبعلك تحذير طويل.
3. `38c2aec` الـ Commit.



شكل 7.3 Detached HEAD: السهم على Commit قديم مباشرة، من غير Branch.

الوضع ده آمن تماماً للتفرج والتجربة. المشكلة الوحيدة: لو عملت Commits هنا ورجعت لـ main، الـ Commits دي ملهاش لافتة، وهتبقى "تايهة" (والـ reflog يقدر يلاقيها). الحل بسيط:

`git switch -`	مجرد تفرج؟ ارجع للـ Branch اللي كنت عليه.
`git switch -c experiment`	عملت Commits وعاوز تحتفظ بيها؟ اعمل Branch هنا، وهيبقى ليهم لافتة.

7.7 مسح وتغيير اسم الـ Branches

بعد شهر شغل، هتكتب `git branch` وتلاقي قايمة طويلة: Branches اتدمجت من زمان، وتجارب فشلت، وواحد اسمه `test2` مش فاكر عملته ليه. مسح الـ Branch مش بيمسح أي Commit اتدمج، هو بيشيل اللافتة بس. وعشان كده Git عنده نوعين مسح: واحد حذر بيرفض لو فيه شغل هيضيع، وواحد "أنا عارف أنا بعمل إيه":

تشريح الأمر

```
git branch -d feature/nav
 1      2  3
```

1. `git branch` إدارة الـ Branches.
2. `-d` Delete: امسح اللافتة، **بس لو** شغلها اتدمج (Merged). لو لأ، Git بيرفض: `error: the branch 'feature/nav' is not fully merged`.
3. `feature/nav` الـ Branch. ومينفعش تمسح الـ Branch اللي إنت عليه.

`git branch -D experiment`	كبيرة = امسح حتى لو مش متدمج (= `-d --force`). الـ Commits لسه في الـ reflog لفترة.
`git branch -m old-name new-name`	Move: غيّر الاسم. ومن غير الاسم القديم = غيّر اسم الحالي.
`git branch -m master main`	مثال مشهور: غيّر main لـ master في Repo قديم.

حاجة مشتركة: الحرف الصغير والكبير

في Git كتير: الصغير = النسخة الآمنة، والكبير = "متأكد، نفّذ". `-d` / `-D` في branch، و `-c` / `-C` في switch، و `-m` / `-M` في branch. نفس فكرة `rm -i` و `rm -f`.

مثال كامل: Branch للتجربة

عاوز تجرّب مكتبة Animation جديدة:

Terminal

```
git status                          # اتأكد إن مفيش تعديلات
git switch -c try/gsap              # خط تجربة جديد
# ... عدّل واعمل Commits ...
git switch main                     # ارجع للنسخة الشغالة، كأن مفيش حاجة حصلت
git branch -D try/gsap              # التجربة فشلت؟ ارميها
# أو: نجحت؟ ادمجها (الفصل الجاي)
```

ملخص الفصل

- الـ Branch = لافتة على Commit، بتتحرك لقدام لوحدها مع كل Commit.
- HEAD = إنت هنا، وعادةً بيشاور على Branch.
- `git switch -c name` تعمل وتتنقل، و `git switch -` ترجع، و `git branch -vv` تعرض.
- اتنقل وانت clean، وإلا Commit أو stash.
- Detached HEAD = على Commit من غير Branch، والحل `switch -` أو `switch -c`. و `-d` تمسح بأمان و `-D` بالقوة.

صح ولا غلط؟

1. عمل Branch جديد بينسخ كل ملفات المشروع. صح / غلط
2. لما تعمل Commit، كل الـ Branches بتتحرك لقدام. صح / غلط
3. `git branch -d` بيرفض يمسح Branch شغله لسه مش متدمج. صح / غلط

أسئلة مراجعة

1. اشرح بالرسم إيه اللي بيحصل لـ main و feature و HEAD لما تعمل Commit على feature.
2. إيه الفرق بين `git branch x` و `git switch -c x` ؟
3. يعني إيه Detached HEAD؟ وإزاي تحتفظ بـ Commits عملتها وانت فيه؟
4. ليه Git رفض يعمل switch وقالك `would be overwritten` ؟ واذكر حلين.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل `feature/footer`، اعمل عليه Commit، ارجع لـ main، وبص على الملفات: التعديل اختفى؟ ارجع تاني واتأكد إنه رجع.
2. متوسط اعمل Commit على main وكمان على feature/footer، وبعدين اعرض الشجرة بـ `git log --oneline --graph --all` واشرحها.
3. صعب روح Detached على Commit قديم، اعمل Commit تجريبي، ارجع لـ main من غير ما تعمل Branch. حاول تلاقي الـ Commit ده تاني بـ `git reflog` واعمله Branch.

08

المرحلة 3 · الـ Branches

الفصل التامن

الـ Merge وحل الـ Conflicts

خلصت الـ Navbar على Branch لوحده واشتغل. دلوقتي عاوز ترجّعه لـ main. ده اسمه Merge. أحياناً Git بيعملها لوحده في ثانية، وأحياناً بيقف ويقولك CONFLICT لأن الخطين عدّلوا نفس السطر. الفصل ده بيشرح نوعين الـ Merge بالرسم، وبيمشي معاك في حل الـ Conflict خطوة بخطوة من غير رعب.

بعد الفصل ده هتكون عارف

- ✓ git merge بيعمل إيه وبيتعمل من أنهي Branch
- ✓ الـ Fast-forward والـ 3-way merge بالرسم، والـ Merge Commit
- ✓ --no-ff و --ff-only وإمتى تستخدمهم
- ✓ ليه الـ Conflict بيحصل وشكل علامات الـ Conflict
- ✓ حل الـ Conflict خطوة بخطوة في الـ Terminal وفي VS Code
- ✓ git merge --abort، وإزاي تقلل الـ Conflicts من الأساس

8.1 git merge: رجّع الشغل للخط الأساسي

تخيل إنك بقالك يومين على `feature/nav`، والـ Navbar الجديد بقى تحفة: Dropdown وبيتقفل على الموبايل وكل حاجة. دلوقتي عاوزه يدخل النسخة الأساسية `main` عشان يتنشر. انت محتاج تقول لـ Git: "خد الشغل اللي على الخط ده وحطه في الخط ده". ده الـ Merge.

والقاعدة الأهم: إنت **بتقف على الـ Branch اللي عاوز تستقبل فيه**، وبتقول اسم الـ Branch اللي جاي منه.

تشريح الأمر

```
1          2  3          4
git switch main && git merge feature/nav
```

1	`git switch main`	اقف على المستقبِل (اللي عاوزه يتحدّث).
2	`&&`	لو نجح، كمّل.
3	`git merge`	هات التغييرات اللي في الـ Branch ده وادمجها في اللي أنا عليه.
4	`feature/nav`	المصدر. هو نفسه مش بيتغير، وتقدر تمسحه بعدها بـ `git branch -d`.

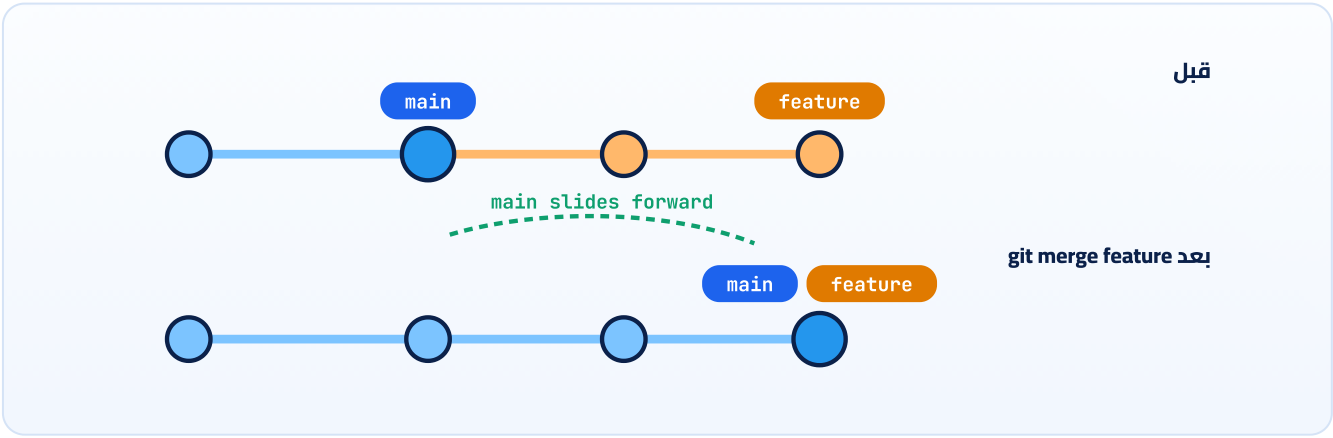
أشهر لخبطة

"ادمج main في feature" غير "ادمج feature في main". اللي إنت واقف عليه هو اللي بيتغير. لو مش متأكد، `git status` الأول.

وفيه نوعين Merge حسب شكل التاريخ:

8.2 النوع 1: Fast-forward

لو `main` متحركش من ساعة ما عملت `feature`، يبقى مفيش حاجة "تتدمج" فعلاً: Git بيحرّك لافتة main لقدام لحد آخر Commit في feature. خلاص.

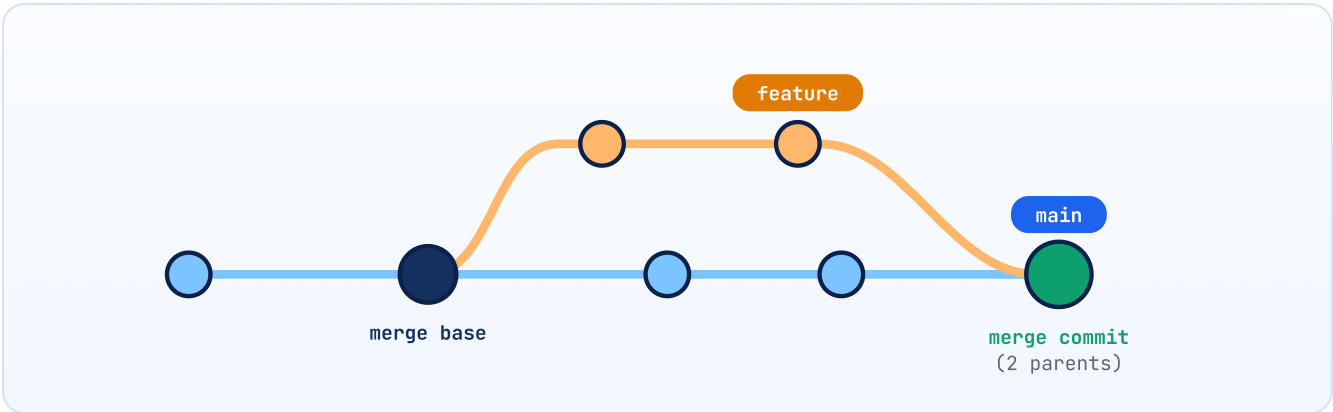


شكل 8.1 Fast-forward: مفيش Commit جديد، اللافتة بس اتزحلقت لقدام.

```
Terminal
you@laptop:~/portfolio$ git merge feature
Updating 48c8db7..a3bc643
Fast-forward
 nav.js | 12 ++++++++++++
 1 file changed, 12 insertions(+)
```

8.3 النوع 2: الـ 3-way Merge و Merge Commit

لو الخطين اتحركوا (عملت Commits على main وعلى feature)، Git مش هيقدر يزحلق. هيعمل **Merge Commit**: Commit جديد ليه **أبين**، بيجمع الاتنين. واسمه 3-way لأن Git بيقارن 3 نسخ: آخر Commit في main، وآخر Commit في feature، و**الجد المشترك** (Merge Base) اللي اتفرعوا منه.



شكل 8.2 3-way merge: Git قارن الجد المشترك بالطرفين، وعمل Commit جديد بأبين.

```
Terminal
you@laptop:~/portfolio$ git merge feature/title
Merge made by the 'ort' strategy.
 index.html | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

الـ `'ort'` ده اسم الخوارزمية اللي Git بيدمج بيها (الافتراضية من Git 2.34). ولو الـ Merge محتاج رسالة، Git هيفتح الـ Editor برسالة جاهزة `Merge branch 'feature/title'`، احفظ واقفل وخلاص.

التحكم في نوع الـ Merge

`git merge --no-ff feature`	No Fast-forward: اعمل Merge Commit حتى لو ينفع Fast-forward، عشان التاريخ يوضح إن فيه Feature اتدمجت كوحدة.
`git merge --ff-only feature`	ادمج بس لو Fast-forward، غير كده ارفض. آمن لما تكون عاوز تحدّث بس من غير Merge Commits.
`git merge --squash feature`	جمّع كل تغييرات feature في الـ Staging من غير Commit، وانت تعمل Commit واحد. (نفس فكرة زرار Squash على GitHub، الفصل 11).
`git merge feature -m "msg"`	رسالة الـ Merge Commit من غير Editor.

8.4 الـ Conflict: ليه بيحصل؟

الكلمة دي (CONFLICT) بحروف كبيرة حمرا بترعب أي حد أول مرة. بس خليني أطمنك بحكاية: تخيل انت وزميلك بتكتبوا نفس المقال في Google Doc. انت عدّلت العنوان، وهو عدّل آخر فقرة، مفيش مشكلة. لكن لو انتوا الاتنين غيرتوا **نفس الجملة** بشكلين مختلفين في نفس الوقت، مين فيكم الصح؟ محدش يقدر يقرر غير بني آدم.

Git بالظبط كده. هو ذكي جداً في الدمج: لو main عدّل سطر 5 و feature عدّل سطر 40، بيدمج لوحده. المشكلة بس لما **الاتنين يعدلوا نفس السطر** (أو واحد يعدّل ملف والتاني يمسحه). ساعتها Git مش هيخمن مين الصح، وبيقف ويسألك:

```
Terminal
you@laptop:~/portfolio$ git merge feature/title
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

> **الـ Conflict مش غلطة**
>
> الـ Conflict مش Bug ولا إنك عملت حاجة غلط. ده Git بيقولك بأدب: "فيه قرار محتاج بني آدم". ومحدش هيتأذي: الـ Merge واقف في النص، وتقدر تلغيه في أي لحظة بـ `git merge --abort`.

شكل الملف وقت الـ Conflict

Terminal

```
you@laptop:~/portfolio$ cat index.html
<<<<<<< HEAD
<h1>Hello</h1>
=======
<h1>Welcome</h1>
>>>>>>> feature/title
```

علامات الـ Conflict

`<<<<<<< HEAD`	بداية الـ Conflict. اللي تحتها = **نسختك** (الـ Branch اللي إنت واقف عليه، هنا main). اسمها "Current".
`<h1>Hello</h1>`	السطر زي ما هو في main.
`=======`	الفاصل بين النسختين.
`<h1>Welcome</h1>`	السطر زي ما هو في الـ Branch الجاي. اسمها "Incoming".
`>>>>>>> feature/title`	نهاية الـ Conflict، واسم الـ Branch الجاي.

8.5 حل الـ Conflict خطوة بخطوة

1. **متتخضش، واعرف مين فيه مشكلة:** `git status` هيوريك `both modified: index.html` تحت `Unmerged paths`.
2. **افتح الملف** ودوّر على `<<<<<<<` (ممكن يبقى فيه كذا Conflict في نفس الملف).
3. **قرر:** نسختك؟ ولا الجاية؟ ولا مزيج من الاتنين؟ ولا حاجة جديدة خالص؟ عدّل لحد ما الكود يبقى الشكل النهائي اللي عاوزه.
4. **امسح العلامات التلاتة** (`<<<` و `===` و `>>>`) كلها. لو نسيت واحدة، الكود هيبوظ.
5. **جرّب:** شغّل المشروع واتأكد إنه شغال.
6. **قول لـ Git إنك حليت:** `git add index.html`. الـ add هنا معناها "اتحل".
7. **كمّل الـ Merge:** `git commit` (هيفتح برسالة جاهزة) أو `git merge --continue`.

Terminal

```
you@laptop:~/portfolio$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
```

```
  (use "git add <file>..." to mark resolution)
        both modified:   index.html
you@laptop:~/portfolio$ code index.html            # عدّل وامسح العلامات
you@laptop:~/portfolio$ git add index.html
you@laptop:~/portfolio$ git commit --no-edit
[main 7b1e2c9] Merge branch 'feature/title'
```

في VS Code أسهل بكتير

VS Code بيلوّن الـ Conflict وبيحط فوقه أزرار: **Accept Current Change** (نسختك) و **Accept Incoming Change** (الجاية) و **Accept Both Changes** (الاتنين تحت بعض) و **Compare Changes**. وفيه كمان **Merge Editor** بـ 3 أعمدة (زرار `Resolve in Merge Editor`). بس افهم الأول العلامات بإيدك، عشان تعرف الزرايز بتعمل إيه.

الرسالة في status	يعني	الحل
`both modified`	الاتنين عدلوا نفس الحتة	عدّل الملف وامسح العلامات، ثم add
`both added`	الاتنين عملوا ملف بنفس الاسم	نفس الكلام
`deleted by them` / `deleted by us`	واحد مسح والتاني عدّل	عاوزه؟ `git add f` . مش عاوزه؟ `git rm f`

جدول 8.1 حالات الـ Conflict الشائعة

تشريح الأمر

```
git checkout --theirs -- package-lock.json
```

1. `git checkout` هنا بنستخدمه عشان يجيب نسخة ملف.
2. `--theirs` خد نسخة الـ **Branch الجاي** كلها للملف ده. وعكسها `--ours` = نسختك. (وتتكتب كمان `git restore --theirs`).
3. `--` اللي بعدي ملفات.
4. `package-lock.json` ملف مولّد مش هتحله بإيدك أبداً. بعدها الأحسن: `npm install` عشان يتولد صح، ثم `git add package-lock.json` .

8.6 خط الرجوع: git merge --abort

تشريح الأمر

```
git merge --abort
 1        2
```

1. `git merge` الـ Merge اللي شغال دلوقتي.
2. `--abort` الغيه، ورجّع كل حاجة زي ما كانت قبل ما تكتب `git merge`. مفيش أي ضرر. (لو كان عندك تعديلات مش متعملها Commit قبل الـ Merge، ممكن متعرفش ترجعها كويس، وعشان كده: Merge وانت clean).

حاجة مشتركة: --abort و --continue

أي عملية في Git ممكن تقف في النص عشان Conflict ليها الزوج ده: `git merge --abort / --continue`، و`git rebase --abort / --continue`، و`git cherry-pick --abort / --continue`، و`git revert --abort / --continue`. و`git status` دايماً بيقولك إنت جوه أنهي عملية.

8.7 إزاي تقلل الـ Conflicts

Branches قصيرة العمر

Branch بيعيش يوم أو اتنين أحسن من Branch عايش شهر. كل ما يطوّل، main بيتحرك أكتر والـ Conflicts بتكبر.

هات main كل شوية

وانت شغال على feature، ادمج main فيه من وقت للتاني (`git merge main`) عشان تحل Conflicts صغيرة بدري.

Commits صغيرة ومركّزة

متعملش "Format" للملف كله في نفس الـ Commit اللي فيه تعديل حقيقي: ده بيعمل Conflict في كل سطر.

اتفقوا على Formatter

Prettier بنفس الإعدادات للفريق كله، عشان محدش يغيّر المسافات والـ Quotes في سطور مش بتاعته.

مع الـ AI

أدوات الـ AI ساعات بتعيد كتابة ملف كامل عشان تغيير صغير. لو اتنين (أو إنت والـ AI على Branchين) عملوا كده، الـ Conflict هيبقى على الملف كله. اطلب من الـ AI تغييرات صغيرة ومحددة، وبص على `git diff --stat` (الفصل 18).

ملخص الفصل

- اقف على المستقبِل، و `git merge source`.
- Fast-forward = اللافتة بتتزحلق، و 3-way = Merge Commit بأبين.
- `--no-ff` دايماً Merge Commit، و `--ff-only` زحلقة بس أو ارفض.
- الـ Conflict: `<<<<<<< HEAD` نسختك، `=======` فاصل، `>>>>>>>` الجاي. عدّل، امسح العلامات، add، commit.
- `git merge --abort` يرجّع كل حاجة، و Branches قصيرة = Conflicts أقل.

صح ولا غلط؟

1. `git merge feature` وانت على main بيغيّر feature. صح / غلط
2. الـ Fast-forward بيعمل Merge Commit جديد. صح / غلط
3. بعد ما تحل الـ Conflict في الملف، لازم `git add` عشان تقول لـ Git إنه اتحل. صح / غلط

أسئلة مراجعة

1. إمتى Git يعمل Fast-forward وإمتى 3-way؟ ارسم الحالتين.
2. إيه هو الـ Merge Base؟ وليه Git محتاجه؟
3. اشرح كل علامة من علامات الـ Conflict.
4. اكتب الخطوات من أول ما تشوف CONFLICT لحد ما الـ Merge يخلص.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Branch، اعمل عليه Commit، ارجع لـ main وادمجه. اتأكد إن الرسالة `Fast-forward`.
2. متوسط اعمل Conflict بإيدك: نفس السطر في `index.html` اتعدل بشكلين على Branchين. ادمج وحل الـ Conflict بمزيج من النسختين.
3. صعب اعمل Conflict تاني، والمرة دي الغيه بـ `--abort`. وبعدين أعد الـ Merge وحل بـ `--theirs` للملف كله. قارن `git log --graph` في الحالتين.

09

المرحلة ٤ · GitHub والتعاون

الفصل التاسع

GitHub والـ Remotes والدخول بأمان

لحد دلوقتي كل حاجة على جهازك. لو الـ Laptop وقع أو اتسرق، التاريخ كله راح. ولو عاوز تشارك مشروعك أو تنشره، محتاج نسخة على النت. هنا ييجي GitHub. الفصل ده عن الحساب، وفكرة الـ Remote، وأهم حتة بتوقع المبتدئين: إزاي جهازك يثبت لـ GitHub إنك إنت، بـ SSH أو بـ HTTPS و Token.

بعد الفصل ده هتكون عارف

- ✅ عمل حساب GitHub وتأمينه بـ FA2
- ✅ عمل Repo على GitHub وربطه بمشروع موجود
- ✅ يعني إيه Remote و origin، و git remote بكل أشكاله
- ✅ الفرق بين روابط HTTPS و SSH
- ✅ عمل مفتاح SSH وإضافته لـ GitHub واختباره
- ✅ HTTPS على الـ Mac بـ gh أو Keychain و Token، وليه الباسورد مبقاش ينفع

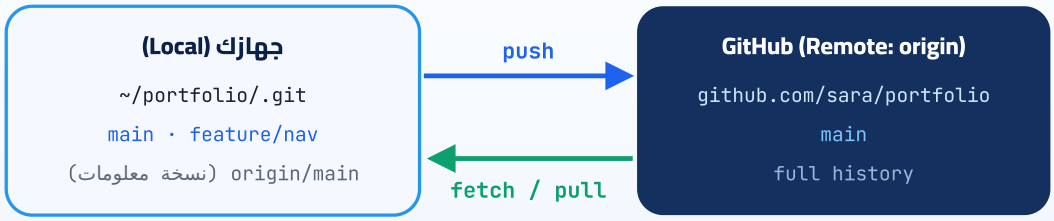
9.1 الحساب على GitHub

1. **اعمل حساب** على github.com. اختار Username محترم، لأنه هيبقى في روابط مشاريعك وموقعك (`sara.github.io`).
2. **فعّل FA2** (Two-factor authentication) من **Settings ← Password and authentication**، بتطبيق Authenticator أو Passkey. GitHub بيلزم بيه أي حد بيساهم بكود، فعّله من الأول واحفظ الـ Recovery codes في مكان آمن.
3. **ظبّط الإيميل الخاص** لو عاوز (↩ 2.3 ص 15).

> **الخطة المجانية كفاية جداً**
>
> الـ Free plan فيه Repos عامة وخاصة بلا عدد، و GitHub Pages للـ Repos العامة، و GitHub Actions مجانية للـ Repos العامة (وللخاصة فيه دقايق شهرية محدودة). يعني كل اللي في الكتاب ده ببلاش.

9.2 يعني إيه Remote؟

Remote = نسخة تانية من نفس الـ Repo في مكان تاني (غالباً على GitHub)، ليها اسم مختصر عندك. والاسم المتعارف عليه للنسخة الأساسية: `origin`. يعني `origin` مش كلمة سحرية، مجرد Nickname بدل ما تكتب الرابط الطويل كل مرة.



شكل 9.1 الـ Local والـ Remote: نسختين كاملتين، والـ push والـ pull بيزامنوهم.

ربط مشروع موجود بـ Repo جديد على GitHub

1. على GitHub: **New repository**، اكتب الاسم (`portfolio`)، اختار Public أو Private، و**متختارش** README أو gitignore. لو المشروع عندك فيه Commits (عشان الـ Repo يبدأ فاضي ومحصلش تعارض).
2. GitHub هيوريك الرابط. انسخ رابط SSH أو HTTPS (تحت).
3. عندك في الـ Terminal:

تشريح الأمر

```
git remote add origin git@github.com:sara/portfolio.git
```

1. `git remote` — إدارة الـ Remotes.
2. `add` — ضيف Remote جديد.
3. `origin` — الاسم المختصر اللي هتستخدمه بعد كده.
4. `git@github.com:sara/portfolio.git` — الرابط (هنا SSH). `sara` = صاحب الـ Repo، و `portfolio.git` = اسمه.

وبعدها أول Push (هنشرحه بالتفصيل في الفصل الجاي): `git push -u origin main`.

`git remote -v`	Verbose: اعرض الـ Remotes وروابطهم (سطر للـ fetch وسطر للـ push).
`git remote set-url origin <url>`	غيّر رابط Remote (مثلاً من SSH لـ HTTPS، أو لو غيرت اسم الـ Repo).
`git remote rename origin old`	غيّر الاسم المختصر.
`git remote remove old`	امسح الربط (مش بيمسح حاجة على GitHub).

```
Terminal
you@laptop:~/portfolio$ git remote -v
origin  git@github.com:sara/portfolio.git (fetch)
origin  git@github.com:sara/portfolio.git (push)
```

9.3 HTTPS ولا SSH؟

هنا بقى الحتة اللي بتوقّع ناس كتير. انت عملت Repo على GitHub، وربطته، وكتبت `git push` بكل ثقة، فالـ Terminal سألك `Username`، كتبته، وبعدين `Password`، كتبت باسورد GitHub بتاعك... و `Authentication failed`. تجرّب تاني وتتأكد من الباسورد، ونفس الرسالة. الباسورد صح والله! المشكلة مش فيك: GitHub مبقاش بيقبل الباسورد خالص في أوامر Git. محتاج طريقة تانية يثبت بيها جهازك إنه انت.

كل Repo على GitHub ليه رابطين. الاتنين بيوصلوا لنفس المكان، الفرق في **إزاي بتثبت إنك إنت**:

	HTTPS	SSH
شكل الرابط	`https://github.com/sara/portfolio.git`	`git@github.com:sara/portfolio.git`
بتثبت نفسك بـ	Token أو تسجيل دخول من المتصفح (مش الباسورد)	مفتاح SSH على جهازك
الإعداد	أسهل في الأول (GCM أو gh بيعملوه)	خطوات مرة واحدة
بعد الإعداد	مفيش أسئلة لو الـ Credential helper شغال	مفيش أسئلة خالص
شبكات بتقفل البورتات	شغال دايماً (بورت 443)	محتاج بورت 22 (أو 443 عن طريق `ssh.github.com`)

جدول 9.1 HTTPS مقابل SSH

> ⚠ **دقة: الباسورد مبقاش بيشتغل**
>
> من **13 أغسطس 2021**، GitHub بطّل يقبل باسورد الحساب في أوامر Git على HTTPS. لو Terminal طلب منك `Password` وكتبت باسوردك، هيطلع `Authentication failed`. اللي بيتكتب مكان الباسورد هو **Personal Access Token**، أو الأحسن تسيب Git Credential Manager أو `gh` يتصرفوا.

9.4 الطريق 1: مفتاح SSH

افتكر من كتاب الـ Terminal (فصل الشبكات و SSH): المفتاح زوج: **Private** بيفضل على جهازك ومحدش يشوفه، و **Public** بتديه لأي حد (هنا GitHub). لو عندك مفتاح أصلاً (`ls ~/.ssh`) فيه `id_ed25519.pub`)، تقدر تستخدمه.

1 · اعمل المفتاح

تشريح الأمر

```
ssh-keygen -t ed25519 -C "sara@example.com"
```

1. `ssh-keygen` برنامج عمل مفاتيح SSH (جزء من OpenSSH، موجود على Mac و Linux و Git Bash و Windows 10+).
2. `-t ed25519` Type: نوع المفتاح. Ed25519 حديث وقصير وآمن، وهو اللي GitHub بينصح بيه.
3. `-C "sara@example.com"` Comment: تعليق بيتحط في آخر المفتاح العام عشان تعرفه بعدين. مش لازم يكون إيميل GitHub.

هيسألك فين يحفظه (Enter للافتراضي `~/.ssh/id_ed25519`)، وعلى **Passphrase**: باسورد للمفتاح نفسه، يحميه لو حد سرق الملف. حط واحد، والـ ssh-agent هيفتكره عشان متكتبوش كل مرة.

2 · خلي الـ ssh-agent يفتكره

Terminal

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

تشريح الأمر

```
eval "$(ssh-agent -s)"
```

1. `ssh-agent -s` شغّل الـ Agent (برنامج بيحفظ المفاتيح المفتوحة في الذاكرة)، و `-s` = اطبع أوامر بصيغة sh/bash بتظبط متغيرات البيئة.
2. `$( )` Command substitution: خد ناتج الأمر اللي جوه (من كتاب الـ Terminal).
3. `eval` نفّذ الناتج ده كأوامر في الـ Shell الحالي، فالمتغير `SSH_AUTH_SOCK` يتظبط.

على الـ Mac: خلي الـ Keychain يفتكر الـ Passphrase

عشان متكتبش الـ Passphrase كل ما تفتح الجهاز: `ssh-add --apple-use-keychain ~/.ssh/id_ed25519`، وضيف في `~/.ssh/config`:

~/.ssh/config

```
Host github.com
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_ed25519
```

`UseKeychain yes` دي خاصة بنسخة OpenSSH بتاعة Apple، ومعناها "هات الـ Passphrase من الـ Keychain"، و `AddKeysToAgent yes` = ضيف المفتاح للـ Agent أول ما يتستخدم. ولو بتستخدم الـ Mac بس، الإعداد ده كفاية ومش محتاج `eval` كل مرة لأن macOS بيشغّل الـ ssh-agent لوحده.

3 · حط المفتاح العام على GitHub

Terminal

```
you@laptop:~/portfolio$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG3x...k9 sara@example.com
```

انسخ السطر ده كله (**الـ `.pub` بس!**)، وروح GitHub: **Settings ← SSH and GPG keys ← New SSH key**، اختار Type: Authentication Key، والصقه واحفظ.

> ⚠ **خلي بالك**
>
> الملف اللي **من غير** `.pub` (`id_ed25519`) ده المفتاح الخاص. ده متبعتهوش لحد، ومتلصقوش في أي موقع أو Chat (ولا لـ AI)، ومتعملوش Commit أبداً.

4 · اختبر

Terminal

```
you@laptop:~/portfolio$ ssh -T git@github.com
Hi sara! You've successfully authenticated, but GitHub does not provide shell access.
```

تشريح الأمر

```
ssh -T git@github.com
1   2  3
```

1. `ssh` — اتصل.
2. `-T` — متطلبش Terminal (TTY) على الناحية التانية، لأن GitHub مش بيديك Shell، إحنا بنختبر الدخول بس.
3. `git@github.com` — اليوزر `git` (ثابت لكل الناس) على السيرفر `github.com`. GitHub بيعرفك من المفتاح مش من اليوزر.

أول مرة هيسألك `Are you sure you want to continue connecting (yes/no)?`: اكتب `yes` (اتأكد إن الـ Fingerprint زي اللي في توثيق GitHub). ولو الرسالة فيها `Hi sara!` يبقى تمام.

مشكلة 1 **(publickey) Permission denied**

الـ push أو `ssh -T` طلع `git@github.com: Permission denied (publickey).`

الحل: GitHub مش لاقي مفتاح يعرفه. اتأكد: (1) المفتاح العام متضاف في GitHub؟ (2) `ssh-add -l` بيعرض المفتاح؟ لو لأ `ssh-add ~/.ssh/id_ed25519`. (3) `ssh -vT git@github.com` (الـ `-v` Verbose) يوريك أنهي مفتاح بيجرّبه. (4) صلاحيات `~/.ssh` = 700 والمفتاح = 600.

9.5 الطريق 2: HTTPS على الـ Mac بالـ Keychain

لو مش عاوز تتعامل مع مفاتيح SSH، HTTPS سهل جداً على الـ Mac، والسر في حاجة اسمها **Credential helper**: برنامج صغير Git بيسأله "معاك بيانات دخول لـ github.com؟"، فيجيبها من مكان آمن بدل ما يسألك. وعلى الـ Mac المكان الآمن ده هو **Keychain**، نفس المكان اللي Safari بيحفظ فيه باسوردات المواقع.

الأسهل: gh يعمل كل حاجة

تشريح الأمر

```
1  2          3              4
gh auth login --git-protocol https --web
```

1. `gh` أداة GitHub الرسمية للـ Terminal (`brew install gh`)، وهنشرحها في الفصل 17).
2. `auth login` سجّل دخول.
3. `--git-protocol https` خلي `git` يستخدم HTTPS (أو `ssh`، وساعتها `gh` ممكن يعملك مفتاح SSH ويرفعه بنفسه).
4. `--web` سجّل من المتصفح: هيديك كود من 8 حروف، تفتح الصفحة وتكتبه وتوافق. ومن غير ما تلصق أي Token.

بعدها `gh` بيحفظ الدخول في الـ Keychain، وبيسجّل نفسه كـ Credential helper لـ Git على github.com. يعني `git push` هيشتغل على طول.

من غير gh: osxkeychain

Git على الـ Mac (نسخة Apple ونسخة Homebrew الاتنين) جاي معاه helper اسمه `osxkeychain`، وغالباً متفعّل من الأول. اتأكد:

Terminal

```
you@laptop:~/portfolio$ git config --show-origin credential.helper
file:/opt/homebrew/etc/gitconfig        osxkeychain
```

تشريح الأمر

```
git config --global credential.helper osxkeychain
```

1 `credential.helper` البرنامج اللي Git يسأله عن بيانات الدخول.

2 `osxkeychain` خزّنها في Keychain بتاع macOS. أول push هيسألك Username وPassword: في الـ Password **الصق** **Token** (تحت) مش الباسورد، والـ Keychain هيفتكره.

Personal Access Token (لو محتاجه يدوي)

لو محتاج Token (مع osxkeychain، أو لسيرفر أو أداة)، اعمله من **Settings ← Developer settings ← Personal access tokens**. فيه نوعين:

النوع	الميزة	نصيحة
Fine-grained	بتحدد Repos معينة وصلاحيات معينة (مثلاً Contents: Read and write)	**المفضّل**، وحط له تاريخ انتهاء
Classic	صلاحيات واسعة على كل الـ Repos (`repo`)	بس لو أداة مش بتدعم الجديد

جدول 9.2 أنواع الـ Tokens

عاملها زي الباسورد

الـ Token يتشاف مرة واحدة: احفظه في Password manager. أقل صلاحيات تكفي، وتاريخ انتهاء. ومتحطهوش في كود ولا في رابط الـ Remote (`https://TOKEN@github.com/...`) لأنه بيتحفظ نص عادي في `.git/config`.

لو حابب تتعمق: الـ Token اتغير أو الـ Keychain فاكر حاجة غلط

لو غيرت الـ Token والـ Keychain لسه بيبعت القديم (وتطلع `Authentication failed`)، امسحه: افتح **Keychain Access** ودوّر على `github.com` وامسح الـ Internet password، أو من الـ Terminal: `printf "protocol=https\nhost=github.com\n" | git credential-osxkeychain erase`. الأمر ده بيكلّم الـ helper بنفس البروتوكول اللي Git بيستخدمه: سطور `key=value` على الـ stdin (افتكر الـ Pipes من كتاب الـ Terminal). وعلى Windows، نفس الدور بيعمله **Git Credential Manager** اللي بييجي مع Git for Windows.

حاجة مشتركة: كتاب الـ Terminal

كل اللي في فصل SSH هناك شغال هنا: `~/.ssh/config` و `ssh-add` والصلاحيات 700/600 و `-v` للتشخيص. والفرق الوحيد إن GitHub مش بيديك Shell.

ملخص الفصل

- Remote = نسخة تانية من الـ Repo، و `origin` مجرد اسم مختصر للرابط.
- `git remote add origin URL` و `-v` و `set-url`.
- HTTPS على الـ Mac: `gh auth login` أو `osxkeychain` مع Token، والباسورد العادي مبقاش بيشتغل من 2021.
- SSH: `ssh-keygen -t ed25519` ← `ssh-add` ← الـ .pub على GitHub ← `ssh -T git@github.com`.
- المفتاح الخاص والـ Token أسرار: مفيش Commit، ولا لصق في Chat.

صح ولا غلط؟

1. `origin` اسم محجوز في Git ومينفعش تغيره. صح / غلط
2. بتحط المفتاح اللي من غير `.pub` على GitHub. صح / غلط
3. لو `ssh -T` قالك `does not provide shell access` يبقى فيه مشكلة. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين رابط HTTPS ورابط SSH لنفس الـ Repo؟
2. ليه بنحط Passphrase على مفتاح SSH؟
3. عندك Repo مربوط بـ HTTPS وعاوز تحوّله لـ SSH. أنهي أمر؟
4. إيه الفرق بين Fine-grained token و Classic؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل حساب GitHub، فعّل FA2، واعمل Repo فاضي اسمه `git-playground`.
2. متوسط اعمل مفتاح SSH، ضيفه على GitHub، واختبره بـ `ssh -T`.
3. صعب اربط الـ playground بالـ Repo على GitHub بـ HTTPS الأول، وبعدين حوّله لـ SSH بـ `set-url`، واتأكد بـ `git remote -v`.

10

المرحلة 4 · GitHub والتعاون

الفصل العاشر

clone و push و fetch و pull

عندك Repo على جهازك ونسخته على GitHub. محتاج تبعت شغلك، وتجيب شغل غيرك، وتبدأ من Repo موجود. أربع أوامر: clone تجيب Repo لأول مرة، و push تبعت، و fetch تجيب المعلومات من غير ما تلمس شغلك، و pull تجيب وتدمج. وهنفهم الـ Upstream ورسالة rejected الشهيرة.

بعد الفصل ده هتكون عارف

- git clone وإيه اللي بيعمله بالظبط
- git push و -u والـ Upstream (Tracking branch)
- الـ Remote-tracking branches زي origin/main
- الفرق بين fetch و pull بالرسم
- قراية ahead و behind و diverged
- حل رسالة rejected (fetch first) وخطأ divergent branches

خليني أحكيلك عن يوم عادي لما تبقى شغال على مشروع مع حد. الصبح بتفتح الـ Mac، وأول حاجة محتاج تعرفها: "زميلي عمل إيه امبارح بالليل؟". تجيب شغله. تشتغل شوية، وبعدين تبعت شغلك عشان هو كمان يشوفه. وفي يوم جه واحد جديد للفريق، محتاج ياخد نسخة من المشروع كله لأول مرة. التلات مواقف دول هم كل اللي الفصل ده عنه، وكل موقف ليه أمر. ونبدأ باللي جديد:

10.1 git clone: هات Repo لأول مرة

تشريح الأمر

```
git clone git@github.com:sara/portfolio.git my-site
```

1	`git clone`	انسخ Repo كامل (كل التاريخ) من Remote لجهازك.
2	`git@github.com:sara/portfolio.git`	الرابط (SSH أو HTTPS، زرار **Code** الأخضر على GitHub).
3	`my-site`	اختياري: اسم الفولدر. من غيره هياخد اسم الـ Repo (`portfolio`).

الـ clone بيعمل 4 حاجات في أمر واحد: يعمل الفولدر، و `git init` جواه، ويضيف Remote اسمه `origin` بالرابط، ويجيب كل التاريخ ويعمل `switch` للـ Branch الافتراضي (main). يعني مش محتاج `git remote add` بعد الـ clone.

بعد clone لمشروع JS

`node_modules` مش موجود (افتكر الـ .gitignore). أول حاجة: `cd my-site` ثم `npm install`. ولو فيه `.env.example`، انسخه لـ `.env` وحط قيمك.

`git clone --depth 1 URL`	Shallow: آخر Commit بس من غير التاريخ. أسرع للمشاريع الضخمة لو عاوز تجرّب بس.
`git clone -b dev URL`	Branch: ابدأ على Branch معين بدل الافتراضي.

10.2 git push: ابعت شغلك

تشريح الأمر

```
git push -u origin main
 1    2  3     4
```

1. `git push` ابعت الـ Commits اللي عندك ومش على الـ Remote.
2. `-u` اختصار `--set-upstream`: افتكر إن `main` المحلي مربوط بـ `main` على origin. بتعملها **أول مرة بس** لكل Branch.
3. `origin` لأنهي Remote.
4. `main` أنهي Branch. وبيتبعت لـ Branch بنفس الاسم على GitHub (ولو مش موجود بيتعمل).

Terminal

```
you@laptop:~/portfolio$ git push -u origin main
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 512 bytes | 512.00 KiB/s, done.
To github.com:sara/portfolio.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

قراية ناتج push

`* [new branch]  main -> main`	Branch جديد اتعمل على GitHub: `main` المحلي ← `main` البعيد.
`branch 'main' set up to track 'origin/main'.`	ده بتاع `-u`: الربط اتعمل. من دلوقتي `git push` و `git pull` من غير أي حاجة.
`a3bc643..25190db  main -> main`	ده شكل push عادي بعد كده: الـ Remote كان عند `a3bc643` وبقى عند `25190db`.

يعني إيه Upstream؟

كل Branch محلي ممكن يبقى "متابع" Branch على الـ Remote، اسمه **Upstream** أو Tracking branch. ده بيخلي Git يعرف يقارن ويقولك ahead و behind، ويخليك تكتب `git push` و `git pull` من غير تفاصيل. شوف الربط بـ `git branch -vv`.

`git push`	بعد `-u`: ابعت الـ Branch الحالي للـ Upstream بتاعه.

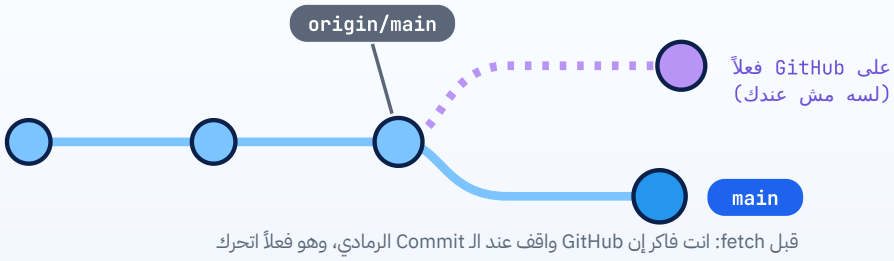
`git push -u origin feature/nav`	أول push لـ Branch جديد (عشان تعمل بيه Pull Request، الفصل 11).
`git push origin --delete feature/nav`	امسح Branch من GitHub (المحلي مش بيتلمس).
`git push --tags`	ابعت الـ Tags (الفصل 14)، لأن push العادي مش بيبعتها.

خليها أسهل: push.autoSetupRemote

`git config --global push.autoSetupRemote true` (من Git 2.37): أول `git push` لأي Branch جديد يعمل الـ `-u` لوحده. مش هتشوف تاني رسالة `has no upstream branch`.

10.3 origin/main: صورة آخر مرة شفت فيها GitHub

دي أكتر حتة بتلخبط في الموضوع ده كله، فخليني أشبّهها: تخيل إنك بتتابع ماتش على تويتر مش على التلفزيون. آخر تويتة شفتها قالت "النتيجة 1-0". ده مش معناه إن النتيجة دلوقتي 1-0، معناه إن دي **آخر معلومة وصلتك**. لحد ما تعمل Refresh. جوه الـ Repo عندك فيه Branches خاصة اسمها **Remote-tracking branches** زي `origin/main`. دي مش GitHub نفسه، دي **آخر معلومة** عندك عن مكان main على GitHub. مبتتحدثش لوحدها، بتتحدث بس مع `fetch` و `pull` و `push`. ومينفعش تعمل عليها Commits.



شكل 10.1 origin/main = معلومتك القديمة عن GitHub لحد ما تعمل fetch.

10.4 git fetch: هات المعلومات بس

تشريح الأمر

```
git fetch origin
    1     2
```

1. `git fetch` نزّل أي Commits جديدة من الـ Remote، وحدّث `origin/main` وأخواتها. **مش بيلمس** الـ Branches بتاعتك ولا ملفاتك خالص. آمن 100%.
2. `origin` من أنهي Remote. من غيرها = origin.

بعد الـ fetch تقدر "تتفرج" قبل ما تدمج:

Terminal

```
git fetch
git status                          # ahead/behind
git log --oneline main..origin/main   # اللي جديد على GitHub
git diff main origin/main --stat      # الملفات اللي هتتغير
```

تشريح الأمر

```
git log --oneline main..origin/main
```

1. `git log --oneline` تاريخ مختصر.
2. `main..origin/main` النقطتين = "الـ Commits اللي في `origin/main` ومش في `main`". يعني اللي هتاخده لو عملت pull. والعكس `origin/main..main` = اللي عندك ولسه ما اتبعتش.

10.5 merge + fetch = git pull

تشريح الأمر

```
git pull
```

1. `git pull` خطوتين: `git fetch` للـ Upstream، وبعدين يدمج `origin/main` في `main` بتاعك (Merge، أو Rebase لو ظبطت كده، الفصل 15). لو انت مغيرتش حاجة، الدمج هيبقى Fast-forward.

	`git fetch`	`git pull`
بيجيب الـ Commits الجديدة	أيوه	أيوه
بيحدّث origin/main	أيوه	أيوه
بيغيّر main والملفات بتاعتك	لأ	أيوه (بيدمج)
ممكن يعمل Conflict	لأ	أيوه
استخدمه لما	عاوز تشوف الأول	عاوز تتحدث على طول

جدول 10.1 fetch مقابل pull

العادة الصح

ابدأ يومك بـ `git pull` وانت على main وفي حالة clean. وقبل ما تعمل Branch جديد: `git switch main && git pull && git switch -c feature/x`، عشان تبدأ من آخر نسخة.

10.6 ahead و behind و diverged

```
Terminal
you@laptop:~/portfolio$ git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)
```

الرسالة	يعني	تعمل إيه
`up to date with 'origin/main'`	متطابقين (حسب آخر fetch)	ولا حاجة
`ahead of 'origin/main' by 2 commits`	عندك 2 لسه ما اتبعتوش	`git push`
`behind 'origin/main' by 3 commits`	GitHub عنده 3 جداد	`git pull`
`have diverged`	الاتنين عندهم Commits جديدة	`git pull` (دمج) وبعدين `git push`

جدول 10.2 رسايل الـ Upstream في git status

10.7 لما الـ push يترفض

```
Terminal
you@laptop:~/portfolio$ git push
To github.com:sara/portfolio.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:sara/portfolio.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

معناها: حد (أو إنت من جهاز تاني، أو من موقع GitHub نفسه لما عدلت README) عمل push قبلك. Git مش هيمسح شغله عشان شغلك. الحل: `git pull` (حل أي Conflict لو ظهر) وبعدين `git push` تاني.

مشكلة 1 **fatal: Need to specify how to reconcile divergent branches**

عملت `git pull` وطلعلك `hint: You have divergent branches` وقايمة بـ 3 اختيارات.

الحل: Git محتاج تقوله يدمج إزاي مرة واحدة. للمبتدئين: `git config --global pull.rebase false` (Merge)، أو لمرة واحدة `git pull --no-rebase` . (لو ظبطته في الفصل 2 مش هتشوفها). والاختيارات التانية: `--rebase` (الفصل 15) و `--ff-only` (ارفض لو مش Fast-forward).

مشكلة 2 **has no upstream branch**

عملت `git push` على Branch جديد وطلع

```
fatal: The current branch feature/nav has no upstream branch.
```

الحل: أول push محتاج `-u` : `git push -u origin feature/nav` (Git نفسه بيكتبهولك في الرسالة). أو ظبّط `push.autoSetupRemote`.

`git push --force`: مش حل

لو قريت على النت "اعمل `git push -f`"، ده معناه "امسح اللي على GitHub وحط اللي عندي مكانه". لو حد تاني عمل push، شغله هيضيع. استخدامه الوحيد المقبول: على Branch بتاعك إنت لوحدك بعد rebase أو amend، وبالنسخة الأأمن `--force-with-lease` (الفصل 15).

ملخص الفصل

- `git clone URL` = mkdir + init + remote add origin + fetch + switch.
- `git push -u origin main` أول مرة، وبعدين `git push` وبس.
- `origin/main` = آخر معلومة عندك عن GitHub، وبتتحدث بالـ fetch.
- `fetch` يجيب بس (آمن)، و `pull` = fetch + merge.
- `rejected (fetch first)` = اعمل pull الأول، و `--force` مش الحل.

صح ولا غلط؟

1. `git fetch` ممكن يغيّر ملفات في الورشة. صح / غلط
2. بعد `git clone` لازم تعمل `git remote add origin` . صح / غلط
3. `origin/main` بيتحدث لوحده كل ما حد يعمل push على GitHub. صح / غلط

أسئلة مراجعة

1. إيه الـ 4 حاجات اللي `git clone` بيعملها؟
2. اشرح `-u` في `git push -u origin main` .
3. إيه الفرق بين `main..origin/main` و `origin/main..main` ؟
4. الـ push اترفض بـ `fetch first` . إيه السبب والحل؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ابعت الـ playground لـ GitHub بـ `push -u`، وافتح صفحة الـ Repo واتأكد إن الـ Commits ظهرت.
2. متوسط عدّل README من موقع GitHub نفسه (زرار القلم) واعمل Commit هناك. عندك: اعمل `git fetch` ثم `git status` ثم `git log main..origin/main`، وبعدين `git pull`.
3. صعب اعمل `git clone` للـ Repo في فولدر تاني (كأنه جهاز تاني). اعمل Commit من كل نسخة على نفس الملف، و push من الأولى، وبعدين حاول push من التانية، وحل الموقف لحد ما الاتنين يتطابقوا.

11

المرحلة 4 · GitHub والتعاون

الفصل الحادي عشر

التعاون: Pull Requests و Forks و Issues

GitHub مش مجرد مكان تحفظ فيه الكود، ده مكان الناس بتشتغل فيه مع بعض. الـ Pull Request هو الطريقة اللي بتقول بيها: خلصت شغل على Branch، ممكن حد يراجعه قبل ما يدخل main؟ والـ Fork هو إزاي تساهم في مشروع مش بتاعك. والـ Issues هي قايمة المهام والمشاكل. وكل ده مفيد حتى لو بتشتغل لوحدك.

بعد الفصل ده هتكون عارف

- يعني إيه Pull Request وإزاي تعمله خطوة بخطوة
- صفحة الـ PR: Conversation و Commits و Checks و Files changed
- الـ Code Review: تعليقات و Suggestions و Approve و Request changes
- طرق الدمج التلاتة على GitHub: Merge و Squash و Rebase
- الـ Fork والـ upstream remote ومزامنة الـ Fork
- الـ Issues والـ Labels وربطها بالـ PR بـ Closes #12

11.1 الـ Pull Request: "ممكن تدمجوا شغلي؟"

تخيل إنك خلصت الـ Contact form في البورتفوليو. شكله حلو، والـ Validation شغالة، فدمجته في main وعملت push وقفلت اللابتوب مبسوط. تاني يوم صاحبك بيقولك: "بعتلك رسالة من الموقع بس قالي الإيميل غلط". تفتح الكود تلاقي إن الـ Validation بترفض أي إيميل فيه `+` أو حروف كبيرة. غلطة صغيرة، وكان أي حد بصّ على الكود دقيقتين هيشوفها. بس محدش شاف، لأن الكود راح من دماغك لـ main على طول.

الشركات اتعلمت الدرس ده من زمان، وعشان كده مفيش كود بيدخل main من غير ما يعدّي على "بوابة": صفحة فيها التعديلات كلها، وحد تاني بيقراها ويعلّق، واختبارات بتشتغل لوحدها، وبعد كده بس زرار الدمج. البوابة دي اسمها **Pull Request**.

الـ **Pull Request** (PR) = طلب على GitHub إن Branch يتدمج في Branch تاني (غالباً main)، ومعاه صفحة نقاش ومراجعة. الاسم معناه "اسحب (Pull) تغييراتي عندك". (في GitLab اسمه Merge Request، نفس الفكرة). وخد بالك: الـ PR مش حاجة في Git نفسه، دي ميزة في GitHub مبنية فوق الـ Branches اللي انت فاهمها خلاص من الفصل 7.

> **ليه PR حتى لو شغال لوحدك؟**
>
> الـ PR بيوريك الـ diff كله في صفحة واحدة مرتبة قبل ما يدخل main، وبيشغّل الاختبارات (Actions، الفصل 17)، وبيحفظ "ليه" عملت التغيير ده. وده كمان تمرين على الشغل الحقيقي: أي شركة هتشتغل فيها هتشتغل بالـ PRs.

خطوات أول PR

1. **ابدأ من آخر main:** `git switch main && git pull`.
2. **Branch للشغل:** `git switch -c feature/contact-form`.
3. **اشتغل واعمل Commits** صغيرة برسايل واضحة.
4. **ابعت الـ Branch:** `git push -u origin feature/contact-form`. GitHub هيطبعلك في الـ Terminal رابط `Create a pull request`.
5. **افتح الـ PR:** من الرابط، أو من الموقع زرار **pull request & Compare**. اختار **base: main** ← **compare: feature/contact-form**.
6. **اكتب عنوان ووصف:** عملت إيه، وليه، وإزاي تجرّبه، وصورة لو فيه تغيير في الشكل.
7. **Create pull request**، واستنى المراجعة (أو راجع بنفسك)، وبعدين **Merge**.
8. **نضّف:** امسح الـ Branch من GitHub (زرار Delete branch)، وعندك: `git switch main && git pull && git branch -d feature/contact-form`.

أول مرة تعمل push لـ Branch جديد، GitHub نفسه بيرد عليك في الـ Terminal ويقولك "تحب تفتح PR؟". السطور اللي بتبدأ بـ `remote:` دي جاية من GitHub مش من Git:

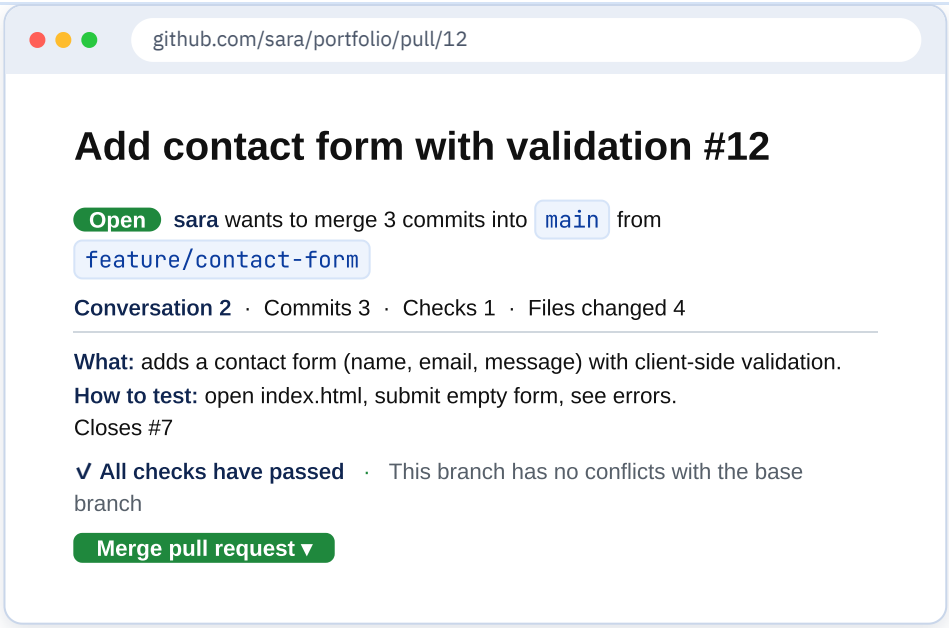
```
Terminal
you@laptop:~/portfolio$ git push -u origin feature/contact-form
...
remote: Create a pull request for 'feature/contact-form' on GitHub by visiting:
remote:      https://github.com/sara/portfolio/pull/new/feature/contact-form
To github.com:sara/portfolio.git
 * [new branch]      feature/contact-form -> feature/contact-form
```

11.2 صفحة الـ PR

فتحت الـ PR. الصفحة اللي قدامك دي هي أوضة الاجتماعات بتاعة التغيير ده: كل حاجة عنه في مكان واحد. فوق العنوان ورقم الـ PR (`#12`، وهو نفس عدّاد الـ Issues)، وتحته جملة بتقول مين عاوز يدمج كام Commit من أنهي Branch لأنهي Branch، وبعدين 4 تابات، وفي الآخر خالص زرار الدمج. وزرار الدمج ده مش بيبقى أخضر غير لما الـ Checks تعدّي ومفيش Conflicts.

التاب	فيه إيه
Conversation	الوصف والنقاش والمراجعات، وزرار الـ Merge تحت
Commits	الـ Commits اللي في الـ Branch ومش في main
Checks	نتايج GitHub Actions (اختبارات، Build)
Files changed	الـ diff كامل، وهنا بتعمل المراجعة سطر بسطر

جدول 11.1 تابات صفحة الـ PR



شكل 11.1 شكل توضيحي لصفحة PR على GitHub (رسمة مش Screenshot).

> **لو حابب تتعمق: الـ PR من جوه**
>
> GitHub بيحفظ كل PR كـ Ref مخفي على الـ Remote اسمه `refs/pull/12/head`. يعني تقدر تجيب أي PR لجهازك بـ Git عادي من غير `gh`: `git fetch origin pull/12/head:pr-12` ثم `git switch pr-12`. و `pull/12/head` = المصدر على GitHub، و `:pr-12` = اسم الـ Branch اللي هيتعمل عندك. ده بالظبط اللي `gh pr checkout 12` بيعمله ورا الكواليس (الفصل 17).

> **PR صغير = مراجعة حقيقية**
>
> PR فيه 50 سطر هيتراجع بجد. PR فيه 2000 سطر هياخد "LGTM" من غير ما حد يقراه. خلي كل PR يعمل حاجة واحدة. ولو لسه شغال وعاوز رأي بدري، اعمله **Draft pull request** (من السهم جنب زرار Create).

11.3 الـ Code Review

أول مرة حد يكتب تعليق على كودك، هتحس إنه بيقولك "انت مش فاهم". وده إحساس طبيعي جداً وبيروح بسرعة، لما تفهم إن المراجعة مش امتحان: دي اتنين بيبصوا على نفس الكود عشان يطلع أحسن. والمراجع اللي بيكتبلك 10 تعليقات غالباً اهتم بشغلك أكتر من اللي كتب "LGTM" (Looks Good To Me) من غير ما يقرا.

المراجع بيفتح **Files changed**، وبيدوس على `+` جنب أي سطر يكتب تعليق. وفي الآخر **Review changes** ويختار واحدة من 3:

النوع	معناه
Comment	ملاحظات عامة من غير قرار
Approve	"تمام، ادمج"
Request changes	"فيه حاجات لازم تتصلح قبل الدمج"

جدول 11.2 أنواع المراجعة

الـ Suggestion: اقترح الكود نفسه

وانت بتعلّق على سطر، فيه زرار **Add a suggestion** (أيقونة ±). بتكتب الكود المقترح جوه بلوك كده:

تعليق مراجعة

```
```suggestion
const isValid = email.includes("@") && email.length > 3;
```
```

وصاحب الـ PR يدوس **Commit suggestion** فيتعمل Commit على الـ Branch من غير ما يفتح الكود.

لما حد يطلب تعديلات

متقفلش الـ PR وتعمل واحد جديد. اعمل التعديلات على **نفس الـ Branch** عندك، Commit و `git push`، والـ PR هيتحدث لوحده. وبعدين رد على التعليق أو دوس **Resolve conversation**.

وانت بتتراجع

اسأل بدل ما تأمر ("إيه رأيك لو...؟")، وعلّق على الكود مش على الشخص، وقول الحاجة الحلوة كمان، وفرّق بين "لازم" و"اقتراح" (بيكتبوا `nit:` للحاجات الصغيرة).

وانت بتتراجع

المراجعة على الكود مش عليك. اشكر، واسأل لو مش فاهم، ولو مختلف اشرح ليه بهدوء. وكل تعليق اترد عليه أو اتصلح.

11.4 طرق الدمج على GitHub

الـ PR اتوافق عليه، والـ Checks خضرا، وانت رايح تدوس Merge. تلاحظ إن جنب الزرار فيه سهم صغير، ولو دوست عليه تلاقي 3 اختيارات. معظم الناس بتتجاهله وتدوس الافتراضي، بس الاختيار ده هو اللي بيحدد شكل تاريخ main بعد سنة: هل هيبقى مليان `wip` و `fix typo again`، ولا كل سطر فيه Feature كاملة؟

والحلو إنك فاهم الـ 3 خلاص، لأنهم نفس أفكار الفصل 8 (وواحدة من الفصل 15) بس بزرار:

الاختيار	بيعمل إيه في main	إمتى
Create a merge commit	كل الـ Commits + Merge Commit (زي `--no-ff`)	لما الـ Commits نضيفة وعاوز تاريخها
Squash and merge	كل الـ Commits تتجمع في **Commit واحد** على main	الأشهر في الفرق: الـ Branch فيه `wip` و `fix typo`
Rebase and merge	الـ Commits تتحط ورا بعض على main من غير Merge Commit	تاريخ مستقيم والـ Commits نضيفة (الفصل 15)

جدول 11.3 Merge مقابل Squash مقابل Rebase

لو حابب تتعمق: Squash بيعمل إيه بالظبط؟

GitHub بياخد الـ Snapshot النهائي للـ Branch، ويعمل منه Commit **جديد** على main ليه Parent واحد بس (آخر main). الرسالة الافتراضية = عنوان الـ PR + رقمه زي `feat: add contact form (#12)`، وتحتها رسايل الـ Commits القديمة كقايمة (تقدر تعدّلها قبل التأكيد). والـ Author = صاحب الـ PR. والـ Commits الأصلية بتفضل موجودة في صفحة الـ PR بس، مش في تاريخ main. ده بالظبط زي `git merge --squash` + `git commit` على جهازك.

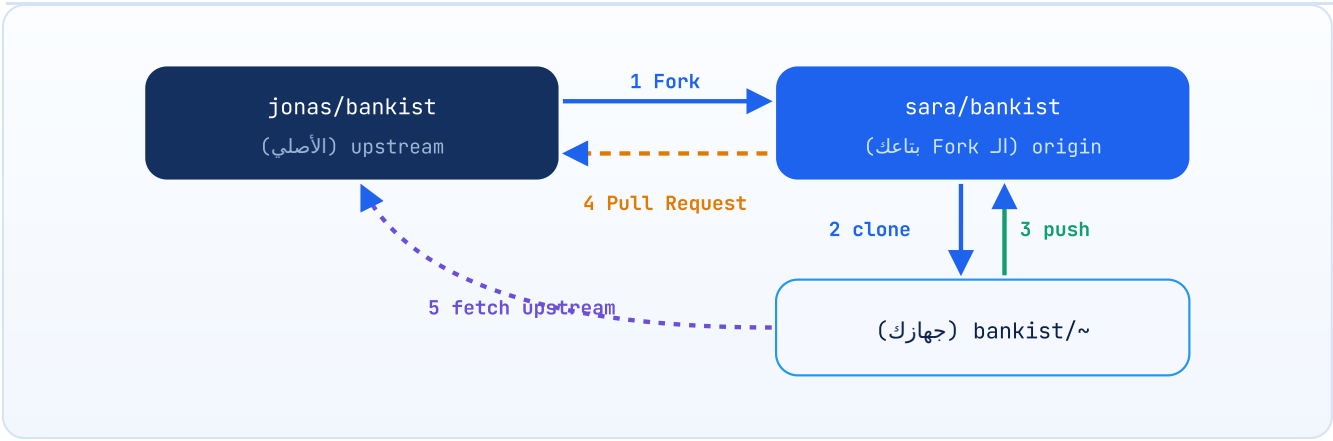
بعد Squash

بعد Squash، الـ Commits اللي على الـ Branch عندك مش هي اللي في main (اتجمعت في واحد جديد). فـ `git branch -d` ممكن يقولك `not fully merged`. ده طبيعي هنا: اتأكد إن الـ PR اتدمج، وامسحه بـ `-D`. ومتكملش شغل جديد على نفس الـ Branch، ابدأ واحد جديد من main.

11.5 الـ Fork: تساهم في مشروع مش بتاعك

وانت بتذاكر من Repo بتاع الكورس، لقيت غلطة إملائية في الـ README، أو لينك مكسور. قلت أصلّحها: دوست على القلم في GitHub، وبدل ما يفتحلك الملف، GitHub قالك: `You need to fork this repository to propose changes`. وده منطقي: لو أي حد على النت يقدر يعمل push على Repo Jonas، المشروع هيبقى خرابة في يوم.

طب إزاي آلاف الناس بيساهموا في مشاريع زي React و VS Code وهم مش في الفريق؟ مش هتقدر تعمل push لـ Repo مش بتاعك، فالحل: **Fork** = نسخة من الـ Repo في حسابك إنت على GitHub. تشتغل عليها براحتك، وبعدين تعمل PR من الـ Fork بتاعك للـ Repo الأصلي.



شكل 11.2 دورة الـ Fork: انسخ على GitHub، هات لجهازك، ابعت للـ Fork، واعمل PR للأصلي.

1. على صفحة المشروع الأصلي: زرار **Fork**.
2. اعمل clone لـ **الـ Fork بتاعك**: `git clone git@github.com:sara/bankist.git`.
3. ضيف الأصلي كـ Remote تاني اسمه `upstream` (عشان تجيب تحديثاته):

تشريح الأمر

```
git remote add upstream https://github.com/jonas/bankist.git
```

1. `git remote add` — ضيف Remote.
2. `upstream` — اسم متعارف عليه للـ Repo الأصلي (زي ما `origin` للـ Fork بتاعك). مجرد اسم.
3. `https://github.com/jonas/bankist.git` — رابط الأصلي. HTTPS كفاية لأنك هتقرا منه بس.

مزامنة الـ Fork مع الأصلي

```
Terminal
git switch main
git fetch upstream
git merge upstream/main      # أو: git pull upstream main
git push origin main         # حدّث الـ Fork على GitHub
```

أو من موقع GitHub: زرار **Sync fork** في صفحة الـ Fork، وبعدين `git pull` عندك. أو `gh repo sync`.

أول مساهمة Open Source

دوّر على Issues عليها Label `good first issue`، واقرا `CONTRIBUTING.md` قبل أي حاجة، واكتب في الـ Issue إنك هتشتغل عليها. تصحيح غلطة في التوثيق مساهمة محترمة جداً، وأنسب حاجة لـ Content Engineer.

11.6 الـ Issues: قايمة المهام والمشاكل

عندك 7 أفكار للبورتفوليو: Dark mode، وصفحة للمشاريع، وتصليح الـ Navbar على الموبايل... ومكتوبين فين؟ 3 في Notes على الموبايل، وواحدة في رسالة لنفسك على WhatsApp، والباقي في دماغك. وبعد شهر نسيت نصهم، ومش فاكر الـ Bug اللي صاحبك قالك عليه كان بيحصل إزاي. المشكلة إن الأفكار والمشاكل عايشة بعيد عن الكود. الـ Issues بتحطهم جنب الكود بالظبط، وبتربطهم بالـ Commits والـ PRs اللي حلّتهم.

Issue = تذكرة لمشكلة أو فكرة أو مهمة. ليها رقم (`#7`)، وعنوان، ووصف، و**Labels** (`bug` و `enhancement` و `documentation`)، وممكن تتعيّن لشخص.

Bug report كويس

عملت إيه بالظبط (خطوات)، كنت متوقع إيه، حصل إيه، Screenshot أو رسالة الخطأ، والمتصفح أو النسخة.

لنفسك

استخدم Issues كـ To-do list لمشروعك: كل Feature في Issue، وكل PR يقفل Issue. و GitHub Projects يعرضهم كـ Board.

ربط PR بـ Issue

لو كتبت في وصف الـ PR (أو رسالة Commit بيدخل main) كلمة من دول ورقم الـ Issue، الـ Issue هيتقفل لوحده لما الـ PR يتدمج:

الكلمة	مثال
`close` / `closes` / `closed`	`Closes #7`
`fix` / `fixes` / `fixed`	`Fixes #12`
`resolve` / `resolves` / `resolved`	`Resolves #3`

جدول 11.4 كلمات قفل الـ Issues

ومجرد كتابة `#7` في أي مكان بتعمل لينك للـ Issue، و `@username` بتعمل Mention للشخص.

ملخص الفصل

- PR = Branch + طلب دمج + صفحة مراجعة. الخطوات: Branch ← Commits ← push -u ← Open PR ← Review ← Merge ← نضّف.
- المراجعة: Comment و Approve و Request changes، و Suggestions بتتعمل Commit بزرار.
- التعديلات بتتعمل على نفس الـ Branch والـ PR بيتحدث لوحده.
- Merge commit و Squash و Rebase: نفس أفكار الفصل 8 بزرار.
- Fork = نسختك على GitHub، و `upstream` = الأصلي، و `Closes #7` يقفل الـ Issue.

صح ولا غلط؟

1. لو المراجع طلب تعديلات، لازم تقفل الـ PR وتعمل واحد جديد. صح / غلط
2. الـ Fork بيتعمل على GitHub، والـ clone على جهازك. صح / غلط
3. Squash and merge بيحط كل الـ Commits الـ Branch في main زي ما هي. صح / غلط

أسئلة مراجعة

1. اكتب كل خطوات PR من أول `git switch main` لحد مسح الـ Branch.
2. إيه الفرق بين `origin` و `upstream` في حالة الـ Fork؟
3. إمتى تختار Squash and merge؟
4. إزاي تخلي Issue يتقفل لوحده لما الـ PR يتدمج؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في الـ playground على GitHub، اعمل Issue "Add README"، واعمل Branch و PR فيه `Closes #1`، وادمجه. اتأكد إن الـ Issue اتقفل.
2. متوسط في PR تاني، راجع نفسك: اعمل تعليق Suggestion على سطر، و Commit suggestion، وبعدين Squash and merge. قارن `git log` قبل وبعد `git pull`.
3. صعب اعمل Fork لأي Repo عام بسيط، ضيف `upstream`، وزامن الـ Fork بالأوامر. لو لقيت غلطة في التوثيق، اعمل PR حقيقي.

12

المرحلة 5 · الشغل المنظّم

الفصل التاني عشر

Workflow للمبتدئين: GitHub Flow

دلوقتي عندك كل الأدوات: Commits و Branches و Merge و Push و PR. بس إمتى تعمل Branch؟ وإمتى تدمج؟ وتعمل إيه كل يوم؟ الـ Workflow هو الروتين اللي بيربط الأدوات دي. هنمشي على أبسط Workflow منتشر في العالم: GitHub Flow، ونطبّقه على مشروع شخصي وعلى شغل مع فريق.

بعد الفصل ده هتكون عارف

- يعني إيه Workflow وليه محتاجه
- قواعد GitHub Flow الست
- تسمية الـ Branches وإزاي Git بيخزنها
- الروتين اليومي من أول pull لحد مسح الـ Branch
- حماية main بـ Branch protection rules
- مقارنة سريعة مع Git Flow و Trunk-based

12.1 ليه Workflow؟

تخيل انت وصاحبك أحمد قررتوا تعملوا مشروع Weather app سوا. يوم الأحد أحمد عمل push على main لحتة نص مخلّصة "عشان متضيعش". يوم الاتنين انت عملت pull، والمشروع مبقاش بيفتح، وقعدت ساعة تدوّر على الـ Bug اللي انت مش عامله. يوم التلات انت فتحت Branch اسمه `new` وفضلت شغال عليه أسبوعين من غير ما تدمج، ولما جيت تدمج لقيت 14 Conflict لأن أحمد غيّر نص الملفات في الوقت ده. ويوم الأربع حد سأل "مين غيّر الألوان؟" وكل الـ Commits اسمها `update`.

محدش فيكم غلط في Git. كل الأوامر اتكتبت صح. المشكلة إن مفيش **اتفاق**: إمتى نعمل Branch؟ إمتى ندمج؟ مين يعمل push على main؟ والاتفاق ده اسمه **Workflow**: شوية قواعد بسيطة الكل ماشي عليها، فالأدوات اللي اتعلمتها تشتغل مع بعض بدل ما تتخانق.

12.2 قواعد GitHub Flow

فيه Workflows كتير، وأشهرها للمواقع وتطبيقات الويب هو اللي GitHub نفسه بيستخدمه ونشره: **GitHub Flow**. سرّه إنه بسيط لدرجة إنه بيتقال في جملة: "main دايماً شغال، وأي حاجة جديدة بتيجي من Branch قصير عن طريق PR". وكل القواعد الست تحت مجرد تفصيل للجملة دي. لاحظ إن كل قاعدة بتحل واحدة من مشاكل أسبوع أحمد:

1 · main دايماً شغال

أي حاجة على main لازم تبقى شغالة وجاهزة تتنشر. محدش يعمل push مباشر عليه.

2 · كل شغل في Branch

Branch جديد من main باسم واضح: Feature، إصلاح، تجربة، حتى تعديل كلمة في README.

3 · Commit و push كتير

Commits صغيرة، و push لـ Branch كل شوية (ده Backup كمان).

4 · افتح PR

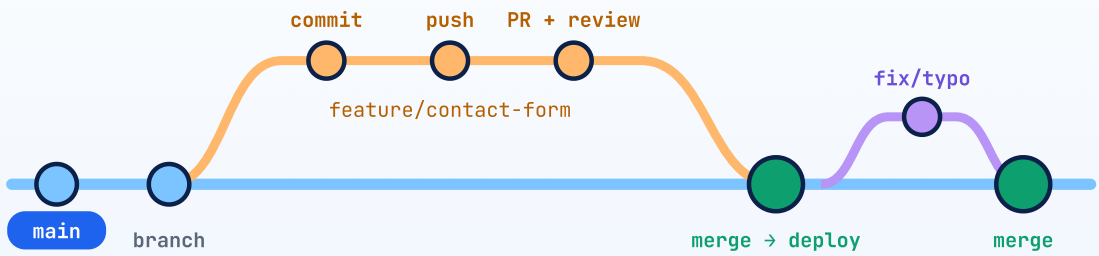
لما تخلص (أو تحتاج رأي بدري كـ Draft)، افتح PR لـ main.

5 · مراجعة واختبارات

حد يراجع (أو إنت تراجع نفسك بعين جديدة)، والـ Checks تعدي.

6 · Merge ونشر ونضافة

ادمج، والموقع يتنشر من main، وامسح الـ Branch.



شكل 12.1 GitHub Flow: main واحد شغال دايماً، و Branches قصيرة بتطلع وترجع بـ PR.

12.3 تسمية الـ Branches

بعد شهر في مشروع فيه فريق، هتعمل `git branch -a` وتلاقي 20 Branch. لو أسماءهم `new` و `test2` و `sara-stuff`، محدش هيعرف إيه اللي ينفع يتمسح. الاسم الكويس بيقول **نوع** الشغل و**موضوعه** من غير ما تفتح ولا Commit:

البادئة	إمتى	مثال
`feature/` أو `feat/`	ميزة جديدة	`feature/dark-mode`
`fix/`	تصليح Bug	`fix/12-email-validation`
`docs/`	توثيق و README	`docs/setup-steps`
`chore/`	صيانة (مكتبات، إعدادات)	`chore/update-vite`
`ai/` أو `try/`	تجربة ممكن ترميها (الفصل 18)	`ai/refactor-modules`

جدول 12.1 اتفاق تسمية منتشر (مش قاعدة في Git)

والقواعد: حروف صغيرة، وشَرطة `-` بدل المسافات (المسافة ممنوعة أصلاً في اسم Branch)، ورقم الـ Issue في الأول لو فيه. والـ `/` مش مجرد شكل: Git بيخزن `feature/dark-mode` كملف جوه فولدر `.git/refs/heads/feature/`، وأدوات زي VS Code بتعرضهم كفولدرات. وعشان كده مينفعش يبقى عندك Branch اسمه `feature` وكمان `feature/x` في نفس الوقت: واحد ملف والتاني فولدر بنفس الاسم.

12.4 الروتين اليومي

القواعد حلوة على الورق، بس شكلها إيه في يوم شغل حقيقي؟ تعال نمشي على يوم كامل: الصبح عاوز تضيف Dark mode للبورتفوليو. كل خطوة تحت هي أوامر انت عارفها خلاص، الجديد بس هو **الترتيب**. ولو حفظت الترتيب ده، 90% من شغلك اليومي بـ Git هيبقى أوتوماتيك:

يوم شغل عادي

1 ابدأ من آخر نسخة. لو فيه تعديلات مش متعملها Commit، الـ switch هيقولك.	`git switch main` `git pull`
2 Branch للشغل الجديد، من main المحدّث.	`git switch -c feature/dark-mode`

3 اشتغل، وكل خطوة شغالة = Commit. شوف قبل ما تضيف (الفصل 13 لـ `add -p`).	`git status`
	`git diff`
	`git add -p`
	`git commit -m "feat: add theme toggle button"`

4 أول push بـ `-u`، وبعد كده `git push` بس.	`git push -u origin feature/dark-mode`
5 لو الشغل طوّل وmain اتحرك: هات الجديد في الـ Branch بتاعك بدري عشان الـ Conflicts تبقى صغيرة.	`git fetch` `git merge origin/main`
6 افتح الـ PR من الموقع أو بـ `gh pr create` (الفصل 17).	`# Open PR → review → Merge (on GitHub)`

7 بعد الدمج: حدّث main وامسح الـ Branch المحلي.	`git switch main`
	`git pull`
	`git branch -d feature/dark-mode`

نضّف الـ Branches القديمة

`git fetch --prune` (أو `-p`) بيمسح من عندك الـ `origin/...` لـ Branches اتمسحت على GitHub. وتقدر تخليها دايماً: `git config --global fetch.prune true`. وفي إعدادات الـ Repo على GitHub فعّل **Automatically delete head branches** عشان الـ Branch يتمسح بعد الـ Merge.

12.5 وانت شغال لوحدك؟

ممكن تقول: "أنا لوحدي، مين هيراجعلي؟ ليه كل ده؟". سؤال منطقي. بس افتكر إن "انت بعد 3 شهور" شخص تاني مش فاكر حاجة، وإن الـ AI اللي بيساعدك بيعدّل كود كأنه زميل في الفريق (ومن أكتر الزملاء اللي محتاجين مراجعة). وكمان لو موقعك منشور من main على GitHub Pages، أي Commit بايظ على main = الموقع بايظ قدام أي حد بيفتحه.

فمش لازم تعمل PR لكل سطر، بس على الأقل:

- main يفضل شغال (خصوصاً لو الموقع منشور منه على GitHub Pages).
- أي حاجة كبيرة أو تجربة أو شغل AI = Branch.

- PR لنفسك للحاجات المهمة: هتشوف الـ diff كله في مكان واحد قبل ما يدخل.
- للحاجات التافهة (typo في README) Commit مباشر على main مقبول.

12.6 حماية main على GitHub

رجعنا لأحمد. اتفقتوا على GitHub Flow، وبعد أسبوع، الساعة 1 بالليل وهو مستعجل، عمل push على main "مرة واحدة بس". الاتفاقات بتتكسر لما الناس تتعب، وعشان كده الأحسن إن القاعدة متبقاش اتفاق، تبقى **قفل**. عشان القاعدة الأولى (main دايماً شغال) متبقاش مجرد اتفاق، GitHub بيخليك تفرضها: **Settings ← Rules ← Rulesets** (أو Branches ← Branch protection rules) على `main`:

القاعدة	بتعمل إيه
Require a pull request before merging	ممنوع push مباشر، لازم PR
Required approvals	عدد المراجعين اللي لازم يوافقوا
Require status checks to pass	الـ Actions لازم تنجح قبل الدمج
Block force pushes	ممنوع `push --force` على main
Restrict deletions	محدش يمسح main بالغلط

جدول 12.2 أهم قواعد حماية الـ Branch

```
Terminal
you@laptop:~/portfolio$ git push
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote: - Changes must be made through a pull request.
 ! [remote rejected] main -> main (push declined due to repository rule violations)
```

لو شفت ده، مفيش حاجة باظت: القاعدة شغالة. اعمل Branch من اللي عندك (`git switch -c fix/x`)، وابعته، وافتح PR. وبعدين رجّع main المحلي زي GitHub: `git switch main && git reset --hard origin/main` (الـ Commits لسه في الـ Branch الجديد).

12.7 Workflows تانية هتسمع عنها

لما تدخل أول شغل، ممكن تسمع "إحنا شغالين Git Flow" أو "Trunk-based". متقلقش: دي مش أدوات جديدة، دي اتفاقات تانية بنفس الأوامر. والفرق بينهم جاي من سؤال واحد: المشروع بيتنشر إزاي؟ موقع بيتنشر 10 مرات في اليوم غير تطبيق موبايل بيطلع إصدار كل شهر ولازم يستنى موافقة الـ App Store.

الـ Workflow	الفكرة	مناسب لـ
GitHub Flow	main + Branches قصيرة + PR	مواقع وتطبيقات ويب ومشاريع شخصية. **ابدأ بيه**
Git Flow	main و develop و /release/* و /hotfix/* و /feature/*	برامج ليها إصدارات رسمية متعددة. معقد على المبتدئ
Trunk-based	الكل بيدمج في main كذا مرة في اليوم، و Branches عمرها ساعات	فرق خبيرة عندها اختبارات أوتوماتيك قوية

جدول 12.3 مقارنة سريعة بين الـ Workflows

الخلاصة

لو حد قالك "إحنا شغالين Git Flow"، الأوامر نفسها هي هي: switch و merge و push و PR. الفرق بس في **أسماء الـ Branches** ومين يتدمج في مين. فاللي اتعلمته شغال مع أي Workflow.

ملخص الفصل

- GitHub Flow: main شغال دايماً، كل شغل في Branch، PR، مراجعة، Merge، امسح.
- الروتين: switch main ← pull ← switch -c ← Commits ← push -u ← PR ← Merge ← pull ← branch -d.
- لوحدك: Branch للتجارب والـ AI، و PR لنفسك للمهم.
- حماية main بالـ Rulesets: PR إجباري و Checks ومنع Force push.
- Git Flow و Trunk-based نفس الأوامر بقواعد مختلفة.

صح ولا غلط؟

1. في GitHub Flow بنعمل push مباشر على main للحاجات الصغيرة في الفرق. صح / غلط
2. `git fetch --prune` بيمسح Branches من GitHub. صح / غلط
3. الـ Branch protection ممكن يمنع `push --force` على main. صح / غلط

أسئلة مراجعة

1. اذكر قواعد GitHub Flow الست بكلامك.
2. ليه بنعمل `git pull` على main قبل ما نعمل Branch جديد؟
3. الـ push اترفض بـ `GH013`. تعمل إيه بالظبط عشان شغلك ميضيعش؟
4. إيه الفرق الأساسي بين GitHub Flow و Git Flow؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل طبّق الروتين اليومي كامل مرة واحدة على الـ playground (Branch ← PR ← Merge ← تنضيف).
2. متوسط فعّل Ruleset على main يمنع الـ push المباشر، وجرّب تعمل push عليه عشان تشوف الرفض. وبعدين صلّح الموقف بالطريقة اللي في الفصل.
3. صعب اعمل Branchين في نفس الوقت (`feature/a` و `feature/b`) بيعدلوا ملفات مختلفة، ادمج الأول بـ PR، وبعدين حدّث التاني بـ `git merge origin/main` قبل ما تدمجه.

13

المرحلة 5 · الشغل المنظّم

الفصل التالت عشر

رسايل Commit محترمة و Commits مركّزة

بعد 3 شهور، هتفتح git log وتلاقي: update، fix، asdf، final، final2. مش هتعرف أي حاجة اتعملت فين. رسالة الـ Commit هي الملاحظة اللي بتكتبها لنفسك في المستقبل ولزمايلك. الفصل ده عن شكل الرسالة الكويسة، وصيغة Conventional Commits المنتشرة، وإزاي تقسّم شغلك لـ Commits مركّزة بـ git add -p.

بعد الفصل ده هتكون عارف

- ليه رسالة الـ Commit مهمة وإيه شكل الرسالة الوحشة
- قواعد الرسالة الكويسة: عنوان قصير بصيغة الأمر وجسم بيشرح ليه
- صيغة Conventional Commits: الأنواع والـ Scope والـ Breaking change
- الـ Atomic commits: Commit واحد = تغيير واحد
- git add -p عشان تختار أجزاء من الملف
- إمتى تصلّح الرسايل وإمتى تسيبها

13.1 المشكلة: تاريخ مالوش معنى

تلات شهور بعد ما خلصت مشروع Bankist، حد بعتلك: "الـ Transfer بيقبل مبالغ بالسالب، والرصيد بيزيد!". فاكر إن الحتة دي كانت شغالة. فتحت `git log` عشان تعرف إمتى اتكسرت، ولقيت 40 Commit: تسعة منهم اسمهم `update`، وخمسة `fix`، وواحد `asdf` من ليلة كنت فيها تعبان. كل Commit لازم تفتحه وتقرا الـ diff بتاعه عشان تعرف عمل إيه. اللي كان المفروض ياخد دقيقة، أخد ساعة ونص.

الكود نفسه بيقول **إيه** اللي اتعمل. بس ليه اتعمل؟ وإمتى؟ وكان بيحل إيه؟ ده مكتوب في مكان واحد بس: رسالة الـ Commit. هي الملاحظة اللي بتسيبها لنفسك في المستقبل ولأي حد هيشتغل على المشروع، ولو اتكتبت وحش، مفيش حاجة تانية تعوّضها. قارن التاريخين دول:

تاريخ وحش

Terminal

```
you@laptop:~/portfolio$ git log --oneline
9f1c2ab update
3b7e0d1 fix
a12c9e4 more changes
77d0b3f asdf
c71f7c3 final
```

تاريخ محترم

Terminal

```
you@laptop:~/portfolio$ git log --oneline
9f1c2ab fix(form): show error on empty email
3b7e0d1 feat(form): add contact form
a12c9e4 style: format with prettier
77d0b3f docs: add setup steps to README
c71f7c3 feat: add home page
```

في التاني تقدر تعرف من غير ما تفتح ولا ملف: فين اتضافت الفورم، وإمتى اتصلح الـ Bug، وأنهي Commit مجرد Format (فتتجاهله وانت بتدوّر على Bug).

13.2 قواعد الرسالة الكويسة

معظم وقتك هتكتب الرسالة في سطر واحد بـ `-m`، وده تمام للحاجات الصغيرة. بس الرسالة الكاملة ليها شكل متعارف عليه من أيام مطوري Linux (اللي عملوا Git أصلاً)، والشكل ده مش ذوق: أدوات كتير معتمدة عليه. `git log --oneline` بيعرض أول سطر بس، و GitHub بيقص العنوان بعد 72 حرف، وصفحة الـ PR بتاخد العنوان والجسم من الـ Commit. وعشان تكتب رسالة كاملة، اعمل `git commit` من غير `-m`، والـ Editor هيفتح (افتكر ↩ 2.5 ص 17):

رسالة Commit كاملة (في الـ Editor)

1 **العنوان:** سطر واحد، حوالي 50 حرف (ومتعديش 72)، بصيغة الأمر (`add` مش `added`)، ومن غير نقطة في الآخر.	`fix(form): show error when email is empty`
2 **سطر فاضي:** لازم، عشان Git والأدوات تعرف فين العنوان وفين الجسم.	

3 **الجسم (اختياري):** بيشرح **ليه** و**إيه**، مش "إزاي" (الكود بيقول إزاي). سطور حوالي 72 حرف.	`Submitting the form with an empty email used to`
	`reload the page silently. Now we validate before`
	`submit and show an inline message.`

4 **Footer:** ربط بـ Issue أو ملاحظات زي `BREAKING CHANGE:`.	`Closes #7`

اختبار الجملة

العنوان الكويس يكمّل الجملة دي: "**If applied, this commit will ...**". مثلاً: "*add contact form* will" ✓، "*added* will" ✗. وده سبب صيغة الأمر. ولو مش عارف تكتب العنوان في سطر واحد، غالباً الـ Commit بيعمل حاجات كتير.

لو حابب تتعمق: أدوات بتساعدك تكتب

`git commit -v` (Verbose) بيعرض الـ diff كله تحت الرسالة في الـ Editor، فتكتب وانت شايف اللي عملته (السطور دي مش بتدخل الرسالة). و `git config --global commit.template ~/.gitmessage` بيخلي الـ Editor يفتح بقالب انت كاتبه (مثلاً أسماء الأنواع كتذكير). و `git log --grep="form"` بيدوّر في الرسايل، وهنا بتظهر قيمة الرسايل الكويسة. وفي الـ Footer فيه "Trailers" متعارف عليها زي `Co-authored-by: Ahmed <ahmed@mail.com>`، اللي GitHub بيعرض بيها الاتنين كأصحاب الـ Commit.

عربي ولا إنجليزي؟

الإنجليزي هو المتعارف عليه، وأسهل مع الأدوات والـ AI والشغل مع أي حد. لكن الأهم إنها تبقى واضحة ومتسقة: لو مشروع شخصي وبتكتب عربي أوضح لك، مفيش مشكلة تقنية.

13.3 Conventional Commits

القواعد اللي فاتت بتخلي الرسالة مقروءة للبني آدمين. بس تخيل لو كمان **البرامج** تقدر تقراها: أداة تعدّي على الرسايل وتقولك "من آخر إصدار فيه 3 ميزات جديدة و 5 إصلاحات، يبقى الإصدار الجاي 1.4.0"، وتكتب صفحة التغييرات (Changelog) لوحدها. ده بالظبط اللي عملته مشاريع زي Angular، ومنها طلعت صيغة اتحولت لمواصفة رسمية اسمها **Conventional Commits** (الإصدار 1.0.0)، ودلوقتي منتشرة جداً في مشاريع JavaScript. وهتلاقي الـ AI نفسه لما تطلب منه رسالة Commit بيكتبها بالشكل ده. شكلها:

تشريح الأمر

```
feat(cart)!: change price format to cents
```

1. `feat` — الـ **Type**: نوع التغيير (الجدول تحت).
2. `(cart)` — الـ **Scope** (اختياري): الجزء اللي اتغير من المشروع، بين قوسين.
3. `!` — اختياري: **Breaking change**، يعني التغيير ده بيكسر حاجة كانت شغالة (لازم اللي بيستخدم الكود يعدّل حاجة).
4. `:` — نقطتين ومسافة بعد النوع.
5. `change price format to cents` — الوصف: بحروف صغيرة غالباً، وبصيغة الأمر.

النوع	استخدامه	مثال
`feat`	ميزة جديدة للمستخدم	`feat: add dark mode toggle`
`fix`	إصلاح Bug	`fix: prevent negative balance`
`docs`	توثيق بس (README، تعليقات)	`docs: add deploy steps`
`style`	شكل الكود (مسافات، Format) من غير تغيير سلوك	`style: run prettier`
`refactor`	إعادة كتابة من غير تغيير السلوك	`refactor: extract calcInterest()`
`perf`	تحسين سرعة	`perf: debounce search input`
`test`	اختبارات	`test: add cases for transfer`
`build`	أدوات البناء والـ Dependencies	`build: upgrade vite`
`ci`	GitHub Actions وأخواتها	`ci: add lint workflow`
`chore`	صيانة مش داخلة في اللي فوق	`chore: update .gitignore`
`revert`	إلغاء Commit	`revert: feat: add dark mode`

جدول 13.1 أنواع Conventional Commits

رسمياً فين؟

المواصفة نفسها بتفرض `feat` و `fix` بس، والباقي متعارف عليه (من إعدادات commitlint المشهورة). وفايدتها مش الشكل بس: أدوات بتقرا الرسايل دي وتعمل منها **CHANGELOG** ورقم الإصدار أوتوماتيك (`fix` = Patch، `feat` = Minor، `!` = Major، الفصل 14).

تشريح الأمر

```
git commit -m "feat(auth)!: require email verification" -m "BREAKING CHANGE:
unverified users can no longer log in."
```

1. `git commit -m "..."` العنوان بالصيغة: نوع و Scope و `!`.
2. `-m "BREAKING CHANGE: ..."` التانية = فقرة الـ Footer. الكلمة `BREAKING CHANGE:` بحروف كبيرة دي جزء من المواصفة، وبتشرح إيه اللي اتكسر.

13.4 Atomic commits: Commit واحد = تغيير واحد

يوم جمعة قعدت 4 ساعات على البورتفوليو: صلّحت Bug في الفورم، وضفت Footer، وعملت Format للملفات كلها، وحدّثت المكتبات. وفي الآخر `git add .` و Commit واحد. الأسبوع اللي بعده اكتشفت إن المكتبة الجديدة كسرت الـ Animation. عاوز تلغي التحديث بس بـ `git revert`؟ مينفعش: الـ revert هيلغي الأربع حاجات مع بعض، وانت عاوز إصلاح الفورم والـ Footer. هتقعد تفصل بإيدك.

الـ Commit الذري (Atomic) بيعمل **حاجة واحدة كاملة**: يتفهم لوحده، والمشروع شغال بعده، ولو عملتله revert بيشيل الحاجة دي بس.

✗ Commit واحد

"update ,format all files ,add footer ,fix form bug packages": لو الـ Footer عمل مشكلة، مينفعش تلغيه من غير ما تلغي إصلاح الفورم.

✓ أربع Commits

`fix(form): ...` و `feat: add footer` و `style: format` و `build: update packages`. كل واحد يتفهم ويتلغي لوحده.

13.5 git add -p: اختار حتت من الملف

"تمام، Commit لكل حاجة" سهل لما كل حاجة في ملف لوحدها. بس الحياة مش كده: وانت بتصلّح Bug في `transfer()` في `app.js`، لاحظت إنك محتاج Function لتنسيق العملة، فكتبتها في آخر نفس الملف. دلوقتي الملف فيه تغييرين ملهمش علاقة ببعض.

طب لو في نفس الملف صلّحت Bug **و** ضفت حاجة جديدة؟ `git add file` هياخد الاتنين. الحل: `git add -p` بيعرضلك كل "حتة" (Hunk) اتغيرت ويسألك:

تشريح الأمر

```
git add -p app.js
 1   2  3
```

1. `git add` ضيف للـ Staging.
2. `-p` (أو `--patch`) Patch: اعرض التغييرات حتة حتة وأنا أختار.
3. `app.js` اختياري: ملف معين. من غيره هيعدي على كل الملفات المتعدلة.

```
Terminal
you@laptop:~/portfolio$ git add -p app.js
@@ -10,6 +10,9 @@ function transfer(from, to, amount) {
+  if (amount <= 0) return false;
(1/2) Stage this hunk [y,n,q,a,d,s,e,?]? y
@@ -40,3 +43,8 @@
+function formatCurrency(value) {
(2/2) Stage this hunk [y,n,q,a,d,s,e,?]? n
```

الحرف	معناه
`y`	Yes: ضيف الحتة دي
`n`	No: سيبها
`s`	Split: قسّمها لحتت أصغر (لو ينفع)
`e`	Edit: عدّلها بإيدك في الـ Editor (متقدم)
`a` / `d`	كل الحتت الجاية في الملف ده / ولا واحدة منهم
`q`	Quit: اخرج (اللي اخترته فاضل في الـ Staging)
`?`	المساعدة

جدول 13.2 اختيارات git add -p

بعدها: `git diff --staged` تتأكد، و `git commit -m "fix: reject non-positive amounts"`، وبعدين `git add -p` تاني للباقي.

حاجة مشتركة: Patch = -p

الـ `-p` في Git غالباً = "اشتغل حتة حتة أو اعرض الـ Patch": `git add -p` و `git restore -p` (ارمي حتت معينة) و `git stash -p` و `git log -p` (اعرض الـ diff). وفي VS Code نفس الفكرة: تختار سطور في الـ diff وتعمل **Stage Selected Ranges**.

مفيد جداً مع الـ AI

الـ AI عدّل 4 حاجات في ملف وانت عاوز 3 بس؟ `git add -p` : اختار الـ 3، Commit، وبعدين `git restore app.js` يرمي الرابعة. ده أحسن من إنك تقبل كله أو ترفض كله.

13.6 صلّحت الرسالة؟ إمتى ينفع

عملت Commit ولسه دايس Enter، ولقيت الرسالة `fix: form bgu` . أو اكتشفت بعد يوم إن 3 Commits ورا بعض رسايلهم `wip` . ينفع تصلّح؟ الإجابة دايماً على نفس السؤال اللي شفناه في التراجع (الفصل 6): **حد تاني شاف الـ Commit ده ولا لأ؟** تعديل الرسالة بيعمل Commit جديد بـ Hash جديد (الرسالة جزء من اللي بيتحسب منه الـ Hash). فلو لسه عندك بس، اعمل اللي انت عاوزه. لو اتشارك، فكّر مرتين:

الموقف	الحل
آخر Commit، لسه ما اتبعتش	`git commit --amend` (↩ 6.3 ص 48)
Commits أقدم، لسه ما اتبعتوش	`git rebase -i` و `reword` (الفصل 15)
اتبعتت على Branch عليه PR	مش مشكلة: Squash and merge هيخليك تكتب رسالة جديدة نضيفة
اتبعتت على main	سيبها، واتعلم للمرة الجاية. تعديل تاريخ main المشترك أسوأ من رسالة وحشة

جدول 13.3 تصليح رسايل الـ Commits

ملخص الفصل

- العنوان: قصير، بصيغة الأمر، من غير نقطة، وسطر فاضي قبل الجسم اللي بيشرح ليه.
- Conventional Commits: `type(scope)!: description` ، والأنواع feat و fix و docs و style و refactor و test و chore...
- `BREAKING CHANGE:` أو `!` للتغييرات اللي بتكسر.
- Atomic Commit: واحد = تغيير واحد كامل.
- `git add -p` : y و n و s و q، تختار حتت من الملف.

صح ولا غلط؟

1. `style:` معناه تغيير في الـ CSS بس. صح / غلط
2. `git add -p` ممكن يضيف جزء من ملف ويسيب الباقي. صح / غلط
3. الأحسن تعدّل رسايل Commits قديمة على main بعد ما اتبعتت. صح / غلط

؟ أسئلة مراجعة

1. أعد كتابة الرسايل دي بـ Conventional Commits: "fixed bug in login"، "README"، "changed colors + new navbar".
2. ليه العنوان بصيغة الأمر؟
3. إمتى تستخدم `refactor` وإمتى `fix`؟
4. إيه فايدة الـ Atomic commits لما تحتاج `git revert`؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل 3 Commits في الـ playground بأنواع مختلفة (feat و docs و chore)، واعرضهم بـ `git log --oneline`.
2. متوسط عدّل ملف في مكانين بعاد عن بعض لغرضين مختلفين، واستخدم `git add -p` عشان تعمل Commitين منفصلين.
3. صعب اكتب Commit فيه عنوان وجسم و Footer فيه `Closes #` رقم Issue حقيقي، من الـ Editor مش بـ `-m`. بعد الـ push، اتأكد إن الـ Issue اتربط.

14

المرحلة 5 · الشغل المنظّم

الفصل الرابع عشر

الـ Stash والـ Tags والـ Releases

أداتين صغيرين بيحلوا مشاكل كبيرة. الـ Stash: انت في نص شغل ومش جاهز تعمل Commit، وفجأة لازم تروح Branch تاني تصلّح حاجة. خبّي شغلك على جنب وارجعله. والـ Tags: علامة ثابتة على Commit مهم زي v1.0.0، ومنها بتعمل Release على GitHub فيه وصف وملفات للتحميل.

بعد الفصل ده هتكون عارف

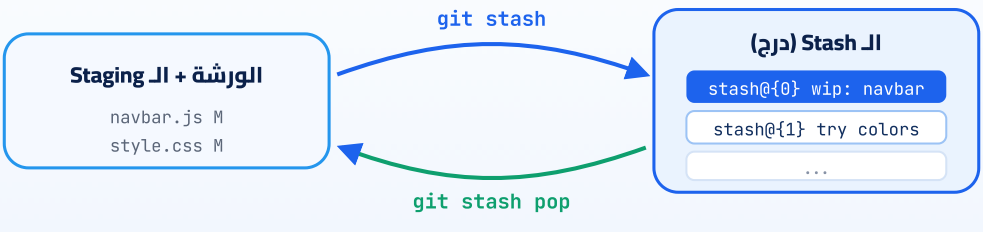
- المشكلة اللي git stash بيحلها
- stash push و list و show و pop و apply و drop
- stash للملفات الجديدة بـ -u، و Branch لـ stash جديد
- يعني إيه Tag والفرق بين Lightweight و Annotated
- Semantic Versioning: Major و Minor و Patch
- push لـ Tags وعمل Release على GitHub

14.1 المشكلة: لازم أسيب اللي في إيدي دلوقتي

الساعة 4 العصر، وانت في نص الـ Dropdown الجديد في الـ Navbar على `feature/navbar`. نص الكود مكتوب، والقايمة بتفتح بس مش بتقفل، وفيه `console.log` في 6 أماكن. فجأة رسالة: "الـ Contact form على الموقع مش بتبعت حاجة، والعميل هيفتح الموقع كمان ساعة!".

لازم تروح main وتعمل Branch للإصلاح دلوقتي. بس تعمل إيه في الشغل اللي في إيدك؟ لو عملت `git switch main`، يا Git يرفض عشان تعديلاتك هتتمسح (↩ 7.5 ص 56)، يا يسمح وينقل تعديلات الـ Navbar معاك لـ main فتتخلط مع الإصلاح. تعمل Commit؟ لكود بايظ مليان `console.log`؟ تنسخ الملفات في فولدر على الـ Desktop؟ رجعنا لأيام `final-final2`. محتاج حاجة زي الدرج اللي في المكتب: تشيل الورق اللي على المكتب كله وتحطه فيه، تشتغل على الموضوع المستعجل على مكتب نضيف، وبعدين تطلّع الورق تاني بنفس ترتيبه.

الحل: `git stash` = خد كل التعديلات اللي مش متعملها Commit، وحطها في درج على جنب، ورجّع الورشة `clean`.



شكل 14.1 الـ Stash درج (Stack): آخر حاجة تتحط فيه هي stash@{0}.

14.2 أوامر الـ Stash

تشريح الأمر

```
git stash push -u -m "wip: navbar dropdown"
```

1. `git stash push` خبّي التعديلات (`git stash` لوحدها = `git stash push`).
2. `-u` Include Untracked: خبّي الملفات الجديدة كمان. من غيرها الملفات الجديدة **بتفضل** في الورشة (ومش هتبقى clean).
3. `-m "wip: navbar dropdown"` اسم للـ Stash ده عشان تفتكره، بدل الاسم الأوتوماتيك "WIP on main: f1c2ab9...".

Terminal

```
you@laptop:~/portfolio$ git stash push -u -m "wip: navbar dropdown"
Saved working directory and index state On feature/navbar: wip: navbar dropdown
you@laptop:~/portfolio$ git status
nothing to commit, working tree clean
you@laptop:~/portfolio$ git stash list
stash@{0}: On feature/navbar: wip: navbar dropdown
stash@{1}: On main: try new colors
```

الأمر	بيعمل إيه
`git stash list`	اعرض الدرج كله. `stash@{0}` الأحدث
`git stash show -p stash@{1}`	اعرض الـ diff بتاع Stash معين (`-p` = Patch كامل، من غيرها ملخص)
`git stash pop`	رجّع آخر Stash للورشة **وامسحه** من الدرج
`git stash apply stash@{1}`	رجّعه بس **سيبه** في الدرج (لو عاوز تطبقه على أكتر من Branch)
`git stash drop stash@{1}`	امسح Stash معين
`git stash clear`	امسح الدرج كله (من غير رجوع تقريباً)
`git stash branch try/x stash@{0}`	اعمل Branch جديد من المكان اللي اتعمل فيه الـ Stash وطبّقه عليه
`git stash -p`	خبّي حتت معينة بس (زي `add -p`)

جدول 14.1 أوامر git stash

حاجة مشتركة: {n}@ والـ Stack

الـ `stash@{0}` هو نفس شكل `HEAD@{1}` اللي هتشوفه في الـ reflog (الفصل 16): اسم + `@{رقم}`. وفي zsh أو PowerShell لو الأقواس عملت مشكلة حطها بين Quotes: `"stash@{1}"` (نفس فكرة `HEAD^` في الفصل 4).

السيناريو كامل

Terminal

```
git stash push -u -m "wip: navbar"     # خبّي
git switch main && git pull
git switch -c fix/contact-form
# ... صلّح، PR، push، Commit ...
git switch feature/navbar
git stash pop                          # رجّع شغلك وكمّل
```

مشكلة 1 **stash pop عمل Conflict**

الـ `pop` طلع `CONFLICT` لأن الملف اتغير من ساعة ما عملت stash.

الحل: حل الـ Conflict عادي زي الفصل 8 (العلامات و `git add`). المهم: لما الـ pop يعمل Conflict، الـ Stash **مش بيتمسح** من الدرج (عشان أمان). بعد ما تحل وتتأكد، امسحه بإيدك: `git stash drop`.

لو حابب تتعمق: الـ Stash من جوه

الـ Stash مش سحر: هو Commits عادية بس مستخبية. كل Stash = Commit ليه 2 أو 3 Parents: آخر Commit كنت عليه، و Commit فيه حالة الـ Staging، ولو استخدمت `-u` Commit تالت فيه الملفات الجديدة. جرّب `git cat-file -p stash@{0}` هتلاقي سطور `parent` كتير. والدرج نفسه Ref اسمه `refs/stash`، و `stash@{1}` مجرد الـ reflog بتاعه (عشان كده نفس شكل `HEAD@{1}`). ودي فايدة عملية: لو عملت `drop` بالغلط، Git طبعلك `Dropped refs/stash@{0} (36ff05b...)`، فـ `git stash apply 36ff05b` بيرجّعه، لأن الـ Commit لسه موجود لحد ما Git ينضّف.

الـ Stash مش مكان تخزين

الـ Stash محلي (مبيتبعتش على GitHub)، وسهل تنساه، والدرج بيتملي حاجات مش فاكر هي إيه. استخدمه للمواقف السريعة (دقايق أو ساعات). أي حاجة هتعيش أكتر من كده: Commit على Branch (حتى لو اسمه `wip:`) واعمله push.

14.3 الـ Tags: علامة ثابتة على Commit

بعتّ لينك البورتفوليو لشركة في أول الشهر، وبعدها كمّلت شغل: غيّرت التصميم، وضفت صفحة، وشلت مشروع قديم. بعد أسبوعين الشركة كلمتك: "عجبنا المشروع اللي كان في الصفحة التانية، ممكن تبعتلنا نفس النسخة؟". النسخة دي فين؟ هتعمل `git log` وتدوّر على Commit من "أول الشهر تقريباً"؟ كان المفروض يوم ما بعتّ اللينك تحط علامة: "النسخة دي اللي اتبعتت".

العلامة دي اسمها **Tag**. الـ Branch لافتة **بتتحرك** مع كل Commit. الـ **Tag** لافتة **ثابتة**: بتتحط على Commit مهم وتفضل عليه للأبد. أشهر استخدام: أرقام الإصدارات (`v1.0.0`).

النوع	الأمر	فيه إيه
Lightweight	`git tag v1.0.0`	مجرد اسم على Commit
Annotated	`git tag -a v1.0.0 -m "msg"`	Object كامل: مين عمله، وإمتى، ورسالة. **استخدمه للإصدارات**

جدول 14.2 نوعين Tags

تشريح الأمر

```
1   2  3          4                      5
git tag -a v1.0.0 -m "First public release" 9f1c2ab
```

1. `git tag` إدارة الـ Tags. من غير Arguments = اعرضهم.
2. `-a` Annotated.
3. `v1.0.0` الاسم. الـ `v` في الأول متعارف عليه.
4. `-m "First public release"` رسالة الـ Tag.
5. `9f1c2ab` اختياري: الـ Commit. من غيره = HEAD.

`git tag`	اعرض كل الـ Tags. و `git tag -l "v1.*"` بـ Pattern.
`git show v1.0.0`	تفاصيل الـ Tag والـ Commit اللي عليه.
`git push origin v1.0.0`	ابعت Tag واحد. (push العادي مش بيبعت Tags).
`git push --follow-tags`	ابعت الـ Commits ومعاها الـ Annotated tags اللي عليهم.
`git tag -d v1.0.0`	امسح محلياً. ومن GitHub: `git push origin --delete v1.0.0`.
`git switch --detach v1.0.0`	اتفرج على المشروع زي ما كان في الإصدار ده (Detached HEAD).

14.4 Semantic Versioning: معنى الأرقام

ليه `v1.0.0` ومش `v1` أو `v2026-10`؟ تخيل إنك بتستخدم مكتبة، وطلع منها إصدار جديد. السؤال الوحيد اللي يهمك: "لو حدّثت، الكود بتاعي هيتكسر؟". لو الأرقام عشوائية، لازم تقرا كل التغييرات عشان تعرف. نظام **Semantic Versioning** (SemVer) بيحط الإجابة في الرقم نفسه: كل جزء من التلاتة ليه معنى ثابت، فمجرد ما تشوف الرقم اتغير إزاي، تعرف المخاطرة.



شكل 14.2 SemVer: MAJOR.MINOR.PATCH، ومرتبطة بأنواع Conventional Commits.

ودي نفس الأرقام اللي بتشوفها في `package.json` بتاع أي مكتبة npm (`"vite": "^7.1.2"`). الـ `^` هناك معناها "أي Minor أو Patch أحدث، بس نفس الـ Major" لأن الـ Major بيكسر.

14.5 الـ Releases على GitHub

الـ Tag معلومة جوه Git: مبرمج يفهمها، بس صاحبك اللي عاوز يجرّب برنامجك مش هيعمل `git switch --detach v1.0.0`. محتاج صفحة يقرا فيها "إيه الجديد" ويدوس Download. وده اللي GitHub بيضيفه فوق الـ Tag.

الـ **Release** = Tag + صفحة على GitHub فيها عنوان، و Release notes (إيه الجديد)، وملفات للتحميل (GitHub بيحط Zip للكود تلقائياً).

1. اعمل Tag وابعته: `git tag -a v1.0.0 -m "v1.0.0"` و `git push origin v1.0.0`.
2. على GitHub: **Releases ← Draft a new release**، اختار الـ Tag.
3. دوس **Generate release notes**: GitHub يجمع عناوين الـ PRs اللي اتدمجت من آخر Release.
4. عدّل، ضيف ملفات لو فيه (مثلاً Build مضغوط)، و **Publish release**.

تشريح الأمر

```
gh release create v1.0.0 --generate-notes
```

1	`gh release create`	اعمل Release من الـ Terminal (الفصل 17).
2	`v1.0.0`	الـ Tag. لو مش موجود على GitHub، `gh` بيعمله على آخر Commit في الـ Branch الافتراضي.
3	`--generate-notes`	اكتب الـ Release notes أوتوماتيك من الـ PRs.

لمشاريعك الشخصية

حتى في مشروع الـ Portfolio، Tag زي `v1.0.0` لما تنشر أول نسخة بيديك نقطة ترجعلها بسهولة: "النسخة اللي بعتها للشركة الفلانية". والـ Releases بتخلي صفحة المشروع على GitHub شكلها محترف.

ملخص الفصل

- `git stash push -u -m "..."` يخبّي كل حاجة حتى الجديد، و `pop` يرجّع ويمسح، و `apply` يرجّع ويسيب.
- `stash list` و `show -p` و `drop`، والـ Conflict في pop مبيمسحش الـ Stash.
- الـ Tag لافتة ثابتة، و `-a` Annotated للإصدارات، والـ push بيتعمل له لوحده.
- SemVer: MAJOR بيكسر، MINOR ميزة، PATCH إصلاح.
- Release = Tag + صفحة و Notes، من الموقع أو `gh release create`.

صح ولا غلط؟

1. `git stash` من غير `-u` بيخبّي الملفات الجديدة Untracked. صح / غلط
2. `git push` العادي بيبعت الـ Tags. صح / غلط
3. لو مكتبة طلعت من 2.4.1 لـ 3.0.0 يبقى غالباً فيه تغيير بيكسر. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `stash pop` و `stash apply`؟
2. إمتى الـ Stash ميكونش الحل الصح؟
3. إيه الفرق بين Tag و Branch؟ وبين Lightweight و Annotated؟
4. مشروعك على 1.3.2: إيه الرقم الجاي لو ضفت ميزة؟ ولو صلحت Bug؟ ولو غيرت شكل API بيكسر؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل عدّل ملفين، اعمل `git stash push -m "test"`، اتأكد إن الورشة clean، وبعدين `git stash pop`.
2. متوسط اعمل ملف جديد وعدّل ملف قديم، وجرّب `git stash -u`: إيه اللي اتخبّى وإيه اللي فضل؟ وبعدين بـ `-u`.
3. صعب اعمل Tag `v0.1.0` Annotated على الـ playground وابعته، واعمل Release من الموقع بـ Generate release notes.

15

المرحلة 6 · أدوات متقدمة بهدوء

الفصل الخامس عشر

الـ Rebase والـ Cherry-pick بهدوء

ناس كتير بيتكلموا عن الـ Rebase كأنه سحر أسود. هو في الحقيقة فكرة بسيطة: بدل ما تدمج خطين بـ Merge Commit، بتنقل الـ Commits بتاعتك كأنك بدأتها من آخر نسخة. وفيه نسخة تفاعلية بتخليك تنضّف تاريخك قبل ما تعمل PR. وهنشوف Cherry-pick: إزاي تاخد Commit واحد بعينه من Branch تاني. كل ده بالرسم، ومع القاعدة الذهبية اللي هتحميك.

بعد الفصل ده هتكون عارف

- الـ Rebase بيعمل إيه بالرسم، وإزاي بيختلف عن الـ Merge
- إمتى تستخدم Rebase وإمتى Merge
- القاعدة الذهبية: متعملش Rebase لحاجة اتشاركت
- حل الـ Conflicts أثناء الـ Rebase و continue-- و abort--
- الـ Interactive rebase: pick و reword و squash و fixup و drop
- git pull --rebase و push --force-with-lease
- git cherry-pick وإمتى يفيد

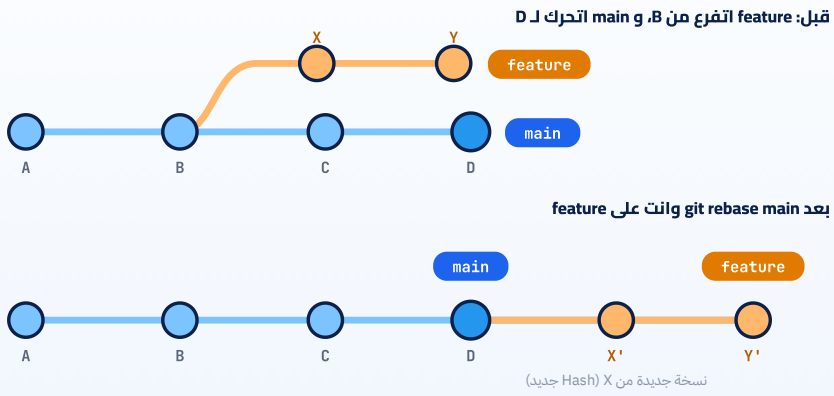
15.1 الحكاية: تاريخ مليان عُقد

افتكر الفصل 13: وانت شغال على `feature/dark-mode` لمدة 3 أيام، main اتحرك، فعملت `git merge origin/main` جوه الـ Branch بتاعك عشان تتحدث. وعملتها تاني اليوم اللي بعده. وتالت يوم. كل مرة Merge Commit جديد اسمه `Merge branch 'main' into feature/dark-mode`. وبعدين عملت PR، و `git log --graph` بقى شكله زي سلك سماعات اتلخبط في الشنطة.

مفيش أي حاجة غلط في كده، والكود شغال. بس فيه ناس بتحب التاريخ يبقى "خط مستقيم" سهل يتقري. وهنا ييجي الـ Rebase بسؤال مختلف: بدل ما "أدمج" الجديد اللي في main جوه شغلي، ليه ما "أعيدش بناء" شغلي كأني بدأته النهارده من آخر main؟

15.2 الـ Rebase بالرسم

اللي Git بيعمله بالظبط: بياخد الـ Commits بتاعتك (X و Y) "على جنب"، وبيحرّك الـ Branch لآخر main (D)، وبعدين بيعيد تطبيق تغييرات X فوق D كـ Commit جديد (X')، وبعدين Y فوقه (Y'). كأنك بدأت الشغل النهارده. وده بالظبط معنى الاسم: **Re-base** = غيّر القاعدة اللي الـ Branch مبني عليها.



شكل 15.1 الـ Rebase بيعيد عمل X و Y فوق D، فالتاريخ يبقى خط واحد. و X' مش X: محتواها نفس التغيير بس Hash مختلف.

تشريح الأمر

```
1        2      3       4
git switch feature && git rebase main
```

1. `git switch feature` — اقف على الـ Branch اللي **هيتنقل** (عكس الـ Merge: هنا انت واقف على اللي هيتغير).
2. `&&` — لو نجح، كمّل.
3. `git rebase` — أعد بناء الـ Commits بتاعتي.
4. `main` — القاعدة الجديدة: فوق آخر Commit في main.

```
Terminal
you@laptop:~/portfolio$ git rebase main
Successfully rebased and updated refs/heads/feature.
```

وبعدها لو رجعت لـ main وعملت `git merge feature`، هيبقى **Fast-forward** (↩ 8.2 ص 62) لأن feature بقى قدام main على نفس الخط. ومفيش Merge Commit خالص.

	Merge	Rebase
شكل التاريخ	بيحتفظ بالفروع زي ما حصلت، ومعاها Merge Commits	خط مستقيم كأن كل حاجة حصلت ورا بعض
الـ Commits القديمة	زي ما هي (نفس الـ Hash)	بتتعمل من جديد (Hash جديد)
الـ Conflicts	مرة واحدة	ممكن مع كل Commit بيتطبق
آمن على Branch مشترك؟	أيوه دايماً	**لأ** (القاعدة الذهبية تحت)
مناسب للمبتدئ	أيوه، ابدأ بيه	على Branch بتاعك بس، لما تتعود

جدول 15.1 Merge مقابل Rebase (بعد ما فهمنا الاتنين)

15.3 القاعدة الذهبية

> ⚠ **متعملش Rebase لـ Commits حد تاني ممكن يكون عنده**
>
> الـ Rebase بيرمي الـ Commits القديمة ويعمل جداد. لو الـ Commits دي اتعملها Push وزميلك عمل Pull، هيبقى عنده النسخة القديمة وعندك الجديدة، وهتحصل لخبطة كبيرة (الـ Commits تظهر مرتين، و Conflicts غريبة). فالقاعدة: **Rebase على Branch بتاعك انت لوحدك**، اللي لسه محدش بنى عليه. **ومتعملش Rebase لـ main أبداً.**

طب ولو الـ Branch بتاعي واتعمله Push عشان الـ PR، وعملت Rebase؟ هنا الـ push العادي هيترفض (لأن التاريخ اختلف)، ومحتاج Force push. بس بالنسخة الآمنة:

تشريح الأمر

```
git push --force-with-lease
```

1. `git push` ابعت.
2. `--force-with-lease` اكتب فوق اللي على GitHub، **بشرط** إنه لسه زي آخر مرة شفته (آخر fetch). لو حد ضاف Commit على الـ Branch من وراك، هيرفض بدل ما يمسح شغله. ده بديل `--force` اللي بيكتب فوق أي حاجة من غير ما يسأل. وعلى main الـ Rulesets (↩ 12.6 ص 94) غالباً بتمنع الاتنين.

15.4 Conflict أثناء الـ Rebase

لأن الـ Rebase بيطبّق Commits واحد واحد، ممكن يقف عند أي واحد فيهم بـ Conflict. والحل شبه الـ Merge بالظبط مع اختلاف بسيط:

1. `git status` هيقولك `interactive rebase in progress` أو `You are currently rebasing` والملف اللي فيه المشكلة.
2. حل الـ Conflict في الملف وامسح العلامات (↩ 8.5 ص 64).
3. `git add file` .
4. `git rebase --continue` (مش `git commit`). هيكمل باقي الـ Commits، وممكن يقف تاني.
5. اتلخبطت؟ `git rebase --abort` يرجّع كل حاجة زي ما كانت قبل الـ Rebase.

> ⓘ **ours و theirs بيتقلبوا في الـ Rebase**
>
> في الـ Merge، "Current/ours" = الـ Branch اللي انت عليه. في الـ Rebase بيتعكسوا: Git واقف على main وبيطبّق Commits بتاعتك عليه، فـ `HEAD` / ours = main، و theirs = الـ Commit بتاعك. ده بيلخبط الكل، فبص على المحتوى نفسه مش على الاسم.

15.5 الـ Interactive rebase: نضّف تاريخك

ده بقى أحلى استخدام للـ Rebase للمبتدئ، وأأمنهم. تخيل إنك قبل ما تفتح PR، بصيت على الـ Commits بتاعتك ولقيت: `feat: add toggle`، و `wip`، و `fix typo`، و `oops`. مش منظر محترم. الـ Interactive rebase بيخليك "تعيد كتابة" آخر كام Commit: تدمجهم، وتغيّر رسايلهم، وترتّبهم، وتمسح منهم.

تشريح الأمر

```
git rebase -i HEAD~4
```

1. `git rebase` أعد بناء.
2. `-i` Interactive: افتحلي قايمة في الـ Editor أقرر فيها هعمل إيه في كل Commit.
3. `HEAD~4` القاعدة: آخر 4 Commits هيظهروا في القايمة (↩ 4.2 ص 34).

هيفتحلك الـ Editor (عشان كده ظبطنا `core.editor`) بقايمة **الأقدم فوق** (عكس git log):

git-rebase-todo

1 `pick` = سيبه زي ما هو.	`pick 83e5807 feat: add theme toggle`
2 `squash` (أو `s`) = ادمجه في اللي فوقه، واسألني على الرسالة المجمعة.	`squash 58b4791 wip`
3 `fixup` (أو `f`) = ادمجه في اللي فوقه وارمي رسالته.	`fixup e44f2a7 fix typo`
4 `reword` (أو `r`) = سيب التغيير بس هغيّر الرسالة.	`reword 9a1b2c3 oops`
5 وفيه كمان `drop` (امسح الـ Commit) و `edit` (اقف عنده أعدّل فيه). وتقدر تغيّر **ترتيب السطور** عشان ترتّب الـ Commits. ولو مسحت كل السطور، الـ Rebase بيتلغي.	`# d, drop <commit> = remove commit`

احفظ واقفل. Git هيمشي على القايمة من فوق لتحت، وهيفتحلك الـ Editor تاني لرسالة الـ squash ورسالة الـ reword. والنتيجة: Commitين نضاف بدل 4.

لو حابب تتعمق: fixup أوتوماتيك

لو عرفت من الأول إن التعديل ده تصليح لـ Commit معين: `git commit --fixup=83e5807` بيعمل Commit رسالته `fixup! feat: add theme toggle`. وبعدين `git rebase -i --autosquash HEAD~5` هيرتّبهم ويحط `fixup` لوحده. وتقدر تخليها دايماً: `git config --global rebase.autoSquash true`.

مع الـ Squash and merge

لو الفريق بيستخدم **Squash and merge** على GitHub (↩ 11.4 ص 86)، فال Commits الصغيرة بتتجمع لوحدها في الآخر، ومش محتاج تنضّف بإيدك. الـ Interactive rebase مفيد أكتر لما الفريق بيحتفظ بكل Commit.

15.6 git pull --rebase

فاكر لما الـ pull عمل Merge Commit لأن انت وGitHub عندكم Commits جديدة (↩ 10.6 ص 80)؟ تقدر بدل كده تخلي pull يعمل fetch وبعدين **Rebase** للـ Commits المحلية بتاعتك فوق اللي جه. وده آمن هنا لأن الـ Commits دي لسه محدش شافها:

تشريح الأمر

```
1       2
git pull --rebase
```

1. `git pull` fetch + دمج.
2. `--rebase` الدمج يبقى بـ Rebase: حط Commits بتاعتي اللي لسه ما اتبعتتش فوق اللي على GitHub. ولو عاوزها دايماً: `git config --global pull.rebase true`.

15.7 git cherry-pick: Commit واحد بعينه

موقف تاني: انت على `feature/redesign`، وفي النص صلّحت Bug مهم في الفورم في Commit لوحده. الـ redesign لسه قدامه أسبوع، بس الإصلاح لازم يروح main النهارده. مش عاوز تدمج الـ Branch كله. عاوز الـ Commit ده بس. زي ما تقطف كرزة واحدة من الشجرة، ومن هنا الاسم:

تشريح الأمر

```
1          2  3           4
git switch main && git cherry-pick 4f2a9c1
```

1. `git switch main` اقف على المكان اللي عاوز الـ Commit يتضاف فيه.
2. `&&` لو نجح، كمّل.
3. `git cherry-pick` خد التغيير اللي في الـ Commit ده، وطبّقه هنا كـ Commit جديد (Hash جديد، ونفس الرسالة).
4. `4f2a9c1` الـ Commit (تلاقيه بـ `git log --oneline feature/redesign`). وتقدر تحط أكتر من واحد، أو مدى `A..B`.

`git cherry-pick -x 4f2a9c1`	ضيف سطر في الرسالة `(cherry picked from commit ...)` عشان تعرف جه منين.
`git cherry-pick -n 4f2a9c1`	No commit: طبّق التغيير في الورشة والـ Staging بس، وانت تعمل Commit.

`git cherry-pick --continue / --abort`	لو وقف بـ Conflict (نفس الزوج، ↩ 8.6 ص 66).

متعملهوش عادة

الـ Cherry-pick بيعمل نسخة من التغيير، فنفس التغيير هيبقى موجود في Commitين مختلفين. لو بتعمله كتير، غالباً الـ Workflow محتاج يتظبط (Branches أصغر). استخدامه الطبيعي: Hotfix، أو Commit اتعمل على الـ Branch الغلط (الفصل 19).

ملخص الفصل

- الـ Rebase بيعيد عمل Commits بتاعتك فوق قاعدة جديدة، فالتاريخ يبقى خط، والـ Hashes تتغير.
- القاعدة الذهبية: Rebase لـ Branch بتاعك بس، ومتعملوش لـ main أبداً، و `--force-with-lease` مش `--force`.
- في الـ Conflict: حل، add، `git rebase --continue`، أو `--abort`.
- `git rebase -i HEAD~n`: pick و reword و squash و fixup و drop.
- `git pull --rebase` للـ Commits المحلية، و `git cherry-pick` لـ Commit واحد.

صح ولا غلط؟

1. بعد الـ Rebase الـ Commits بتحتفظ بنفس الـ Hash. صح / غلط
2. في الـ Rebase بتقف على الـ Branch اللي هيتنقل. صح / غلط
3. `--force-with-lease` بيرفض لو حد ضاف Commits على الـ Remote من آخر fetch. صح / غلط

أسئلة مراجعة

1. ارسم التاريخ قبل وبعد `git rebase main`، وقارنه بـ `git merge main`.
2. ليه الـ Rebase على Branch مشترك بيعمل مشاكل؟
3. إيه الفرق بين `squash` و `fixup`؟
4. إمتى تستخدم `cherry-pick`؟ وليه متعملهوش عادة؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Branch بـ 2 Commits، واعمل Commit على main، وبعدين `git rebase main` من الـ Branch. قارن `git log --graph` قبل وبعد.
2. متوسط اعمل 4 Commits صغيرة بأسماء وحشة، ونضّفهم بـ `git rebase -i HEAD~4` لـ Commitين بأسماء Conventional.
3. صعب اعمل Conflict أثناء Rebase، حله بـ `--continue`. وبعدين خد Commit واحد من Branch تاني بـ `cherry-pick -x`.

16

المرحلة 6 · أدوات متقدمة بهدوء

الفصل السادس عشر

الـ Reflog شبكة الأمان، و Bisect و Blame

فيه لحظة هتمر بيها أكيد: عملت reset --hard غلط، أو مسحت Branch فيه شغل يومين، وقلبك وقع في رجليك. الفصل ده هيطمنك: Git فاكر كل مكان HEAD راحله في آخر 90 يوم، في دفتر اسمه reflog. وبعدين أداتين بيخلوك محقق: bisect بيلاقي الـ Commit اللي دخّل الـ Bug بالبحث الثنائي، و blame بيقولك كل سطر اتكتب إمتى ومين وليه.

بعد الفصل ده هتكون عارف

- الـ reflog بيسجل إيه، وإزاي تقراه
- git bisect خطوة بخطوة، و bisect run الأوتوماتيك
- ترجّع Commits بعد reset --hard أو Branch اتمسح
- git blame وأهم الـ Flags بتاعته
- حدود الـ reflog: محلي ومؤقت ومبيحفظش اللي ما اتعملوش Commit
- ربط blame بـ log و show عشان تعرف ليه

16.1 الحكاية: "مسحت شغل يومين"

الساعة 1 بالليل، وانت عاوز تنضّف حاجة في التاريخ. قريت على النت إن `git reset --hard HEAD~3` هيحلها. كتبتها، و Enter، وبصيت على `git log` ... آخر 3 Commits اختفوا. ومعاهم صفحة الـ Dashboard اللي قعدت فيها يومين. مش شايفهم. ولا أي Branch بيشاور عليهم.

خد نفس. الـ Commits دي **لسه موجودة**. فاكر إن Git بيخزّن كل Commit كـ Object في `.git/objects`؟ الـ reset مش بيمسح Objects، هو بس حرّك اللافتة لورا. الـ Commits بقت "يتيمة" (مفيش اسم بيشاور عليها)، بس هي هناك. ومحتاجين بس نعرف الـ Hash بتاعها. وهنا ييجي البطل.

16.2 git reflog: دفتر تحركات HEAD

`git log` بيوريك تاريخ **المشروع** (الـ Commits وأبهاتهم). أما `git reflog` (Reference log) فبيوريك تاريخ **انت**: كل مرة HEAD اتحرك فيها على جهازك، بأي سبب: commit، أو switch، أو reset، أو merge، أو rebase، أو amend.

Terminal

```
you@laptop:~/portfolio$ git reflog
f32164c HEAD@{0}: reset: moving to HEAD~3
2cfdb95 HEAD@{1}: commit: feat: add dashboard charts
8ab01d4 HEAD@{2}: commit: feat: add dashboard layout
5c7e9f2 HEAD@{3}: commit: feat: fetch account data
f32164c HEAD@{4}: checkout: moving from main to feature/dashboard
```

قراية الـ reflog

`f32164c`	HEAD كان فين بعد الحركة دي.
`HEAD@{0}`	اسم المكان: `{0}` = دلوقتي، `{1}` = الحركة اللي قبلها، وهكذا. وده اسم ينفع تستخدمه في أي أمر زي الـ Hash.
`reset: moving to HEAD~3`	إيه اللي حرّكه. هنا الـ reset الغلط.
`2cfdb95 HEAD@{1}: commit: ...`	قبل الـ reset على طول، HEAD كان على `2cfdb95` : ده آخر Commit "اختفى". لقيناه.

الإنقاذ

تشريح الأمر

```
git reset --hard HEAD@{1}
```

1. `git reset --hard` حرّك الـ Branch وخلي الورشة زي المكان ده بالظبط (اتأكد إن مفيش تعديلات مش متعملها Commit).
2. `HEAD@{1}` المكان اللي HEAD كان فيه من حركة واحدة، يعني قبل الـ reset الغلط. وتقدر تكتب الـ Hash `2cfdb95` بدلها.

الأأمن: اعمل Branch الأول

لو مش متأكد إن ده المكان الصح، متعملش reset على طول. اعمل Branch على المكان القديم واتفرج براحتك: `git branch rescue 2cfdb95` ثم `git log --oneline rescue`. لو هو ده، ادمجه أو اعمل reset عليه. الـ Branch مبيضرّش حد.

إنقاذ Branch اتمسح

مسحت `feature/dashboard` بـ `git branch -D`؟ Git بيطبعلك الـ Hash وهو بيمسح:

Terminal

```
you@laptop:~/portfolio$ git branch -D feature/dashboard
Deleted branch feature/dashboard (was 2cfdb95).
you@laptop:~/portfolio$ git branch feature/dashboard 2cfdb95      # رجع!
```

ولو الرسالة ضاعت، دوّر في الـ reflog على آخر Commit كنت عليه في الـ Branch ده (`git reflog | grep dashboard`، افتكر `grep` من كتاب الـ Terminal).

الموقف	ينفع؟	الحل
`reset --hard` غلط	أيوه	`git reset --hard HEAD@{1}`
Branch اتمسح بـ `-D`	أيوه	`git branch name <hash>`

الموقف	ينفع؟	الحل
Rebase بوّظ الدنيا	أيوه	دوّر على `rebase (start)` وارجع للي قبله
`commit --amend` **وعاوز القديم**	أيوه	الـ Commit القديم في `HEAD@{1}`
Commits في Detached HEAD	أيوه	`git branch keep <hash>`
تعديلات ما اتعملهاش Commit واترمت بـ restore أو reset --hard	**لأ**	ملهاش أي أثر في Git

جدول 16.1 الـ reflog بينقذ إيه؟

> **لو حابب تتعمق: حدود الـ reflog**
>
> الـ reflog **محلي**: موجود على جهازك بس، ومبيتبعتش لـ GitHub، و `git clone` جديد بيبدأ بـ reflog فاضي. وهو **مؤقت**: Git بيمسح المداخل القديمة أوتوماتيك وقت الصيانة (`git gc`): الافتراضي 90 يوم للمداخل العادية، و 30 يوم للـ Commits اللي بقت مش موجودة في أي Branch (الإعدادين `gc.reflogExpire` و `gc.reflogExpireUnreachable`). وكل Branch ليه reflog خاص بيه كمان: `git reflog show main` أو `main@{2}`، وفيه شكل بالوقت: `main@{yesterday}` و `HEAD@{2.hours.ago}`. وده بالظبط ليه قلنا من أول الكتاب: **اللي اتعمله Commit تقريباً مبيضيعش**.

16.3 git bisect: لاقي الـ Commit اللي دخّل الـ Bug

موقف تاني: زرار التحويل في تطبيق البنك كان شغال من أسبوعين في الإصدار `v1.0.0`. النهارده مش شغال. وبينهم 64 Commit. هتفتح كل واحد وتجرّب؟ ده يوم كامل. طب تفتكر لعبة "فكّر في رقم من 1 لـ 100 وأنا أخمّنه"؟ أذكى طريقة إنك تقول 50، والتاني يقولك "أكبر" أو "أصغر"، فتقول 75، وهكذا. كل سؤال بيقسم المسافة نصين. بـ 7 أسئلة بس تلاقي أي رقم من 100. ده اسمه **البحث الثنائي** (Binary search)، و `git bisect` بيعمله على التاريخ: 64 Commit = 6 تجارب بس.

1. **ابدأ:** `git bisect start`.
2. **قوله فين البايظ:** `git bisect bad` (من غير حاجة = HEAD الحالي).
3. **وفين السليم:** `git bisect good v1.0.0` (Tag أو Hash).
4. **Git بيعمل switch** لـ Commit في النص بالظبط (Detached HEAD). جرّب التطبيق.
5. **جاوب:** شغال؟ `git bisect good`. بايظ؟ `git bisect bad`. ومش عارف تختبر الـ Commit ده؟ `git bisect skip`.
6. **كرر** لحد ما Git يقولك `... is the first bad commit` ويعرضه.
7. **ارجع:** `git bisect reset` يرجّعك للـ Branch اللي كنت عليه.

```
Terminal
you@laptop:~/portfolio$ git bisect start
you@laptop:~/portfolio$ git bisect bad
you@laptop:~/portfolio$ git bisect good v1.0.0
Bisecting: 31 revisions left to test after this (roughly 5 steps)
[bfef333] feat: add currency formatter
you@laptop:~/portfolio$ git bisect good
Bisecting: 15 revisions left to test after this (roughly 4 steps)
...
4e58f56 is the first bad commit
commit 4e58f565b8178581883a702b51b150635e327e21
    feat: refactor transfer validation
you@laptop:~/portfolio$ git bisect reset
```

تشريح الأمر

```
git bisect run npm test
```

1. `git bisect run` بدل ما تجاوب بإيدك، شغّل الأمر ده على كل Commit، وحدد good أو bad من الـ **Exit code** بتاعه.
2. `npm test` أي أمر: لو خرج بـ `0` يبقى good، ولو من 1 لـ 127 (ما عدا 125) يبقى bad، و `125` = skip. ده نفس الـ Exit codes اللي اتعلمناها في كتاب الـ Terminal، وأي Script بيرجع 0 أو 1 ينفع.

bisect أوتوماتيك بسطر واحد

لو الـ Bug إن كلمة معينة ظهرت في ملف، تقدر تختبر بـ grep: `git bisect run sh -c '! grep -q "debugger" app.js'` . الـ `grep -q` بيرجّع 0 لو لقى الكلمة، والـ `!` بتعكس الـ Exit code، فلو الكلمة موجودة = bad. ولما نجرّبها على 8 Commits، Git بيوصل للـ Commit المذنب في تجربتين أو تلاتة.

16.4 git blame: مين كتب السطر ده؟

الاسم بيخوّف ("لوم")، بس الغرض الحقيقي مش إنك تلوم حد، هو إنك تفهم: السطر الغريب ده جه ليه؟ ممكن يكون في سبب منطقي في رسالة الـ Commit اللي ضافه. (ونص المرات هتكتشف إن اللي كاتبه انت من شهرين.)

تشريح الأمر

```
git blame -L 10,20 app.js
```

1. `git blame` جنب كل سطر في الملف: آخر Commit غيّره، ومين، وإمتى.
2. `-L 10,20` Lines: من سطر 10 لـ 20 بس، بدل الملف كله.
3. `app.js` الملف.

```
Terminal
you@laptop:~/portfolio$ git blame -L 12,13 app.js
4e58f565 (Sara  2026-10-12 14:02:11 +0300 12)   if (amount > balance) return;
a12c9e40 (Omar  2026-09-30 09:15:40 +0300 13)   balance -= amount;
```

`git blame -w file`	تجاهل تغييرات المسافات (عشان الـ Commit الـ Prettier ميطلعش هو "الكاتب" لكل حاجة).
`git blame -C file`	اكتشف السطور اللي اتنقلت أو اتنسخت من ملفات تانية.
`git show 4e58f565`	الخطوة الجاية: اقرا الـ Commit كله ورسالته عشان تفهم ليه.
`git log -L 12,13:app.js`	تاريخ السطرين دول بس عبر كل الـ Commits.

على GitHub و VS Code

في أي ملف على GitHub فيه زرار **Blame** فوق الكود، وبيخليك تدوس على Commit وترجع لقبله. وفي VS Code إضافة GitLens بتعرض الـ blame جنب السطر وانت واقف عليه. وعشان الـ Formatting commits متلخبطش الـ blame على GitHub، الفرق بتحطها في ملف `.git-blame-ignore-revs` .

ملخص الفصل

- `git reflog` = دفتر كل حركات HEAD على جهازك، و `HEAD@{n}` اسم لأي مكان فيه.
- بينقذ: reset غلط، Branch اتمسح، Rebase بايظ، amend. اعمل `git branch rescue <hash>` الأول.
- مبينقذش اللي ما اتعملهوش Commit، وهو محلي ومؤقت (90/30 يوم).
- `git bisect start / bad / good / reset` بحث ثنائي، و `bisect run` بالـ Exit codes.
- `git blame -L` و `-w` ، وبعدين `git show` تعرف ليه.

صح ولا غلط؟

1. الـ reflog بيتبعت لـ GitHub مع الـ push. صح / غلط
2. `git reset --hard` بيمسح الـ Commits نهائياً من أول لحظة. صح / غلط
3. مع `bisect run` ، Exit code صفر معناه Commit سليم. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `git log` و `git reflog` ؟
2. اكتب خطوات إنقاذ Branch اتمسح بـ `-D` .
3. بين Commit سليم وCommit بايظ فيه 1000 Commit. bisect هيحتاج كام تجربة تقريباً؟ وليه؟
4. ليه `git blame -w` مفيد بعد Commit فيه Format للمشروع كله؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل 3 Commits، و `git reset --hard HEAD~3`، ورجّعهم بالـ reflog.
2. متوسط اعمل Branch فيه Commit، وامسحه بـ `-D`، ورجّعه من غير ما تبص على رسالة المسح (من الـ reflog بس).
3. صعب اعمل Script بيعمل Commit 20، ويدخّل كلمة `BUG` في Commit رقم 13. لاقيه بـ `git bisect run` وسطر `grep`.

.

17

المرحلة 7 · GitHub والـ AI والإنقاذ

الفصل السابع عشر

GitHub كمنصة: README و Pages و Actions و gh

مشروعك على GitHub دلوقتي، بس لو حد فتح الصفحة هيلاقي قايمة ملفات وبس. في الفصل ده هنحوّله لحاجة تتفرج عليها: README يشرح المشروع بـ Markdown، وموقع شغال على النت ببلاش بـ GitHub Pages، وروبوت صغير بـ GitHub Actions بيختبر الكود مع كل push، وأداة gh اللي بتعمل كل ده من الـ Terminal.

بعد الفصل ده هتكون عارف

- README كويس فيه إيه، وأساسيات Markdown بالتفصيل
- نشر مشروع JavaScript عادي على GitHub Pages
- نشر مشروع Vite على Pages بـ Actions و base
- GitHub Actions: الـ Workflow والـ Jobs والـ Steps سطر سطر
- أوامر gh الأساسية: repo و pr و issue و run

17.1 الـ README: واجهة مشروعك

تخيل إنك بتقدّم على شغل، والـ Recruiter فتح لينك GitHub بتاعك. عنده 30 ثانية بالكتير لكل مشروع. لو فتح المشروع ولقى قايمة ملفات `index.html` و `script.js` وبس، هيقفل. لو لقى صورة للتطبيق، وجملتين بيقولوا ده بيعمل إيه، ولينك يجرّبه، وقايمة باللي اتعلمته وانت بتعمله، هيقف شوية. الفرق ده كله ملف واحد: `README.md`.

GitHub بيعرض الملف ده أوتوماتيك تحت قايمة الملفات في صفحة أي Repo. والـ `.md` يعني **Markdown**: طريقة تكتب بيها نص منسّق برموز بسيطة، من غير HTML. وانت كـ Content Engineer غالباً قابلتها قبل كده، وهي نفس اللغة اللي بتكتب بيها Issues و PRs والتعليقات على GitHub.

README.md

```
# Bankist

A minimal banking app built with
**vanilla JavaScript**.

![App screenshot](docs/screenshot.png)

**Live demo:** [sara.github.io/bankist]
(https://sara.github.io/bankist/)
```

1. `#` = عنوان كبير (H1). و `##` = H2، لحد `######`. لازم مسافة بعد الـ `#`.
2. `**bold**` = عريض، و `*italic*` = مايل، و `~~text~~` = مشطوب.
3. صورة: `!` + النص البديل بين `[ ]` + المسار بين `( )`. حط الصورة جوه الـ Repo.
4. لينك: نفس الصورة من غير `!`.

5 قايمة بـ `-` (أو `*`). و `- [x]` / `- [ ]` = Checkbox Task) list).	`## Features`
	`- Login with PIN`
	`- Transfers and loans`
	`- [x] Sorting movements`
	`- [ ] Dark mode`

6 كود: 3 Backticks + اسم اللغة عشان الألوان، وتقفل بـ 3. وكود في نص السطر بـ Backtick واحدة: `` `npm install` ``.	`## Run locally` ` ```bash ` `git clone` `git@github.com:sara/bankist.git` `cd bankist && npx serve .` ` ``` `
7 `>` = اقتباس. و `[!NOTE]` و `[!TIP]` و `[!WARNING]` = صناديق ملونة خاصة بـ GitHub (Alerts).	`> [!NOTE]` `> Demo accounts: js / 1111`

8 جدول: `|` بين الأعمدة، والسطر التاني `---` بيفصل العنوان.	`| Tech | Why |`
	`|------|-----|`
	`| JS | course project |`

README مشروع كورس

عشان الـ README يبيّن إنك فاهم مش ناسخ: اكتب **اللي اتعلمته** (Closures، الـ Array methods...)، و**اللي ضفته من عندك** زيادة على الكورس، والحاجات اللي ناوي تعملها. ولو المشروع من كورس، اذكر ده بأمانة. واعمل كمان Repo خاص باسم حسابك بالظبط (`sara/sara`): الـ README بتاعه بيظهر في صفحة البروفايل.

17.2 GitHub Pages: موقعك على النت ببلاش

مشروع HTML و CSS و JavaScript عادي؟ GitHub Pages بيستضيفه على `https://sara.github.io/bankist/` مجاناً، من غير سيرفر ولا Hosting ولا أي حاجة. ولأنه بيعرض ملفات ثابتة (Static)، فهو مناسب لأي مشروع Frontend، مش لـ Backend بـ Node.

مشروع Vanilla JS: من غير Build

1. اتأكد إن `index.html` في أول الـ Repo (الجذر)، وإنه اتعمله push على main.
2. على GitHub: **Settings ← Pages**.
3. في **Build and deployment** ← **Source** اختار **Deploy from a branch**.
4. اختار Branch `main` وفولدر `/ (root)`، و Save.
5. استنى دقيقة أو اتنين، والرابط هيظهر فوق في نفس الصفحة. وكل push جديد على main = الموقع يتحدث لوحده.

> ⚠ **الموقع شغال بس الـ CSS والصور مش ظاهرين**
>
> غالباً المسارات. موقعك مش على `/`، هو على `/bankist/`. فـ `<link href="/style.css">` (بـ `/` في الأول) هيدوّر على `sara.github.io/style.css` ومش هيلاقيه. استخدم مسارات نسبية: `href="style.css"` أو `./style.css` (افتكر المسارات المطلقة والنسبية من كتاب الـ Terminal). وكمان GitHub Pages بيفرّق بين الحروف الكبيرة والصغيرة: `Logo.png` غير `logo.png`، حتى لو على الـ Mac اشتغلت.

مشروع Vite: محتاج Build

لو بتستخدم Vite (أو Parcel اللي في كورس Jonas)، الكود اللي بتكتبه مش هو اللي بيتنشر: لازم `npm run build` يطلّع فولدر `dist`. و `dist` في الـ gitignore.. فإزاي GitHub ينشره؟ نخلي GitHub يعمل الـ Build بنفسه بـ Actions. خطوتين:

1 · قول لـ Vite إن الموقع هيبقى في فولدر فرعي:

vite.config.js

```
import { defineConfig } from "vite";

export default defineConfig({
  base: "/bankist/", // اسم الـ Repo بين شرطتين
});
```

2 · في **Settings ← Pages ← Source** اختار **GitHub Actions**، واعمل الملف ده:

.github/workflows/deploy.yml

```
name: Deploy to Pages
on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: 24
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v5
        with:
          path: dist
```

```yaml
deploy:
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${{ steps.deployment.outputs.page_url }}
  steps:
    - id: deployment
      uses: actions/deploy-pages@v5
```

`on: push: branches: [main]`	اشتغل مع كل push على main.
`permissions`	صلاحيات الـ Token اللي الـ Workflow بيشتغل بيه: يقرا الكود، ويكتب على Pages، و `id-token` عشان يثبت إن النشر جاي من الـ Repo ده.
`jobs: build`	هات الكود، وسطّب Node 24، و `npm ci` (تسطيب نضيف من `package-lock.json` بالظبط)، و Build، وارفع `dist` كـ Artifact.
`needs: build`	الـ Job التاني (deploy) يستنى الأول يخلص بنجاح.
`${{ … }}`	تعبير GitHub Actions: هنا رابط الموقع بعد النشر، بيظهر في صفحة التشغيل.
`deploy-pages`	بينشر الـ Artifact على Pages.

دقة: الإصدارات دي من أكتوبر 2026

الأرقام بعد `@` هي الـ Major version للـ Action: `checkout@v7` و `setup-node@v7` (بيشتغلوا على Node 24)، و `upload-pages-artifact@v5` و `deploy-pages@v5`. لو بتقرا الكتاب بعد كده بفترة، شوف صفحة كل Action على GitHub لآخر رقم. وفيه نسخ أقدم في شروحات كتير على النت (`@v3` و `@v4`) لسه ممكن تلاقيها.

17.3 GitHub Actions: روبوت بيشتغل مع كل push

شفنا Actions بيعمل Build وينشر. بس استخدامه الأشهر حاجة اسمها **CI** (Continuous Integration): كل push أو PR، GitHub يشغّل الاختبارات والـ Linter على جهاز نضيف في السحابة. لو حاجة فشلت، هتلاقي ✗ حمرا جنب الـ Commit وفي الـ PR، قبل ما الكود البايظ يدخل main. وده بالظبط الـ "Checks" اللي شفناها في صفحة الـ PR (↩ 11.2 ص 84).

YAML في سطرين

YAML صيغة لكتابة الإعدادات، شبه JSON بس من غير أقواس: كل سطر `key: value`، والمسافات في أول السطر هي اللي بتقول مين جوه مين (زي Python بالظبط، ومفيش Tab، مسافات بس)، والشرطة `-` في أول السطر = عنصر في List، و `#` = تعليق. هتشوفها تاني في Docker Compose و Kubernetes، وشرحها الكامل في كتاب APIs و JSON (فصل 21).

الفكرة بسيطة لما تعرف الكلمات: **Workflow** = ملف YAML جوه `.github/workflows/`. جواه **on** = إمتى يشتغل، و **jobs** = شغلانات، كل Job بيشتغل على جهاز (Runner) لوحده، وجوه كل Job **steps** = خطوات ورا بعض: يا `uses` (استخدم Action جاهز) يا `run` (شغّل أمر Shell).

.github/workflows/ci.yml

1 الاسم اللي هيظهر في تاب Actions.	`name: CI`

2 الـ Triggers: كل push على main، وكل PR.	`on:`
	`  push:`
	`  branches: [main]`
	`  pull_request:`

3 Job اسمه `test` على جهاز Ubuntu جديد كل مرة.	`jobs:`
	`  test:`
	`  runs-on: ubuntu-latest`

4 هات كود الـ Repo على الجهاز (من غيرها الجهاز فاضي).	`  steps:` `  - uses: actions/checkout@v7`

5 سطّب Node 24، و `cache: npm` يحفظ المكتبات بين المرات عشان يبقى أسرع.	`  - uses: actions/setup-node@v7`
	`  with:`
	`  node-version: 24`
	`  cache: npm`

6 أوامر Shell عادية. لو أي واحد خرج بـ Exit code غير صفر، الـ Job يفشل والـ ✗ تظهر. نفس فكرة `set -e` من كتاب الـ Terminal.	`  - run: npm ci`
	`  - run: npm run lint`
	`  - run: npm test`

ببلاش؟

للـ Repos الـ Public: Actions مجانية على الـ Runners بتاعة GitHub. للـ Private: فيه دقايق مجانية كل شهر حسب خطتك، وبعدها بفلوس. وتقدر تشوف كل تشغيل وكل سطر Log في تاب **Actions**، وتشغّل Workflow بإيدك لو ضفت `workflow_dispatch:` تحت `on`.

الأسرار في Actions

لو الـ Workflow محتاج API Key، متكتبهوش في الملف أبداً. حطه في **Settings ← Secrets and variables ← Actions**، واستخدمه كده: `${{ secrets.WEATHER_API_KEY }}`. GitHub بيخفيه من الـ Logs.

17.4 gh: GitHub من الـ Terminal

كل اللي عملناه في الموقع (Repo جديد، PR، Issues، Releases) ينفع من الـ Terminal بأداة GitHub الرسمية `gh`. تسطيبها على الـ Mac: `brew install gh`، وبعدين `gh auth login` (↩ 9.5 ص 73).

تشريح الأمر

```
gh repo create bankist --public --source=. --remote=origin --push
 1       2      3        4          5              6
```

1. `gh repo create` اعمل Repo جديد على GitHub.
2. `bankist` اسمه.
3. `--public` عام (أو `--private`).
4. `--source=.` من الـ Repo المحلي اللي في الفولدر الحالي (الـ `.`) بدل ما يعمل واحد فاضي.
5. `--remote=origin` ضيفه كـ Remote بالاسم ده (بدل `git remote add`).
6. `--push` واعمل push للـ Commits على طول. كده الفصل 9 كله في سطر.

تشريح الأمر

```
gh pr create --fill --base main
 1       2      3
```

1. `gh pr create` افتح PR للـ Branch اللي انت عليه.
2. `--fill` خد العنوان والوصف من الـ Commits أوتوماتيك.
3. `--base main` عاوزه يتدمج في main.

الأمر	بيعمل إيه
`gh repo clone sara/bankist`	clone من غير ما تجيب الرابط
`gh repo view --web`	افتح صفحة الـ Repo في المتصفح
`gh pr status` / `gh pr list`	الـ PRs المفتوحة / حالة الـ PRs بتاعتك
`gh pr checkout 12`	هات Branch الـ PR رقم 12 عندك عشان تجرّبه
`gh pr diff 12`	اعرض الـ diff بتاعه
`gh pr merge 12 --squash --delete-branch`	ادمجه Squash وامسح الـ Branch
`gh issue create -t "title" -b "body"`	Issue جديد (`-t` عنوان و `-b` Body)
`gh issue list --label bug`	الـ Issues بـ Label معين
`gh run watch` / `gh run list`	تشغيلات Actions / تابع اللي شغال لايف
`gh release create v1.0.0 --generate-notes`	Release (↩ 14.5 ص 108)

جدول 17.1 أوامر gh اللي هتستخدمها

حاجة مشتركة: نفس شكل git

`gh` ماشي على نفس النمط: برنامج + موضوع (`repo` و `pr` و `issue`) + فعل (`create` و `list` و `view`) + Flags. و `gh help` و `gh pr create --help` للمساعدة، و `--web` في أوامر كتير بتفتح نفس الحاجة في المتصفح.

ملخص الفصل

- README بـ Markdown: `#` عناوين، `**` عريض، `[]()` لينك، `![]()` صورة، و 3 Backticks للكود.
- Pages لمشروع عادي: Deploy from a branch، ومسارات نسبية.
- Vite: `base: "/repo/"` + Workflow بيعمل Build وينشر `dist`.
- Actions: `on` و `jobs` و `steps` (`uses` أو `run`)، وأي Exit code غير صفر = فشل.
- `gh`: `repo create --source=. --push` و `pr create --fill` و `pr checkout`.

صح ولا غلط؟

1. GitHub Pages ينفع يشغّل Backend بـ Express. صح / غلط
2. في مشروع على Pages، `href="/style.css"` هيشتغل عادي. صح / غلط
3. كل Job في Actions بيشتغل على جهاز جديد. صح / غلط

أسئلة مراجعة

1. اكتب README صغير فيه عنوان، وصورة، ولينك، وقايمة، وبلوك كود.
2. ليه مشروع Vite محتاج `base` على Pages؟
3. اشرح الفرق بين `uses` و `run` في الـ steps.
4. إزاي تحط API Key يستخدمه الـ Workflow من غير ما يتكتب في الكود؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل انشر مشروع HTML بسيط من الكورس على GitHub Pages بـ Deploy from a branch.
2. متوسط اكتب README للمشروع فيه Screenshot ولينك الـ Live demo و Features كـ Task list.
3. صعب ضيف Workflow CI بيعمل `npm ci` و `npm test`، واعمل PR فيه اختبار فاشل عمداً عشان تشوف الـ ✗، وبعدين صلّحه.

18

المرحلة 7 · GitHub والـ AI والإنقاذ

الفصل الثامن عشر

Git والـ AI: اشتغل بأمان

انت بتبني مشاريع بمساعدة الـ AI، وده ممتاز. بس الـ AI ساعات بيعدّل عشر ملفات وانت طالب منه سطر، وساعات بيحط API Key في الكود كأنه حاجة عادية. Git هو حزام الأمان بتاعك هنا: Checkpoint قبل أي تعديل، و Branch لكل تجربة، و diff تقراه قبل ما تقبل، وقواعد صارمة للأسرار.

بعد الفصل ده هتكون عارف

- Commit قبل ما الـ AI يلمس الكود، وليه
- Branch لكل تجربة AI، والرجوع منها في ثانية
- تقرا تعديلات الـ AI بـ diff --stat و add -p
- .env و .env.example. وليه الأسرار مكانها مش Git
- تعمل إيه لو Secret اتسرّب، و Push protection

18.1 الـ AI عدّل 14 ملف

الساعة 2 بالليل. مشروع الـ Weather app شغال تمام، وناقص بس إن الـ Search يقبل اسم المدينة بالعربي. كتبت للـ AI: "خلّي الـ search يدعم العربي". رد بثقة: "تم! عملت بعض التحسينات كمان". فتحت المتصفح... الصفحة بيضا. الـ Console فيه 3 Errors. والـ AI "حسّن" ملف الـ CSS، وغيّر اسم Function بتستخدمها في 4 أماكن، وحدّث `package.json`.

تطلب منه يصلّح؟ هيصلّح حاجة ويبوّظ حاجة تانية، وانت مش فاكر الكود كان شكله إيه. السيناريو ده بيحصل لكل اللي بيشتغلوا مع AI، والفرق الوحيد بين اللي بيخسر ساعتين واللي بيخسر 3 ثواني هو سطر واحد كان المفروض يتكتب **قبل** ما يطلب من الـ AI أي حاجة.

18.2 القاعدة الأولى: Commit قبل الـ AI

قبل ما تدّي الـ AI أي مهمة بتعدّل كود، خلّي الـ Working directory نضيف: `git status` يقول `nothing to commit, working tree clean`. كده أي حاجة هتتغير بعد كده هي تعديل الـ AI **بس**، وتقدر تشوفه كله وترجع منه كله.

Terminal

```
git status
git add -A
git commit -m "chore: checkpoint before AI arabic search"
```

ولو الـ AI بوّظ الدنيا؟ ترجع لآخر Commit بأمر واحد (افتكر ↩ 6.2 ص 47 و ↩ 6.6 ص 51):

تشريح الأمر

```
git restore . && git clean -fd
```

1. `git restore .` رجّع كل الملفات المتتبَّعة لحالة آخر Commit.
2. `&&` ولو نجح، كمّل (من كتاب الـ Terminal).
3. `git clean -fd` امسح الملفات الجديدة اللي الـ AI عملها ومش متتبَّعة: `-f` تأكيد و `-d` الفولدرات كمان. جرّب `-n` الأول تشوف هيمسح إيه.

مش لازم Commit؟

لو مش حابب تعمل Commit لنص شغلك، فيه حل تاني: `git stash` (↩ 14.2 ص 105)، أو على الأقل `git add -A` من غير Commit. كده الـ Staging area فيها نسختك، و `git diff` (من غير `--staged`) يوريك تعديلات الـ AI بس، و `git restore .` يرجّع للنسخة اللي في الـ Staging. بس الـ Commit أوضح وأأمن.

18.3 Branch لكل تجربة

لو المهمة كبيرة ("اعمل Refactor للمشروع كله لـ Modules")، خليها في Branch لوحدها. لو التجربة نجحت، تدمجها. لو فشلت، تمسح الـ Branch ولا كأن حاجة حصلت، و main عمره ما اتلمس.

Terminal

```
git switch -c ai/modules-refactor
# ... الـ AI يشتغل، وانت تعمل commits صغيرة بعد كل خطوة ناجحة
git switch main
git merge --squash ai/modules-refactor    # نجحت: Commit واحد نضيف
git branch -D ai/modules-refactor         # فشلت: امسحها بالـ D الكبيرة
```

Commits صغيرة أثناء الشغل مع الـ AI

كل ما الـ AI يعمل خطوة شغالة، اعمل Commit. لو الخطوة الخامسة باظت، ترجع للرابعة مش للأول خالص. و `merge --squash` في الآخر بيجمعهم في Commit واحد على main لو مش عاوز التاريخ ده يظهر (↩ 11.4 ص 86).

لو حابب تتعمق: git worktree

عاوز تخلّي أداة AI تشتغل على Branch في نفس الوقت اللي انت شغال فيه على main؟ `git worktree add ../weather-ai ai/try` بيعمل فولدر تاني فيه نفس الـ Repo على Branch تانية، من غير clone جديد. الاتنين بيشاركوا نفس `.git`. و `git worktree list` تشوفهم و `git worktree remove ../weather-ai` تمسحه. أدوات AI كتير بتستخدم الفكرة دي عشان تشغّل أكتر من Agent على المشروع.

18.4 اقرا قبل ما تقبل

الـ AI مش زميل بتثق فيه ثقة عمياء. هو زميل شاطر جداً وسريع جداً، وساعات بيكدب بثقة. فتعامل مع تعديلاته زي PR من حد جديد في الفريق: تراجعها. والمراجعة ليها 3 مستويات:

تشريح الأمر

```
git diff --stat
```
1

1 `--stat` ملخص: أنهي ملفات اتغيرت، وكام سطر اتضاف (`+`) واتشال (`-`) في كل واحد. أول نظرة: لو طلبت تعديل في `search.js` ولقيت 14 ملف، فيه حاجة غلط.

Terminal

```
package.json   |   4 +-
src/search.js  |  23 ++++++++---
src/style.css  | 112 ++++++++------------------------
3 files changed, 48 insertions(+), 91 deletions(-)
```

هنا تسأل: ليه `style.css` اتغير 112 سطر وانا ماطلبتش؟ وليه `package.json` ؟ بعد كده `git diff src/search.js` تقرا التعديل اللي طلبته فعلاً سطر سطر. والمستوى التالت إنك تقبل حتة حتة:

تشريح الأمر

```
git add -p
```
1

1 `-p` (`--patch`) يعرض كل تعديل (Hunk) لوحده ويسألك: `y` ضيفه، `n` لأ، `s` قسّمه أصغر، `q` خلاص (↩ 13.5 ص 100). تقبل تعديل الـ search، وترفض الـ CSS.

وبعد ما تعمل Commit للي قبلته، ترمي الباقي بـ `git restore .` . وبكده خدت من الـ AI اللي انت عاوزه بالظبط.

> **بصّ على package.json و lock files كويس**
>
> لو الـ AI ضاف مكتبة جديدة، اتأكد إنها موجودة فعلاً وإن اسمها مظبوط على npmjs.com. الـ AI ساعات بيخترع أسماء مكتبات مش موجودة، وفيه ناس بتسجّل الأسماء دي بالظبط بكود خبيث، مستنية حد ينزّلها.

إمتى	الأمر
قبل	`git status` نضيف، أو `git commit` Checkpoint، أو `git switch -c ai/...`
بعد، نظرة أولى	`git diff --stat`
بعد، قراية	`git diff path/file.js`

إمتى	الأمر
قبول جزئي	`git add -p` ثم `git commit`
رفض الباقي	`git restore .` و `git clean -fd`
Commit اتعمل وطلع غلط	`git reset --hard HEAD~1` (محلي) أو `git revert` (اتعمله push)

جدول 18.1 الـ Workflow الكامل مع الـ AI

18.5 الأسرار: القاعدة اللي مفيهاش هزار

الـ Weather app محتاج API Key. الـ AI كتب لك الكود كده:

src/api.js (غلط)

```
const API_KEY = "sk_live_9f3a7c2e81b4d6...";  // متعملش كده
fetch(`https://api.weather.dev/v1?q=${city}&key=${API_KEY}`);
```

عملت Commit و push على Repo Public. خلال دقايق، فيه Bots بتلف على GitHub 24 ساعة بتدوّر على مفاتيح زي دي. ولو المفتاح ده لخدمة بتدفع فيها (AWS، OpenAI...)، ممكن تصحى على فاتورة بآلاف الدولارات. وحتى لو مسحت السطر في Commit جديد، المفتاح لسه في الـ **History**: أي حد يعمل `git log -p` يلاقيه. Git مصمم إنه ما ينساش، وده هنا ضدك.

الحل المعروف: ملف `.env` للأسرار، متجاهَل في Git، وملف `.env.example` متتبَّع فيه أسماء المتغيرات بس:

.env (في .gitignore)

.env

```
WEATHER_API_KEY=sk_live_9f3a7c2e81b4d6
MAPS_TOKEN=pk_83hd92
```

.env.example (يتعمله commit)

.env.example

```
WEATHER_API_KEY=your_key_here
MAPS_TOKEN=your_token_here
```

.gitignore

```
.env
.env.*
!.env.example
```

الـ `!` بتعمل استثناء (↩ 5.2 ص 41): تجاهل كل ملفات `.env` ما عدا الـ example. وقبل أول Commit في أي مشروع، اتأكد: `git check-ignore -v .env` لازم يطبع القاعدة (↩ 5.5 ص 43).

مهم جداً لـ Frontend

أي مفتاح بتستخدمه في كود بيشتغل في المتصفح (حتى لو جاي من `.env` بـ `import.meta.env.VITE_KEY` في Vite)، بيتحط جوه ملفات الـ Build، وأي حد يفتح DevTools يشوفه. الـ `.env` بيحميه من Git، مش من المتصفح. المفاتيح الحساسة فعلاً مكانها Backend أو Serverless function. والمفاتيح اللي معمولة للمتصفح (Public keys) اعمل لها قيود من لوحة الخدمة (Domains مسموحة مثلاً).

قول للـ AI القواعد

لما تطلب من الـ AI كود فيه API، قوله صراحة: "استخدم متغير بيئة من `.env`، وماتكتبش أي مفتاح في الكود، وضيف `.env.example`". ومتلصقش مفاتيحك الحقيقية في الـ Chat أصلاً: اديله `your_key_here`.

18.6 المفتاح اتسرّب. اعمل إيه؟

حصل. ماتتكسفش، بيحصل لناس بخبرة سنين. الترتيب هنا مهم جداً:

1. **الغي المفتاح فوراً (Rotate / Revoke)** من لوحة الخدمة واعمل واحد جديد. ده أهم خطوة وأول خطوة. افترض إن المفتاح القديم اتشاف خلاص، حتى لو الـ Repo Private، وحتى لو مسحته بعد دقيقة.
2. حط الجديد في `.env`، وتأكد إن `.env` في الـ gitignore..
3. شيل المفتاح من الكود واعمل Commit.
4. **لو لسه ماعملتش push:** صلّح الـ Commit نفسه: `git commit --amend` لو هو آخر واحد، أو `git rebase -i` لو أقدم (↩ 15.5 ص 113). كده المفتاح مش في الـ History خالص.
5. **لو اتعمله push:** تنظيف الـ History ممكن (بأداة زي `git filter-repo`)، بس معقد ومحتاج force push، والنسخ اللي اتعملها clone أو fork لسه فيها المفتاح. عشان كده الخطوة 1 هي اللي بتحميك فعلاً، مش التنظيف.

GitHub بيساعدك: Push protection g Secret scanning

GitHub بيعمل **Secret scanning** على الـ Repos الـ Public: بيدوّر على أنواع مفاتيح معروفة، وكتير من الشركات (زي AWS) بتبلّغها GitHub فتلغي المفتاح بنفسها. وفيه كمان **Push protection**: لو حاولت تعمل push لـ Commit فيه مفتاح معروف، الـ push يترفض برسالة زي `GH013: Repository rule violations found` و `Push cannot contain secrets`، وبيقولك المفتاح في أنهي ملف وأنهي سطر. ساعتها ماتعملش "Allow" وتكمّل: صلّح الـ Commit (خطوة 4) وغيّر المفتاح. تقدر تتأكد إنها شغالة على حسابك من **Settings ← Code security**. بس دي شبكة أمان، مش بديل عن الـ gitignore..

ملخص الفصل

- Working directory نضيف أو Commit Checkpoint قبل أي مهمة للـ AI.
- التجارب الكبيرة في `ai/...` Branch: تدمجها أو تمسحها.
- `diff --stat` ثم `diff file` ثم `add -p`، والباقي `restore .` و `clean -fd`.
- الأسرار في `.env` (متجاهَل)، والأسماء في `.env.example`.
- مفتاح اتسرّب = Rotate الأول، وبعدين نصّف.

صح ولا غلط؟

1. لو مسحت المفتاح في Commit جديد وعملت push، المشكلة اتحلت. صح / غلط
2. `.env.example` المفروض يكون فيه المفاتيح الحقيقية عشان الفريق. صح / غلط
3. `git diff --stat` بيوريك أنهي ملفات اتغيرت وكام سطر. صح / غلط

أسئلة مراجعة

1. ليه الـ Working directory النضيف قبل الـ AI بيسهّل المراجعة والرجوع؟
2. إزاي تقبل جزء من تعديلات الـ AI وترفض الباقي؟
3. اكتب قواعد gitignore. تتجاهل كل ملفات env. ما عدا env.example..
4. رتّب خطوات التعامل مع مفتاح اتعمله push.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في مشروع، اعمل Checkpoint، واطلب من الـ AI تعديل، واقرا `git diff --stat` قبل ما تفتح الكود.
2. متوسط اطلب من الـ AI تعديلين في نفس الملف، واقبل واحد بس بـ `git add -p`.
3. صعب في Repo تجريبي، اعمل Commit فيه مفتاح وهمي (مش push)، وبعدين شيله من الـ History بـ `rebase -i` واتأكد بـ `git log -p` إنه مش موجود.

19

المرحلة 7 · GitHub والـ AI والإنقاذ

الفصل التاسع عشر

الإنقاذ: الغلطات الشائعة وحلها

الفصل ده مش هتقراه مرة واحدة. ده الفصل اللي هترجعله الساعة 2 بالليل وقلبك بيدق. كل غلطة فيه حصلت لكل واحد اشتغل بـ Git، وكلها ليها حل. هنبدأ ببروتوكول الهدوء، وبعدين الغلطات متقسمة حسب الموقف، وكل واحدة بالرسالة اللي هتشوفها والحل خطوة بخطوة.

بعد الفصل ده هتكون عارف

- بروتوكول أول 60 ثانية لما حاجة تبوظ
- Commit على Branch غلط، أو رسالة أو Author غلط
- Detached HEAD، وشغل ضاع، و Merge أو Rebase نص نص
- رسائل الـ push والـ pull المشهورة وحلها
- ملف كبير، DS_Store.، ومشكلة git init في الـ Home

19.1 أول 60 ثانية: متعملش حاجة

أكتر حاجة بتبوّظ Repos مش الغلطة الأولى، دي الغلطة التانية: تتخض، تنسخ أول أمر من أول نتيجة في Google فيه `--force` أو `reset --hard`، وتشغّله. الخبر الحلو اللي اتعلمناه في ↩ 16.2 ص 116: أي حاجة اتعملها Commit تقريباً مستحيل تضيع. فخد نفس، وامشي بالترتيب ده:

1. **اقرا** `git status` **كويس.** Git بيقولك انت فين (أنهي Branch، في نص Merge ولا Rebase)، وغالباً بيقولك الأمر اللي يطلّعك.
2. **اعمل نسخة احتياطي من الفولدر كله** لو قلقان: `cp -R weather weather-backup`. كده أي حاجة تجرّبها مفيهاش مخاطرة.
3. **شوف التاريخ:** `git log --oneline --graph --all -10` و `git reflog -10`.
4. **دوّر على حالتك في الفصل ده** بالرسالة اللي ظاهرة لك.
5. **أي أمر فيه** `--hard` **أو** `--force` **أو** `clean -f`، افهمه الأول. التلاتة دول بيمسحوا حاجات مش متسجّلة.

> **الحاجة الوحيدة اللي ممكن تضيع بجد**
>
> التعديلات اللي **عمرها ما اتعملها Commit ولا add**. دي Git مش شايفها أصلاً، فلو `restore` أو `reset --hard` مسحها، مفيش reflog يرجّعها. عشان كده: Commit كتير، حتى لو "wip". تقدر تنضّفهم بعدين بـ `rebase -i`.

19.2 غلطات في الـ Commits

أشهر موقف: عملت 3 Commits لـ Feature جديدة وانت فاكر إنك على `feature/search`، وبصيت لقيت نفسك على `main`. ولسه ماعملتش push. الفكرة إن الـ Branch مجرد لافتة (↩ 7.2 ص 54): نحط لافتة جديدة على الـ Commits دي، ونرجّع لافتة

main مكانها.

مشكلة 1 عملت Commits على main بدل Branch جديدة (مفيش push)

الـ Commits المفروض تكون في Branch لوحدها.

الحل:

```
Terminal
git branch feature/search          # لافتة جديدة على الـ commits دي
git reset --hard origin/main       # رجّع main لمكانه على GitHub
git switch feature/search          # كمّل شغلك هنا
```

الـ Working directory لازم يكون نضيف قبل `reset --hard`. ولو main مالوش origin، بدل `origin/main` استخدم `HEAD~3` (عدد الـ Commits).

مشكلة 2 عملت Commit على Branch غلط (موجودة)

Commit واحد المفروض يروح `feature/nav` بس اتعمل على `feature/search`.

الحل: انقله بـ cherry-pick (↩ 15.7 ص 114) وبعدين شيله من المكان الغلط:

```
Terminal
git log --oneline -1               # خد الـ SHA، مثلاً a1b2c3d
git switch feature/nav
git cherry-pick a1b2c3d
git switch feature/search
git reset --hard HEAD~1
```

مشكلة 3 رسالة الـ Commit غلط، أو نسيت ملف

آخر Commit رسالته `fix` بس، أو نسيت تضيف `style.css`.

الحل: `git commit --amend -m "fix(search): trim city input"`. ولو ملف ناقص:
`git add style.css` ثم `git commit --amend --no-edit` (↩ 6.3 ص 48). ولو Commit أقدم:
`git rebase -i HEAD~3` واختار `reword` أو `fixup`. ولو اتعمله push، هتحتاج
`git push --force-with-lease`، وعلى Branch بتاعتك بس.

مشكلة 4 الـ Commits طالعة باسم أو إيميل غلط

على GitHub الـ Commits مش مربوطة بحسابك، أو طالعة باسم الـ Laptop.

الحل: الأول صلّح الإعداد (↩ 2.3 ص 15) والإيميل لازم يكون مضاف لحسابك على GitHub. وبعدين صلّح آخر Commit:

```
Terminal
git commit --amend --reset-author --no-edit
```

ولآخر 5 Commits مرة واحدة:

```
Terminal
git rebase --exec "git commit --amend --reset-author --no-edit" HEAD~5
```

`--reset-author` = خد الاسم والإيميل من الإعدادات الحالية، و `--exec` = شغّل الأمر ده بعد كل Commit في الـ Rebase.

مشكلة 5 عملت push لـ Commit بايظ على main

الموقع باظ، والـ Commit على GitHub، وزمايلك عملوا pull.

الحل: متعملش reset ولا force push على main. اعمل `git revert <sha>` (↩ 6.5 ص 50): Commit جديد بيلغي التاني، و push عادي. التاريخ بيفضل صادق ومحدش يتأذي.

19.3 "شغلي ضاع!"

جملة بتتقال كتير، ومعظم المرات مش حقيقية. الشغل غالباً مستخبي في حتة من التلاتة دول: Commit مش عليه لافتة، أو Stash، أو Branch تانية.

مشكلة 6 HEAD detached وعملت Commits

`git status` بيقول `HEAD detached at 3f2a1bc` وانت عملت شغل.

الحل: حط لافتة على مكانك قبل ما تتحرك: `git switch -c save-my-work` (↩ 7.6 ص 57). ولو اتحركت خلاص وقفلت الـ Commits: Git طبع لك تحذير فيه الـ SHA، أو دوّر عليه في `git reflog`، وبعدين `git branch save-my-work <sha>`.

مشكلة 7 reset --hard أو مسحت Branch بالغلط

كان فيه Commits وراحت.

الحل: `git reflog` ، دوّر على السطر اللي قبل الغلطة، و `git reset --hard HEAD@{1}` أو `git branch rescue <sha>` (↩ 16.2 ص 116). Branch اتمسحت؟ Git طبع الـ SHA وقت المسح: `Deleted branch x (was 9c1e2d4)` ، فـ `git branch x 9c1e2d4` .

مشكلة 8 عملت stash ومش لاقي التعديلات

الملفات رجعت قديمة والتعديلات مش موجودة.

الحل: `git stash list` ، وبعدين `git stash show -p stash@{0}` تتأكد إنه هو، و `git stash pop` . وبتاخد بالك إنك على نفس الـ Branch اللي عملت عليها الـ Stash (↩ 14.2 ص 105).

19.4 محبوس في نص عملية

الـ Prompt عندك في zsh ممكن يكتب `main|MERGING` أو `REBASE 2/5` ، أو `git status` يقول `You have unmerged paths` . انت في نص عملية، وليك طريقين: تكمّل أو تلغي. والإلغاء دايماً بيرجّعك لحالة ما قبل البداية بالظبط.

انت في نص	تكمّل (بعد حل الـ Conflicts و add)	تلغي
merge	`git commit`	`git merge --abort`
rebase	`git rebase --continue`	`git rebase --abort`
cherry-pick	`git cherry-pick --continue`	`git cherry-pick --abort`
revert	`git revert --continue`	`git revert --abort`
stash pop بـ Conflict	حل، `add` ، ثم `git stash drop`	`git reset --merge` (الـ Stash لسه موجود)
bisect	—	`git bisect reset`

جدول 19.1 طرق الخروج من كل عملية

مشكلة 9 error: Your local changes would be overwritten by checkout

بتحاول `switch` أو `pull` وعندك تعديلات مش متسجّلة هتتلخبط.

الحل: يا Commit، يا `git stash` ثم العملية ثم `git stash pop` . ولو التعديلات دي مش عاوزها: `git restore .` .

19.5 رسائل الـ push والـ pull

الجزء ده بالرسالة اللي هتشوفها، عشان تدوّر عليها بعينك على طول.

مشكلة 10 ! [rejected] main -> main (fetch first)

زميل (أو انت من جهاز تاني أو من موقع GitHub) عمل push قبلك.

الحل: `git pull` (أو `git pull --rebase`)، وحل أي Conflict، وبعدين `git push` (↩ 10.7 ص 80). متعملش `--force` هنا: هيمسح شغل زميلك.

مشكلة 11 fatal: Need to specify how to reconcile divergent branches

الـ pull رفض يشتغل لأن فيه Commits عندك وعلى GitHub الاتنين.

الحل: Git عاوزك تختار مرة واحدة: `git config --global pull.rebase false` (merge، اللي ظبطناه في ↩ 2.6 ص 18) أو `true` (rebase). أو للمرة دي بس: `git pull --no-rebase` أو `git pull --rebase`.

مشكلة 12 fatal: refusing to merge unrelated histories

عملت Repo على GitHub بـ README، وعملت `git init` عندك كمان، وحاولت تعمل pull. الاتنين ملهمش أصل مشترك.

الحل: `git pull origin main --allow-unrelated-histories`، حل أي Conflict في README، و push. والمرة الجاية: يا تعمل Repo فاضي على GitHub (من غير README)، يا تبدأ بـ `clone`.

مشكلة 13 error: src refspec main does not match any

بتعمل `git push -u origin main` والـ Branch "مش موجودة".

الحل: يا إما مفيش أي Commit لسه (اعمل واحد)، يا إما الـ Branch اسمها `master`: شوف بـ `git branch`، وغيّر الاسم بـ `git branch -m main` (↩ 2.4 ص 17).

مشكلة 14 git@github.com: Permission denied (publickey)

الـ SSH مش شغال.

الحل: اختبر: `ssh -T git@github.com`. لو فشل: اتأكد إن المفتاح موجود (`ls ~/.ssh`)، وإنه في الـ Agent (`ssh-add --apple-use-keychain ~/.ssh/id_ed25519`)، وإن الـ Public key متضاف في GitHub ← Settings ← SSH keys (↩ 9.4 ص 70). وتأكد إن الـ Remote بـ SSH: `git remote -v`.

مشكلة 15 **remote: Support for password authentication was removed**

كتبت باسوورد GitHub في الـ Terminal.

الحل: من 2021 مينفعش باسوورد. استخدم `gh auth login` أو Token (↩ 9.5 ص 73). ولو الـ Keychain حافظ حاجة قديمة غلط، امسحها من Keychain Access (ابحث عن github.com) وجرّب تاني.

مشكلة 16 **Updates were rejected because the tip of your current branch is behind (بعد rebase)**

عملت rebase لـ Branch كان ليها push قبل كده.

الحل: لو الـ Branch بتاعتك لوحدك: `git push --force-with-lease` (↩ 15.3 ص 112). لو حد تاني شغال عليها: اتكلم معاه الأول.

19.6 مشاكل الملفات

مشكلة 17 **this exceeds GitHub's file size limit of 100.00 MB ;File is 120.00 MB**

GitHub بيرفض أي ملف أكبر من 100 MB (وبيحذّر من 50 MB). غالباً فيديو أو ملف Build أو Database.

الحل: المشكلة إن الملف في Commit، فمسحه في Commit جديد مش كفاية (الـ push بيبعت التاريخ كله). لو في آخر Commit: `git rm --cached demo.mp4` ثم `git commit --amend --no-edit`. لو أقدم: `git reset --soft origin/main` (يرجّع كل الـ Commits اللي ماتبعتتش لـ Staging)، وشيل الملف بـ `git rm --cached`، وضيفه لـ gitignore.، واعمل Commit من جديد.

لو حابب تتعمق: Git LFS

لو محتاج فعلاً ملفات كبيرة في الـ Repo (فيديوهات، تصميمات)، فيه **Git LFS** (Large File Storage): `brew install git-lfs` ثم `git lfs install` و `git lfs track "*.mp4"`. الـ Repo بيحفظ ملف صغير فيه "مؤشر"، والملف الحقيقي بيترفع على سيرفر GitHub .LFS بيدي مساحة LFS محدودة مجاناً.

مشكلة 18 **.DS_Store في كل حتة**

macOS بيعمل ملف `.DS_Store` في كل فولدر بتفتحه في Finder، وظهر في `git status` أو اتعمله Commit.

الحل: مرة واحدة على جهازك: ضيفه لـ Global ignore (↩ 5.4 ص 42). ولو اتعمله Commit: `git rm --cached -r --ignore-unmatch '*.DS_Store'` ثم Commit.

مشكلة 19 غيرت Logo.png لـ logo.png و Git مش شايف

نظام ملفات الـ Mac مبيفرقش بين الحروف الكبيرة والصغيرة، بس GitHub Pages و Linux بيفرقوا، فالصورة تبوظ على الموقع.

الحل: غيّر الاسم من خلال Git: `git mv Logo.png logo.png` ثم Commit (↩ 5.7 ص 44).

مشكلة 20 git status بيعرض كل ملفات الـ Mac!

كتبت `git status` في فولدر عادي، وطلع لك `Desktop/` و `Documents/` و `Library/` كلهم Untracked. معناها إنك عملت `git init` مرة في الـ Home بالغلط، والـ Home كله بقى Repo.

الحل: اتأكد الأول: `git rev-parse --show-toplevel` . لو طبع `/Users/sara` (الـ Home نفسه) يبقى هو ده. امسح الـ `.git` اللي هناك بس: `rm -rf ~/.git` . اقرا الأمر ده مرتين قبل Enter، وتأكد إنه `~/.git` بالظبط مش حاجة تانية. ده بيمسح تتبع Git بس، ملفاتك سليمة.

مشكلة 21 fatal: not a git repository (or any of the parent directories): .git

بتشغّل أمر Git في فولدر مش Repo.

الحل: `pwd` ، وادخل فولدر المشروع بـ `cd` . ولو ده المشروع فعلاً ومحتاج Repo: `git init` .

ملخص الفصل

- متتخضش: `status` ، ونسخة احتياطي، و `log --graph` و `reflog` ، وبعدين الحل.
- اللي اتعمله Commit مبيضيعش. اللي عمره ما اتسجّل هو اللي يضيع.
- Branch غلط = لافتة جديدة + `reset` ، أو `cherry-pick` .
- في نص عملية: `--continue` أو `--abort` .
- على main المشتركة: `revert` مش reset و force.

صح ولا غلط؟

1. تعديلات عمرها ما اتعملها add ممكن ترجّعها بـ reflog. صح / غلط
2. `git merge --abort` يرجّعك لحالة ما قبل الـ Merge. صح / غلط
3. لو ملف كبير اتعمله Commit، كفاية تمسحه في Commit جديد عشان الـ push ينجح. صح / غلط

؟ أسئلة مراجعة

1. عملت 2 Commits على main بدل Branch ومفيش push. اكتب الأوامر بالترتيب.
2. إيه الفرق بين إصلاح Commit بايظ على main اتعمله push، وواحد لسه محلي؟
3. ليه `refusing to merge unrelated histories` بتحصل، وإزاي تتجنبها؟
4. الـ Branch اتمسحت بـ `-D`. إزاي ترجّعها؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في Repo تجريبي، اعمل Commit على main، وانقله لـ Branch جديدة بالطريقة اللي في الفصل.
2. متوسط اعمل Merge فيه Conflict، واخرج منه بـ `--abort`، وبعدين اعمله تاني وحله.
3. صعب امسح Branch فيها Commits بـ `-D`، وارجّعها من غير ما تبص على الرسالة اللي Git طبعها (استخدم reflog بس).

ملحق أ

Cheat Sheet وجدول الـ Flags وجدول الرموز

ورقة مراجعة لكل أوامر Git و gh المهمة في الكتاب مقسّمة حسب الموضوع، وبعدها جدول الـ Flags ومعانيها في كل أمر، وجدول لكل رمز ممكن تشوفه في Git: HEAD~ و .. و علامات الـ Conflict وأنماط الـ gitignore..

Setup — التظبيط

`brew install git gh`	تسطيب على الـ Mac
`git config --global user.name "Sara"`	اسمك
`git config --global user.email me@x.com`	إيميلك
`git config --global init.defaultBranch main`	main افتراضي
`git config --global core.editor "code --wait"`	الـ Editor
`git config --list --show-origin`	كل الإعدادات ومنين
`git help cmd / git cmd -h`	المساعدة

Start — البداية

`git init`	Repo جديد هنا
`git clone URL [dir]`	نسخة من Remote
`git status / -sb`	الحالة / مختصرة
`git add f / . / -A / -p`	جهّز ملف / الكل / حتة حتة
`git commit -m "msg"`	سجّل
`git commit -am "msg"`	add للمتتبَّع + commit

History — التاريخ

`git log --oneline --graph --all`	الشجرة كلها
`git log -p -- f`	تعديلات ملف
`git log -S "text"`	مين ضاف الكلمة
`git diff`	الورشة مقابل Staging
`git diff --staged`	Staging مقابل آخر Commit
`git diff --stat`	ملخص الملفات
`git show sha / sha:f`	Commit / ملف في Commit

Ignore — التجاهل

`echo node_modules/ >> .gitignore`	تجاهل فولدر
`git check-ignore -v f`	ليه متجاهَل
`git rm --cached f`	بطّل تتابعه وسيبه
`git mv old new`	غيّر اسم

Undo — التراجع

`git restore f`	ارمي تعديلات الملف
`git restore --staged f`	شيله من الـ Staging

`git commit --amend [--no-edit]`	صلّح آخر Commit
`git reset --soft HEAD~1`	الغي الـ Commit وسيب كله staged
`git reset HEAD~1`	الغي وسيب في الورشة
`git reset --hard HEAD~1`	الغي وامسح (خطر)
`git revert sha`	Commit عكسي (آمن بعد push)
`git clean -nd / -fd`	جرّب / امسح Untracked

Branches — الفروع

`git branch [-a] [-vv]`	اعرض
`git switch -c name`	اعمل وروح
`git switch name / -`	روح / اللي قبله
`git branch -d / -D name`	امسح / بالقوة
`git branch -m new`	غيّر الاسم
`git merge name`	ادمج في الحالي
`git merge --abort`	الغي Merge

Remotes — الريموت

`git remote -v`	اعرض
`git remote add origin URL`	اربط
`git remote set-url origin URL`	غيّر الرابط
`git push -u origin main`	أول push
`git push / git pull`	ابعت / هات وادمج
`git fetch --prune`	هات المعلومات ونضّف
`git push --force-with-lease`	بعد rebase (Branch بتاعتك)

`ssh -T git@github.com`	اختبر SSH

Stash & Tags — الرف والتاجات

`git stash push -u -m "msg"`	خبّي (مع الجديد)
`git stash list / show -p`	اعرض
`git stash pop / apply`	رجّع
`git tag -a v1.0.0 -m "msg"`	Annotated tag
`git push origin v1.0.0 / --tags`	ابعت Tag

Advanced — المتقدم

`git rebase main`	انقل الـ Branch فوق main
`git rebase -i HEAD~3`	نضّف آخر 3
`git rebase --continue / --abort`	كمّل / الغي
`git cherry-pick -x sha`	انسخ Commit
`git reflog`	كل حركات HEAD
`git branch rescue HEAD@{2}`	أنقذ من الـ reflog
`git bisect start / good / bad / reset`	دوّر على الـ Bug
`git blame -w f`	مين كتب كل سطر

gh — GitHub CLI

`gh auth login`	ادخل
`gh repo create name --public --source=. --push`	Repo من هنا
`gh pr create --fill`	افتح PR
`gh pr checkout 12`	جرّب PR
`gh pr merge --squash --delete-branch`	ادمج

`gh issue create / list`	Issues
`gh run watch`	تابع Actions

AI safety	مع الـ AI
`git status`	لازم clean قبل الـ AI

`git switch -c ai/try`	تجربة في Branch
`git diff --stat`	إيه اللي اتغير
`git add -p`	اقبل جزء
`git restore . && git clean -fd`	ارمي الباقي

1.أ جدول الـ Flags المشتركة

نفس الحرف ممكن يبقى ليه معنى مختلف من أمر للتاني. الجدول ده بيجمع أشهر الحروف ومعانيها حسب الأمر. ولما تشك: `git cmd -h` أو `git help cmd`.

الـ Flag	في الأوامر	المعنى
`-m`	stash push / tag / commit	Message: الرسالة
`-m`	branch	Move: غيّر الاسم
`-a`	commit	All: كل المتتبَّع المتعدّل
`-a`	branch	All: حتى الـ Remote
`-a`	tag	Annotated
`-A`	add	All: كل التغييرات في الـ Repo
`-u`	add	Update: المتتبَّع بس
`-u`	push	Set upstream: اربط الـ Branch
`-u`	stash	Untracked: الجديد كمان
`-p`	stash / restore / add	Patch: حتة حتة
`-p`	show / log	Patch: اعرض الـ diff
`-d / -D`	branch	Delete / بالقوة
`-d`	clean	Directories كمان
`-c / -C`	switch	Create / بالقوة
`-f`	clean	Force: لازم عشان يمسح
`-f / --force`	push	اكتب فوق الـ Remote (خطر)
`--force-with-lease`	push	Force بس لو محدش ضاف حاجة

الـ Flag	في الأوامر	المعنى
`-n`	add / clean	Dry run: جرّب
`-n`	log	عدد الـ Commits (`-n 5` = `-5`)
`-v`	branch / remote	Verbose: تفاصيل
`-vv`	branch	ومعاه الـ upstream و ahead/behind
`-s`	status	Short
`-b`	status	Branch في الـ Short
`-x`	cherry-pick	اكتب أصل الـ Commit في الرسالة
`-i`	rebase	Interactive
`-w`	blame / diff	تجاهل المسافات
`-S`	log	دوّر على نص اتضاف أو اتشال
`--staged / --cached`	rm / restore / diff	الـ Staging (الاسمين واحد في diff)
`--soft / --mixed / --hard`	reset	حرّك الـ Branch بس / + Staging / + الورشة
`--oneline`	log	سطر لكل Commit
`--graph / --all`	log	رسم / كل الـ Branches
`--stat`	show / diff / log	ملخص الملفات
`--amend`	commit	صلّح آخر Commit
`--no-edit`	merge / commit	من غير ما تفتح الـ Editor
`--abort / --continue`	cherry-pick / rebase / merge	الغي / كمّل
`--global / --local / --system`	config	مستوى الإعداد
`--prune`	fetch	امسح /origin* اللي اتمسحت
`--rebase`	pull	rebase بدل merge
`--squash`	merge	جمّع في Staging من غير Merge commit
`--no-ff`	merge	اعمل Merge commit حتى لو ينفع FF
`--ours / --theirs`	restore / checkout	في Conflict: نسختي / نسختهم
`--fill`	gh pr create	العنوان والوصف من الـ Commits
`--web`	gh	افتح في المتصفح

جدول أ.1 الـ Flags ومعانيها حسب الأمر

أ.2 جدول الرموز

الرمز	المعنى	مثال
`HEAD`	انت فين دلوقتي	`git show HEAD`
`HEAD~ / HEAD~2`	الأب / الجد (أول Parent)	`git reset HEAD~2`
`HEAD^ / HEAD^2`	الأب / الأب التاني في Merge commit	`git show HEAD^2`
`@`	اختصار لـ HEAD	`git log @~3..@`
`@{n} / HEAD@{2}`	مكان في الـ reflog	`git reset --hard HEAD@{1}`
`@{u}`	الـ upstream بتاع الـ Branch	`git log @{u}..`
`ORIG_HEAD`	قبل آخر reset أو merge	`git reset --hard ORIG_HEAD`
`a..b`	اللي في b ومش في a	`git log main..feature`
`a...b`	اللي في واحد منهم بس (diff: من الـ base)	`git diff main...feature`
`--`	بعدها أسماء ملفات	`git log -- app.js`
`sha:path`	ملف في Commit	`git show HEAD~1:app.js`
`origin/main`	صورة main على الـ Remote	`git diff origin/main`
`sha (7 حروف)`	أول حروف الـ Hash كفاية	`git show a1b2c3d`
`<<<<<<< / ======= / >>>>>>>`	علامات Conflict: نسختي / فاصل / نسختهم	`<<<<<<< HEAD`
`@@ -a,b +c,d @@`	مكان الـ Hunk: قديم / جديد	`@@ -10,4 +10,6 @@`
`+ / -`	سطر اتضاف / اتشال في diff	`+const x = 1;`
`M A D R ?? !!`	status -s: متعدّل / جديد staged / ممسوح / اتغير اسمه / Untracked / متجاهَل	`?? new.js`
`MM`	staged ومتعدّل تاني	`MM app.js`
`*`	في branch: الحالي. في gitignore.: أي حروف	`*.log`
`**`	في gitignore.: أي عمق	`**/temp`
`!`	في gitignore.: استثناء	`!.env.example`
`/`	في gitignore.: أول = الجذر، آخر = فولدر	`/dist build/`
`#`	تعليق في gitignore. وفي الـ Editor	`# deps`

الرمز	المعنى	مثال
`.`	الفولدر الحالي	`git add .`
`~`	الـ Home	`~/.gitconfig`
`#12`	في GitHub: Issue أو PR رقم 12	`Closes #12`
`@user`	في GitHub: منشن	`@sara review`
`type(scope)!:`	Conventional Commits، و ! = تغيير بيكسر	`feat(api)!: drop v1`
`v1.2.3`	Tag بـ SemVer	`git tag -a v1.2.3`
`${{ }}`	تعبير في GitHub Actions	`${{ secrets.KEY }}`
`<x>` / `[x]`	في الشرح: قيمتك / اختياري	`git branch [-d] <name>`

جدول أ.2 كل رمز في Git ومعناه

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعله كل ما تقابل كلمة ومش فاكر معناها.

A

Actions

خدمة GitHub لتشغيل Workflows أوتوماتيك (اختبارات، Build، نشر) مع أحداث زي push و PR.

Ahead / Behind

عدد الـ Commits اللي عندك ومش على الـ Remote / اللي على الـ Remote ومش عندك.

Alias

اسم مختصر لأمر Git: `git config --global alias.st status`.

Amend

تعديل آخر Commit (رسالته أو محتواه) بـ `--amend`، وبيعمل Commit جديد بـ Hash جديد.

Annotated tag

Tag كامل فيه اسم وتاريخ ورسالة: `git tag -a`.

Atomic commit

Commit بيعمل حاجة واحدة بس، فيسهل فهمه وإلغاؤه.

B

Bisect

بحث ثنائي في التاريخ عن أول Commit بايظ: `git bisect`.

Blame

مين آخر حد عدّل كل سطر وفي أنهي Commit: `git blame`.

Blob

Object بيخزن محتوى ملف واحد (من غير اسمه).

Branch

لافتة متحركة بتشاور على Commit، وبتتحرك مع كل Commit جديد.

Branch protection / Ruleset

قواعد على GitHub بتحمي Branch: ممنوع push مباشر، لازم PR ومراجعة.

C

Cherry-pick

نسخ Commit واحد من مكان لمكان: `git cherry-pick <sha>`.

CI

Continuous Integration: تشغيل الاختبارات أوتوماتيك مع كل تغيير.

Clone

نسخة كاملة من Repo بتاريخه على جهازك.

Commit

Snapshot متسجّل للمشروع، فيه Author ووقت ورسالة و Parent.

Conflict

Git مش عارف يدمج لوحده لأن نفس السطر اتغير بشكلين، فبيسيبلك تختار.

Conventional Commits

صيغة رسايل: `type(scope): description` زي `feat:` و `fix:`.

D

Detached HEAD

HEAD بيشاور على Commit مباشرة مش على Branch.

Diff

الفرق بين نسختين، سطر سطر بـ `+` و `-`.

Distributed VCS

نظام كل نسخة فيه معاها التاريخ كامل (زي Git).

F

Fast-forward

Merge من غير Commit جديد: اللافتة بتتحرك لقدام بس.

Fetch

تنزيل الجديد من الـ Remote وتحديث `origin/*` من غير ما يلمس شغلك.

Force push

push بيكتب فوق تاريخ الـ Remote. الآمن: `--force-with-lease`.

Fork

نسخة من Repo حد تاني على حسابك في GitHub.

G

gh

أداة GitHub الرسمية للـ Terminal: Repos و PRs و Issues و Releases.

GitHub Flow

Workflow بسيط: main دايماً شغال، وكل شغل في Branch و PR.

GitHub Pages

استضافة مجانية لمواقع Static من Repo.

.gitignore

ملف فيه أنماط الملفات اللي Git يتجاهلها.

H

Hash / SHA

بصمة الـ Object المحسوبة من محتواه (40 حرف Hex في SHA-1).

HEAD

مؤشر بيقول انت فين دلوقتي: غالباً على Branch.

Hunk

جزء متصل من التغييرات في diff، بيبدأ بـ `@@`.

I

Index / Staging area

المكان اللي بتجهز فيه الـ Commit الجاي بـ `git add`.

Interactive rebase

`rebase -i`: تعدّل وتجمع وترتب وتمسح Commits.

Issue

تذكرة على GitHub لـ Bug أو فكرة أو مهمة.

K

Keychain

مخزن كلمات السر في macOS. Git و SSH بيحفظوا فيه بيانات الدخول.

L

Lightweight tag

Tag مجرد اسم على Commit من غير معلومات إضافية.

M

Markdown

طريقة تنسيق نص برموز بسيطة، بيستخدمها README و GitHub.

Merge

دمج تاريخ Branch في Branch تاني.

Merge base

آخر Commit مشترك بين Branchين، Git بيقارن بيه في الـ 3-way merge.

Merge commit

Commit ليه Parentين بيجمع خطين.

O

Object

وحدة التخزين في `.git/objects`: blob أو tree أو commit أو tag.

ORIG_HEAD

مكان HEAD قبل آخر عملية خطيرة زي reset أو merge.

origin

الاسم الافتراضي للـ Remote اللي عملت منه clone.

origin/main

Remote-tracking branch: صورة main على GitHub آخر مرة عملت fetch.

P

Personal access token

بديل الباسوورد للـ HTTPS، بصلاحيات وتاريخ انتهاء.

Pull

`fetch` + `merge` (أو rebase).

Pull Request (PR)

طلب دمج Branch في Branch، مع مراجعة ونقاش.

Push

إرسال الـ Commits بتاعتك للـ Remote.

Push protection

ميزة GitHub بترفض push فيه مفاتيح سرية معروفة.

R

Rebase

إعادة تشغيل Commits فوق Base جديد، بـ Hashes جديدة.

Ref
اسم بيشاور على Commit: Branch أو Tag أو HEAD.

Reflog
سجل محلي بكل مكان HEAD والـ Branches كانوا فيه.

Release
صفحة على GitHub مربوطة بـ Tag فيها ملاحظات وملفات.

Remote
نسخة من الـ Repo على سيرفر تاني (زي GitHub).

Repository (Repo)
المشروع + فولدر `.git` اللي فيه التاريخ كله.

Reset
تحريك الـ Branch لـ Commit تاني، بـ `--soft` أو `--mixed` أو `--hard`.

Restore
رجّع ملف لنسخة من الـ Staging أو من Commit.

Revert
Commit جديد بيلغي تأثير Commit قديم من غير ما يمسحه.

S

Secret scanning
فحص GitHub للـ Repos عن مفاتيح وكلمات سر متسرّبة.

Semantic Versioning
ترقيم `MAJOR.MINOR.PATCH`: كسر / ميزة / إصلاح.

Snapshot
صورة كاملة لكل الملفات لحظة الـ Commit.

Squash
جمع كذا Commit في Commit واحد.

SSH key
زوج مفاتيح (خاص وعام) للدخول على GitHub من غير باسوورد.

Stash
رف مؤقت بتحط عليه تعديلات مش جاهزة: `git stash`.

T

Tag
اسم ثابت على Commit معين، غالباً لإصدار زي `v1.0.0`.

Tracked / Untracked
ملف Git بيتابعه / ملف جديد Git شايفه بس مش بيتابعه.

Tree
Object بيمثل فولدر: أسماء الملفات والـ blobs والـ trees اللي جواه.

U

Upstream
الـ Branch على الـ Remote اللي الـ Branch المحلي متابعه، أو المشروع الأصلي في حالة الـ Fork.

W

Workflow (Actions)
ملف YAML في `.github/workflows` بيحدد إمتى وإيه اللي يتشغّل.

Working directory
الملفات اللي قدامك وبتعدّلها.

Worktree
فولدر تاني لنفس الـ Repo على Branch تانية: `git worktree add`.

ملحق ج

الإجابات والحلول

جاوب بنفسك الأول، وبعدين قارن. لكل فصل: إجابات أسئلة المراجعة، وبعدها صح ولا غلط، وبعدها حلول التحديات. أغلب التحديات تجارب عملية، فلو وصلت لنفس النتيجة بطريقة تانية فهي صح.

الفصل 1: ليه Git أصلاً؟

1. مثلاً: مساحة كبيرة متكررة، ومش عارف إيه اللي اتغير بين نسختين، وأسماء زي `final-final2` بتلخبط، ومفيش طريقة تدمج شغل اتنين.
2. كل واحد عنده التاريخ كامل على جهازه: يشتغل من غير نت، ولو السيرفر وقع مفيش حاجة تضيع.
3. Snapshot للملفات (tree)، والـ Author والوقت، والرسالة، والـ Parent (اللي قبله)، والـ Hash (SHA) بتاعه.
4. لأن الـ Hash محسوب من المحتوى ومن الـ Parent، فأي تغيير بيدّي Hash مختلف ويغيّر كل اللي بعده. ده بيضمن إن التاريخ متلعبش فيه.

صح ولا غلط: (1) غلط: الـ Commit محلي خالص. · (2) غلط: بيخزن Snapshot كامل (والملفات اللي متغيرتش بيشاور عليها). · (3) غلط: Git الأداة، و GitHub موقع بيستضيف Repos.

التحديات:

1. تجربة: على الـ Mac غالباً هيطلع رقم من Apple Git أو Homebrew.
2.
```
find ~ -iname "*final*" -o -iname "*copy*" 2>/dev/null | head
```
3. إجابة شخصية: آلاف الـ Commits من مئات الناس في نفس الوقت محتاجة تاريخ ودمج أوتوماتيك.

الفصل 2: تسطيب Git وتظبيطه

1. الإيميل الشخصي بـ `--global`، وجوه Repo الشغل: `git config user.email work@company.com` (يعني `--local`، وبيغلب).
2. عشان يطابق GitHub (من 2020) ويطابق اللي Git 3.0 هيعمله افتراضي، فمتلاقيش master عندك و main على GitHub.
3. `-h` ملخص قصير في الـ Terminal، و `--help` الـ Manual كامل.
4.
```
git config --show-origin --show-scope --get-all user.email
```
(أو `--list --show-origin`) بيوريك القيمة جاية من أنهي ملف.

صح ولا غلط: (1) صح. · (2) غلط: Git مش هيستنى وهيعتبر الرسالة فاضية. · (3) صح: عشان كده فيه noreply email.

التحديات:

1. `git config --global user.name "…"` و `user.email` و `init.defaultBranch main`، ثم `git config --global --list`.
2.
```
git config --global core.editor "code --wait"
```
ثم `git config --global --edit`.
3. `git config --global alias.st "status -sb"` و
```
git config --global alias.last "log -1 HEAD --stat"
```
.

الفصل 3: الأماكن التلاتة والـ Commit

1. الـ Working directory: الملفات اللي بتعدلها في VS Code. الـ Staging: اللي اخترته بـ `add` للـ Commit الجاي. الـ Repo (`.git`): الـ Commits المتسجلة.
2. عشان تختار: عدّلت 3 حاجات لأغراض مختلفة، فتعمل 3 Commits مركّزة بدل واحد ملخبط.
3. الحرف الأول = حالة الـ Staging، والتاني = الورشة. MM = الملف اتعدّل واتعمله add، وبعدين اتعدّل تاني.
4. `.` كل التغييرات في الفولدر الحالي وتحته، و `-A` كل التغييرات في الـ Repo كله، و `-u` الملفات المتتبَّعة بس (مش الجديدة).

صح ولا غلط: (1) غلط: المتتبَّعة بس. · (2) غلط: النسخة اللي وقت الـ add بس. · (3) غلط: بيمسح التاريخ، والملفات بتفضل.

التحديات:

1. `git init git-playground`، وبعدين لكل ملف: `git add f` و `git commit -m "…"`.
2. النسخة اللي اتعملها add دخلت، والتعديل التاني فاضل `M` في الورشة.
3. `cat-file -p HEAD` بيطبع `tree <sha>` والـ author و committer والوقت، وبعد سطر فاضي الرسالة.

الفصل 4: قراية التاريخ

1. `git log --oneline -5 -- index.html`
2. `git diff --staged`
3. في النسخة القديمة الجزء ده يبدأ من السطر 10 وطوله 4 سطور، وفي الجديدة من 10 وطوله 6.
4. بتفصل الـ Options والـ Revisions عن أسماء الملفات، فـ Git ميتلخبطش لو اسم ملف شبه اسم Branch.

صح ولا غلط: (1) صح. · (2) صح. · (3) غلط: بيعرضه بس.

التحديات:

1. تجربة.
2. `diff` = الورشة مقابل الـ Staging، و `--staged` = الـ Staging مقابل آخر Commit، و `HEAD` = الورشة مقابل آخر Commit (الاتنين مع بعض).
3. `git log -S "word" --oneline` ثم `git show <sha>`.

الفصل 5: الـ .gitignore

1. `dist/` أي فولدر بالاسم ده في أي مكان، و `/dist` اللي في الجذر بس (ملف أو فولدر)، و `dist` أي ملف أو فولدر بالاسم ده في أي مكان.
2. `.env` فيه أسرار حقيقية، و `.env.example` بيقول للفريق محتاجين أنهي متغيرات من غير قيم.
3. لما الفولدر نفسه متجاهَل، Git مبيدخلوش أصلاً فمبيشوفش الاستثناء. الصح: `logs/*` ثم `!logs/keep.log`.
4. ضيف `node_modules/` للـ gitignore، و `git rm -r --cached node_modules`، و Commit.

صح ولا غلط: (1) غلط: لازم `git rm --cached`. · (2) غلط: بيتعمله Commit عشان الكل ينزّل نفس الإصدارات. · (3) غلط: بيشيله من Git بس.

التحديات:

1. تجربة.
2. `git check-ignore -v .env` يطبع القاعدة، و `git check-ignore -v .env.example` مبيطبعش حاجة.
3. `git rm --cached debug.log` ثم Commit، و `git ls-files` مبقاش فيه الملف.

الفصل 6: التراجع

1. `git add file` ثم `git commit --amend --no-edit`.
2. التلاتة بيحركوا الـ Branch. `--soft` بيسيب الـ Staging والورشة، و `--mixed` بيفضّي الـ Staging ويسيب الورشة، و `--hard` برجّع التلاتة (ويمسح التعديلات).
3. `git revert <sha>` ثم push: Commit جديد بيلغيه. الـ reset بيغيّر تاريخ عند زميلك وهيعمل مشاكل.
4. `restore` برجّع ملف متتبَّع لحالته، و `clean -f` بيمسح ملفات Untracked.

صح ولا غلط: (1) غلط: بيشيله من الـ Staging بس. · (2) غلط: بيضيف Commit جديد. · (3) صح.

التحديات:

1. تجربة: `git restore f`، ثم `git add f` و `git restore --staged f` و `git diff` يبيّن التعديل.
2. `git reset --soft HEAD~3` ثم `git commit -m "…"`.
3. `ORIG_HEAD` بيتحفظ قبل الـ reset، فـ `git reset --hard ORIG_HEAD` برجّعه.

الفصل 7: الـ Branches و HEAD

1. feature بيتحرك للـ Commit الجديد، و main فاضل مكانه، و HEAD بيشاور على feature وبيتحرك معاه.
2. `branch x` بيعمل اللافتة بس، و `switch -c x` بيعملها ويتنقل عليها.
3. HEAD بيشاور على Commit مباشرة مش على Branch. اعمل `git switch -c new-branch` قبل ما تمشي.
4. عندك تعديلات هتتمسح بنسخة الـ Branch التاني. الحل: Commit، أو `git stash`.

صح ولا غلط: (1) غلط: لافتة (ملف صغير فيه SHA). · (2) غلط: اللي عليه HEAD بس. · (3) صح: محتاج `-D`.

التحديات:

1. التعديل بيختفي على main ويرجع على feature.
2. هتشوف خطين بيتفرعوا من Commit مشترك.
3. `git reflog`، دوّر على الـ Commit، ثم `git branch rescue <sha>`.

الفصل 8: الـ Merge والـ Conflicts

1. Fast-forward لما main معملش Commits من ساعة ما اتفرعت (الخط مستقيم). 3-way لما الاتنين عملوا Commits.
2. آخر Commit مشترك بين الاتنين. Git بيقارن كل Branch بيه عشان يعرف مين غيّر إيه.
3. `<<<<<<< HEAD` بداية نسختك، و `=======` الفاصل، و `>>>>>>> feature` نهاية النسخة الجاية.
4. `git status`، افتح الملف واختار، امسح العلامات، `git add`، `git commit`.

صح ولا غلط: (1) غلط: بيغيّر main. · (2) غلط: بيحرّك اللافتة بس. · (3) صح.

التحديات:

1. الرسالة فيها `Fast-forward`.
2. تجربة.
3. `git merge --abort`، وبعدين `git checkout --theirs index.html` و add و commit.

الفصل 9: GitHub والـ Remotes

1. HTTPS: `https://github.com/u/r.git` بـ Token أو Keychain. SSH: `git@github.com:u/r.git` بمفتاح.
2. لو حد خد الملف الخاص، ميقدرش يستخدمه من غير الـ Passphrase. والـ Keychain بيحفظها فمش هتكتبها كل مرة.
3. `git remote set-url origin git@github.com:u/r.git`
4. الـ Fine-grained ليه Repos محددة وصلاحيات محددة وتاريخ انتهاء، و Classic صلاحيات عامة على كل حاجة.

صح ولا غلط: (1) غلط: اسم افتراضي. · (2) غلط: الـ `.pub`. · (3) غلط: ده نجاح.

التحديات:

1. تجربة.
2. `ssh-keygen -t ed25519 -C "you@mail.com"`، و `pbcopy < ~/.ssh/id_ed25519.pub`، ثم `ssh -T git@github.com`.
3. `git remote add origin https://…`، ثم `git remote set-url origin git@…` و `git remote -v`.

الفصل 10: clone و push و fetch و pull

1. بيعمل الفولدر، وينزّل كل التاريخ، ويضيف origin، ويعمل Branch محلي متابع للـ Default.
2. `--set-upstream`: بيربط main المحلي بـ origin/main، فبعد كده `git push` و `git pull` من غير أسماء.
3. الأولى = اللي على GitHub ومش عندك (behind)، والتانية = اللي عندك ومش على GitHub (ahead).
4. فيه Commits على GitHub مش عندك. `git pull` وبعدين `git push`.

صح ولا غلط: (1) غلط: بيحدّث origin/* بس. · (2) غلط: clone بيضيفه. · (3) غلط: مع fetch أو pull بس.

التحديات:

1. تجربة.
2. `status` هيقول behind 1، و `log main..origin/main` هيعرض الـ Commit.
3. الـ push التاني يترفض، فـ `git pull` وحل الـ Conflict و push، وبعدين pull في الأولى.

الفصل 11: التعاون

1. `switch main`، `pull`، `switch -c feat/x`، وشغل و Commits، `push -u origin feat/x`، افتح PR، مراجعة وتعديلات، Merge، و `switch main` و `pull` و `branch -d feat/x`.
2. `origin` = الـ Fork بتاعك (بتعمل push عليه)، و `upstream` = المشروع الأصلي (بتعمل منه fetch).
3. لما الـ Branch فيه Commits كتير صغيرة و wip ومش عاوزها كلها في main.
4. تكتب في وصف الـ PR `Closes #12` (أو Fixes أو Resolves).

صح ولا غلط: (1) غلط: تعمل push على نفس الـ Branch والـ PR يتحدث. · (2) صح. · (3) غلط: بيجمعهم في Commit واحد.

التحديات:

1. تجربة.
2. بعد Squash هتلاقي Commit واحد جديد على main.
3. `git remote add upstream …` ، و `git fetch upstream` ، و `git merge upstream/main` ، و `git push` .

الفصل 12: GitHub Flow

1. main دايماً شغال، وكل شغل في Branch باسم واضح، و Commits و push بانتظام، و PR للمراجعة، و Merge بعد الموافقة والـ Checks، والـ Deploy من main.
2. عشان تبدأ من آخر نسخة وتقلل الـ Conflicts بعد كده.
3. `git switch -c fix/x` (الـ Commits بتنتقل معاك)، و `git push -u origin fix/x` ، وافتح PR. وصلّح main المحلي: `git switch main` و `git reset --hard origin/main` .
4. GitHub Flow فيه main و Branches قصيرة بس، و Git Flow فيه develop و release و hotfix لمشاريع بإصدارات مجدولة.

صح ولا غلط: (1) غلط. · (2) غلط: بيمسح الـ /origin* اللي اتمسحت هناك من عندك. · (3) صح.

التحديات:

1. تجربة.
2. الـ push يترفض بـ `GH013` ، وتصلّح بالطريقة اللي في السؤال 3.
3. `git switch feature/b` و `git fetch` و `git merge origin/main` .

الفصل 13: رسايل الـ Commits

1. `fix(auth): handle wrong PIN on login` ، و `docs: add README` ، ودي اتنين: `feat(nav): add navbar` و `style: update color palette` .
2. عشان يكمّل الجملة: "لو اتطبق، الـ Commit ده هي... add navbar. وده نفس أسلوب الرسايل اللي Git بيعملها (Merge branch...).
3. `fix` بيصلح سلوك غلط، و `refactor` بيغيّر شكل الكود من غير ما يغيّر السلوك.
4. تلغي تغيير واحد بس من غير ما تشيل معاه حاجات تانية ملهاش علاقة.

صح ولا غلط: (1) غلط: تنسيق الكود (مسافات و Format) مش شكل الموقع. · (2) صح. · (3) غلط.

التحديات:

1. تجربة.
2. `git add -p` : `y` للأول و `n` للتاني، Commit، وبعدين `git add` للباقي و Commit.
3. `git commit` من غير `-m` ، وفي آخر الجسم سطر `Closes #3` .

الفصل 14: الـ Stash والـ Tags

1. `pop` بيطبّق ويمسحه من القايمة، و `apply` بيطبّق ويسيبه.
2. لما الشغل هيقعد أيام: اعمل Branch و Commit wip أحسن.
3. الـ Tag ثابت على Commit واحد، والـ Branch بيتحرك. الـ Annotated فيه اسم وتاريخ ورسالة (Object كامل)، والـ Lightweight مجرد اسم.
4. 1.4.0، و 1.3.3، و 2.0.0.

صح ولا غلط: (1) غلط: محتاج `-u` . · (2) غلط: `git push origin v1.0.0` أو `--tags` . · (3) صح.

التحديات:

1. تجربة.
2. من غير `-u` الملف الجديد بيفضل في الورشة.
3. `git tag -a v0.1.0 -m "…"` و `git push origin v0.1.0` .

الفصل 15: الـ Rebase والـ Cherry-pick

1. الـ Rebase بيعيد Commits الـ Branch فوق آخر main فالخط مستقيم، والـ Merge بيسيب الخطين ويضيف Merge Commit.
2. لأنه بيعمل Commits جديدة بـ Hashes جديدة، والناس اللي عندهم القديمة هيلاقوا تاريخين.
3. الاتنين بيدمجوا مع اللي قبله، بس `squash` بيخليك تكتب الرسالة، و `fixup` بيرمي رسالته.
4. لما محتاج Commit واحد (Fix مثلاً) في Branch تاني. مش عادة لأنه بيعمل نسخة مكررة بـ Hash تاني.

صح ولا غلط: (1) غلط: Hashes جديدة. · (2) صح. · (3) صح.

التحديات:

1. تجربة.
2. `pick` للأول و `fixup` للي بعده، و `reword` للرسايل.
3. حل وبعدين `git add` و `git rebase --continue`، ثم `git cherry-pick -x <sha>`.

الفصل 16: الـ Reflog و Bisect و Blame

1. `log` = تاريخ الـ Commits المتشاركة، و `reflog` = مذكرات جهازك: كل مكان HEAD راحله.
2. `git reflog`، دوّر على آخر Commit كان عليه، و `git branch name <sha>`.
3. حوالي 10، لأن كل تجربة بتقسم الباقي نصين ($2^{10} = 1024$).
4. `-w` بيتجاهل تغييرات المسافات، فيظهر اللي كتب السطر فعلاً مش اللي عمل Format.

صح ولا غلط: (1) غلط: محلي بس. · (2) غلط: بيفضلوا موجودين لحد ما gc يمسحهم بعد المدة. · (3) صح.

التحديات:

1. `git reset --hard HEAD@{1}`
2. `git reflog` ودوّر على `checkout: moving from x`.
3. `git bisect start HEAD <first>` ثم `git bisect run sh -c '! grep -q BUG file'`.

الفصل 17: README و Pages و Actions و gh

1. مثال: `# Title`، و `![shot](img.png)`، و `[Demo](url)`، و `- item`، وبلوك بـ 3 Backticks.
2. لأن الموقع على `/repo/` مش `/`، فالملفات اللي Vite بيعملها لازم مساراتها تبدأ بيه.
3. `uses` بيشغّل Action جاهز، و `run` بيشغّل أمر Shell.
4. في Settings ← Secrets and variables ← Actions، واستخدمه بـ `${{ secrets.NAME }}`.

صح ولا غلط: (1) غلط: Static files بس. · (2) غلط: لازم مسار نسبي. · (3) صح.

التحديات:

1. تجربة.
2. تجربة.
3. تجربة: الـ PR هيظهر فيه ✗ لحد ما الاختبار يعدّي.

الفصل 18: Git والـ AI

1. عشان أي تغيير بعد كده هو بتاع الـ AI بس، فـ `diff` يعرضه لوحده و `restore` يرجّع بالظبط.
2. `git add -p` للي عاوزه ثم Commit، وبعدين `git restore .` للباقي.
3. `.env` و `.env.*` و `!.env.example`.
4. الغيه واعمل واحد جديد، وحط الجديد في env.، وشيله من الكود و Commit، ولو ينفع نضّف التاريخ.

صح ولا غلط: (1) غلط: لسه في التاريخ. · (2) غلط: أسماء بس. · (3) صح.

التحديات:

1. تجربة.
2. تجربة.
3. `git rebase -i` و `edit` على الـ Commit، شيل المفتاح، `git commit --amend`، و `git rebase --continue`، وبعدين `git log -p | grep KEY` فاضي.

الفصل 19: الإنقاذ

1. `git branch feature/x` ثم `git reset --hard origin/main` (أو `HEAD~2`) ثم `git switch feature/x`.
2. اللي اتعمله push: `revert` عشان التاريخ المتشارك ميتغيرش. المحلي: `reset` أو `amend` عادي.
3. تاريخين ملهمش Commit مشترك (README على GitHub و init عندك). ابدأ بـ clone أو Repo فاضي.
4. `git reflog` ودوّر على الـ SHA، ثم `git branch name <sha>`.

صح ولا غلط: (1) غلط: Git عمره ما شافها. · (2) صح. · (3) غلط: لازم يتشال من الـ Commits نفسها.

التحديات:

1. تجربة.
2. تجربة.
3. `git reflog` ودوّر على السطر اللي كان HEAD فيه على الـ Branch.

تاريخك بقى في أمان

أحسن طريقة تثبّت بيها اللي اتعلمته: كل مشروع جديد يبدأ بـ `git init`، وكل خطوة شغالة تبقى Commit، وكل تجربة تبقى Branch. ولما حاجة تبوظ، افتكر إن اللي اتعمله Commit مبيضيعش، وارجع لفصل الإنقاذ ولـ Cheat Sheet.

```
$ git commit -m "feat: finish the book"
```